



Lecture 14: Analysis by Intrinsic

Li Yi

May 29, 2025

Recap

- Introduction
- Rigid transformation estimation
 - Closed-form solution given correspondence
 - Iterative closet point (ICP)
- Learning-based approaches
 - Direct approaches
 - Indirect approaches

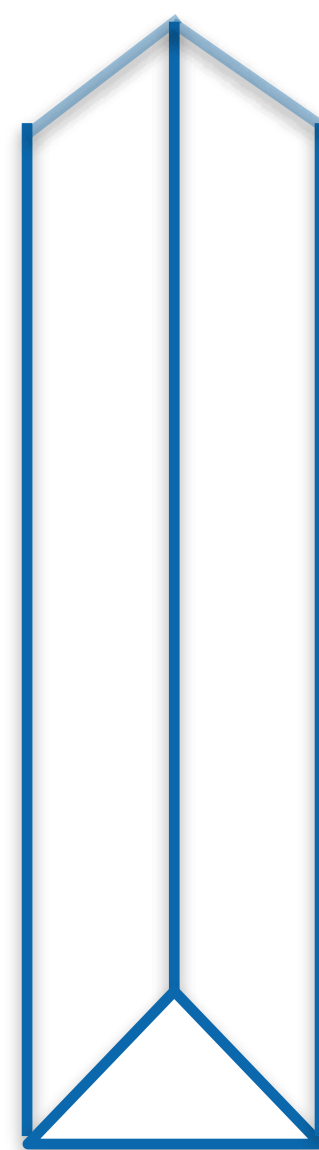
Loss for Direct Approaches

- shape-agnostic: distance between (R, t) and (R_{GT}, t_{GT})
- shape-aware: distance between the same shape $X \in \mathbb{R}^{3 \times n}$ transformed by (R, t) and (R_{GT}, t_{GT}) respectively

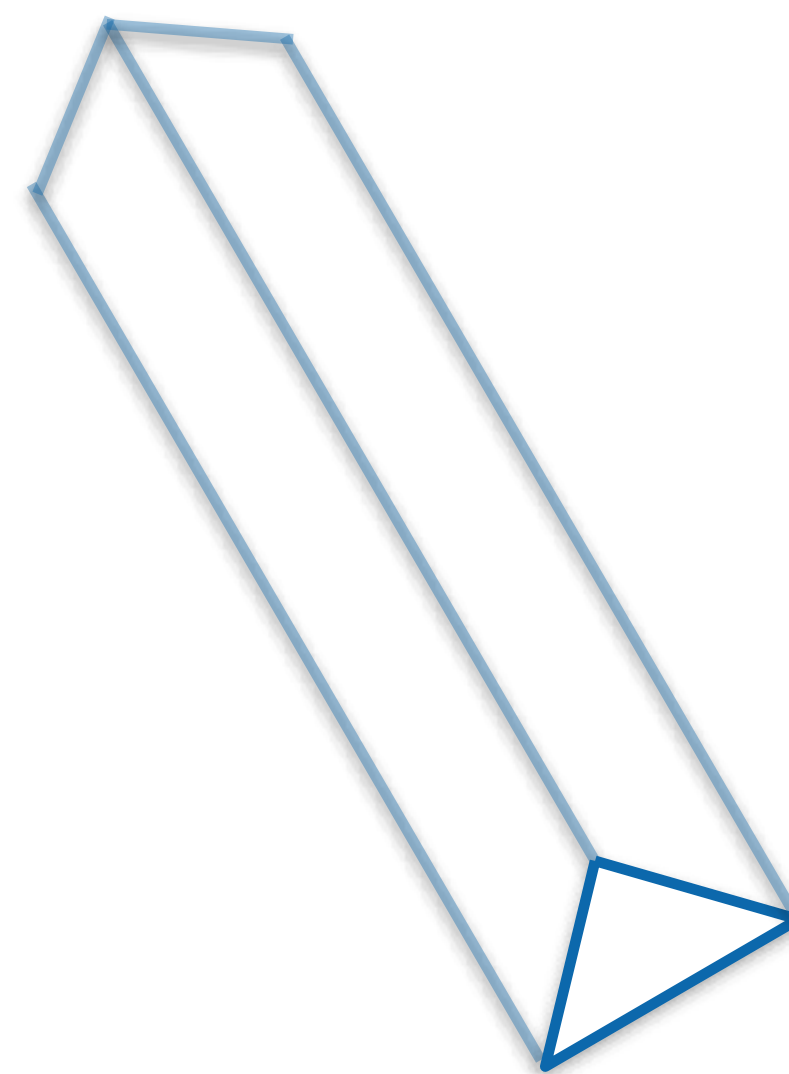
Shape-agnostic Loss

- Rotation
 - Geodesic distance (angle difference, relative rotation error) on $SO(3)$: $\arccos[\frac{1}{2}(\text{tr}(R_{GT}R^T) - 1)]$
 - Mean square error over $\mathbb{R}^{3 \times 3}$: $\|R - R_{GT}\|_F^2$
 - L_p distance on representation x : $\|x - x_{GT}\|_p$
- Translation
 - L_p distance: $\|t - t_{GT}\|_p$

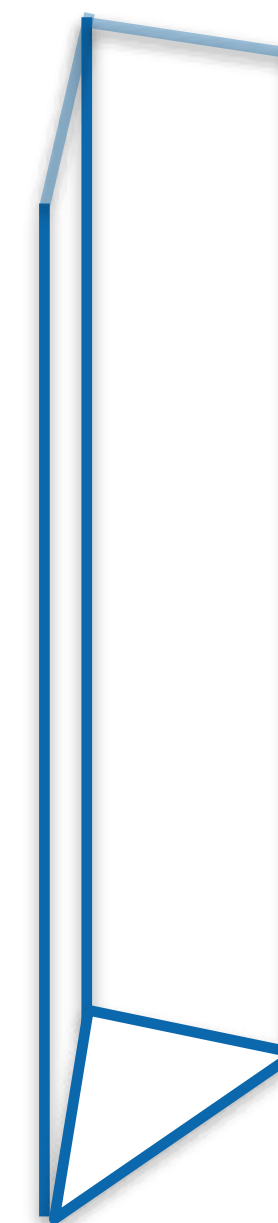
Motivation of Shape-aware Loss



target pose



predicted pose1



predicted pose2

Similar angle difference, but very different perception error

Shape-aware Loss

- Given a shape $X \in \mathbb{R}^{3 \times n}$, the loss for rotation can be the distance between RX and $R_{GT}X$
- e.g., $L = \|RX - R_{GT}X\|_F^2$ (per-point Mean Square Estimation (MSE))

Shape-aware Loss for Symmetry Objects

- With $\|RX - R_{GT}X\|_F^2$ as the distance metric, we can also apply the *Min of N* loss

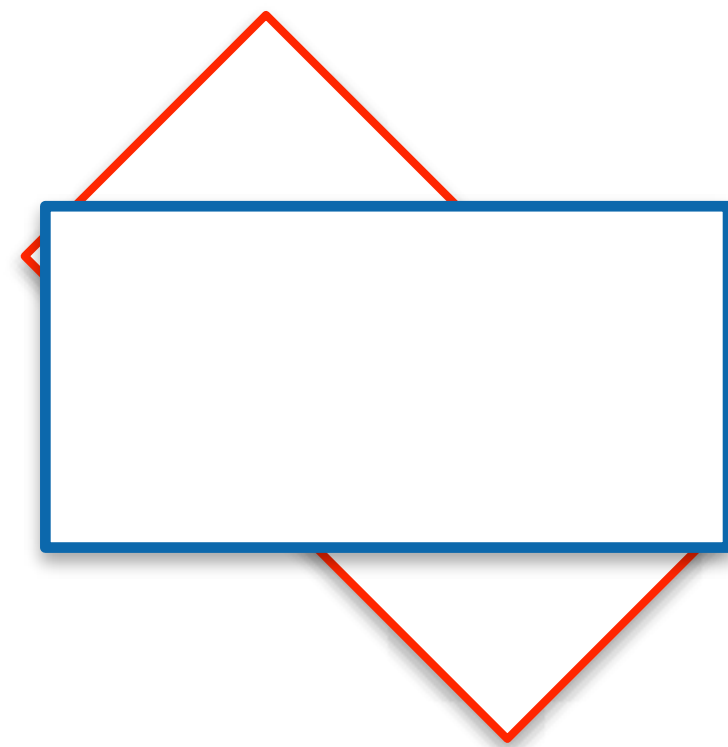
$$L = \min_{R_{GT}^i \in \mathcal{R}} \|RX - R_{GT}^i X\|_F^2$$

Shape-aware Loss for Symmetry Objects

- Recall the distance metrics for point clouds
 - Chamfer distance
 - earth mover distance
- Those distance metrics are compatible with symmetry (even infinite order)
- But requires a shape and might get stuck in local minima

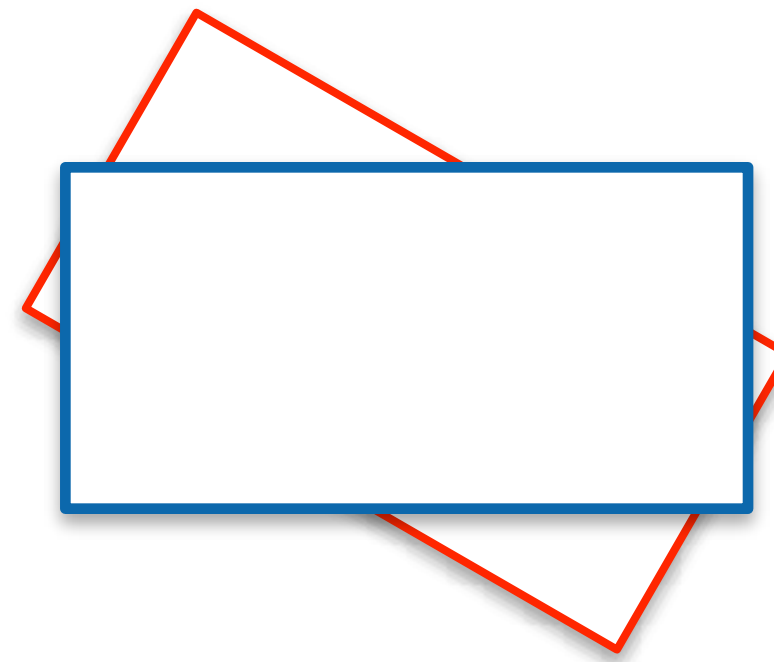
Example of Local Minima for Chamfer Distance

Rotate the red left a bit?



CD=2.02

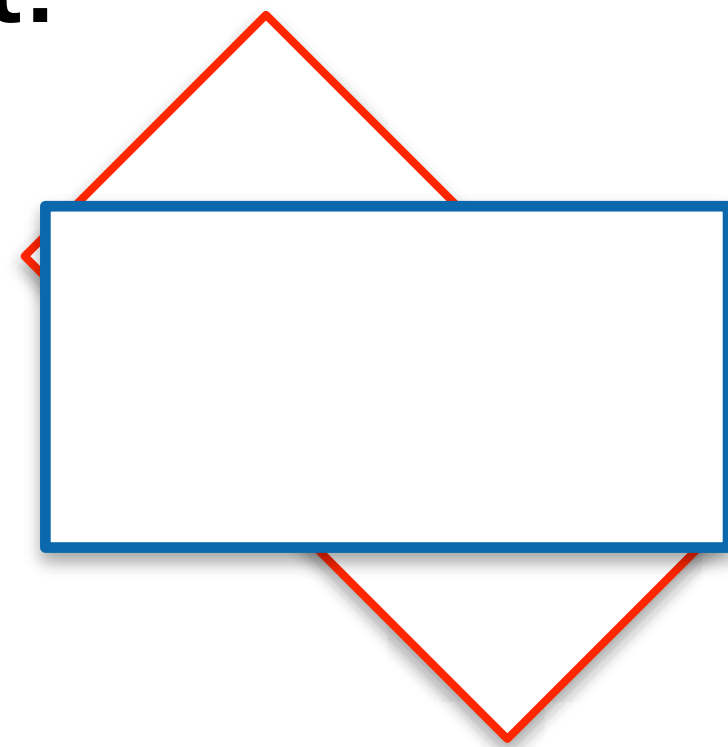
Example of Local Minima for Chamfer Distance



CD=2.05

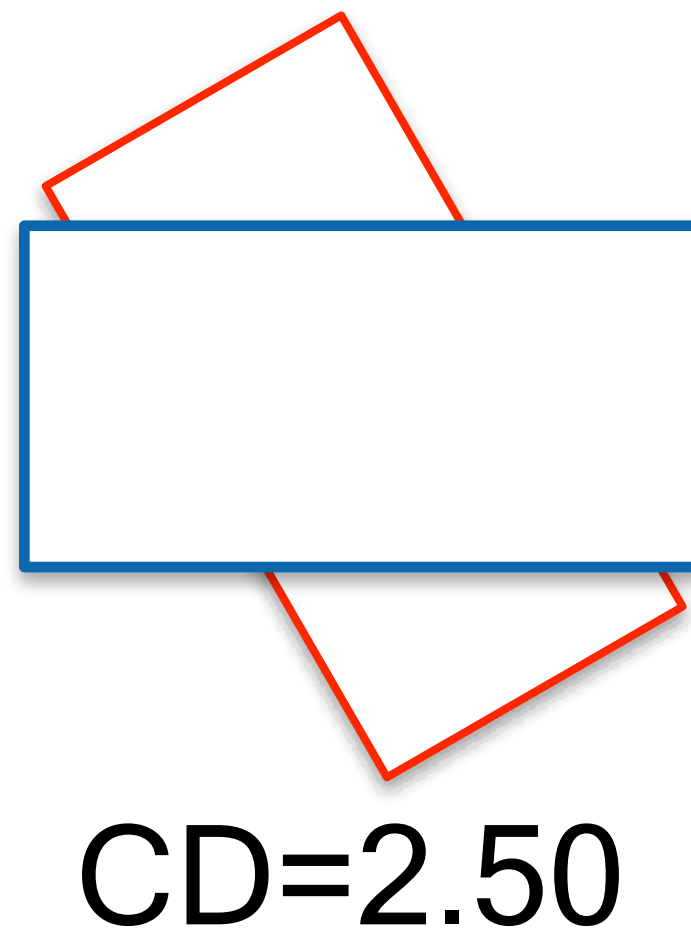
Example of Local Minima for Chamfer Distance

Reset, and try rotate right:

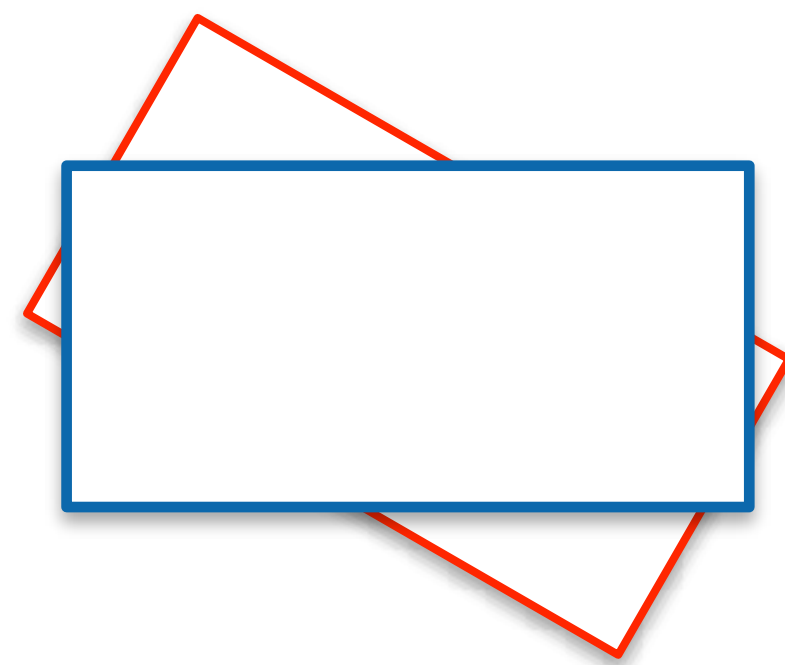


CD=2.02

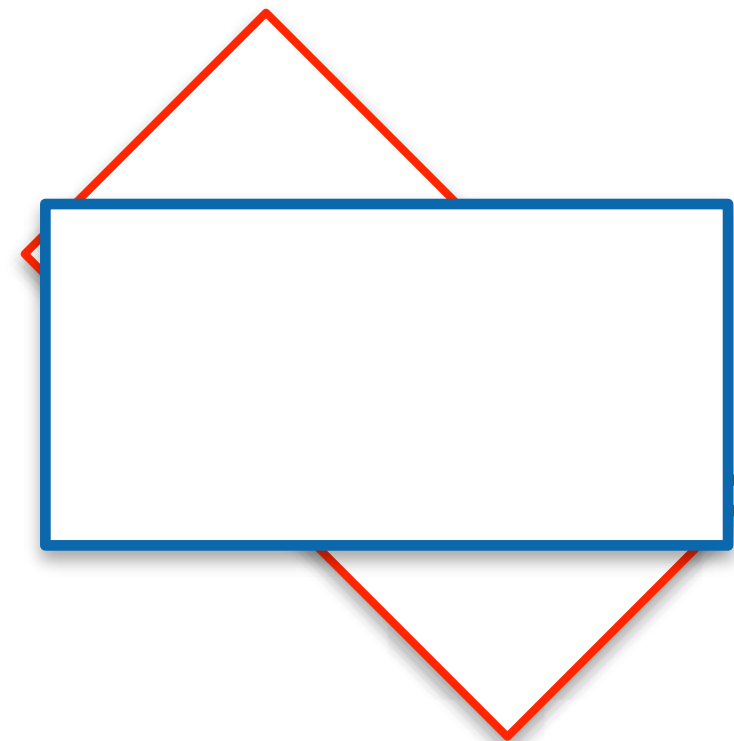
Example of Local Minima for Chamfer Distance



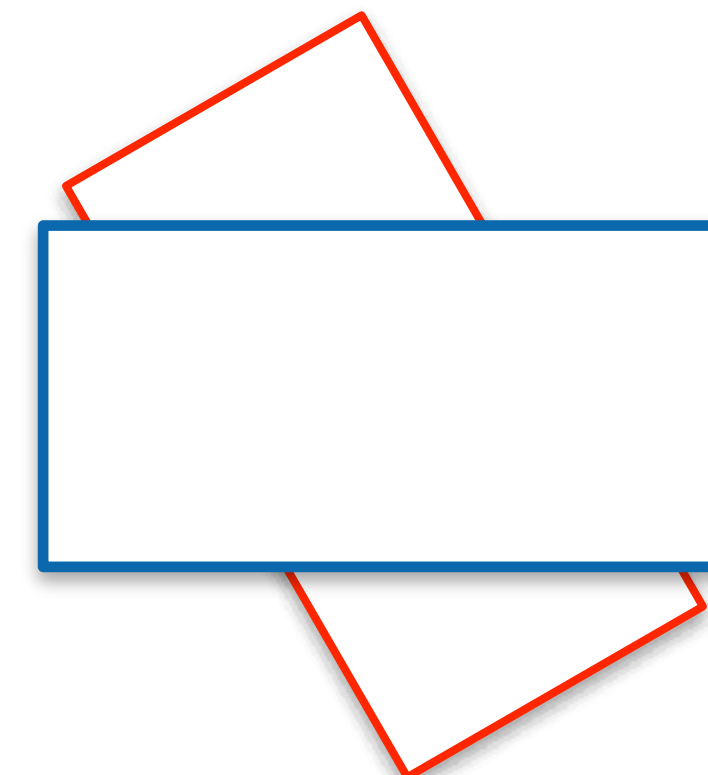
Example of Local Minima for Chamfer Distance



CD=2.05



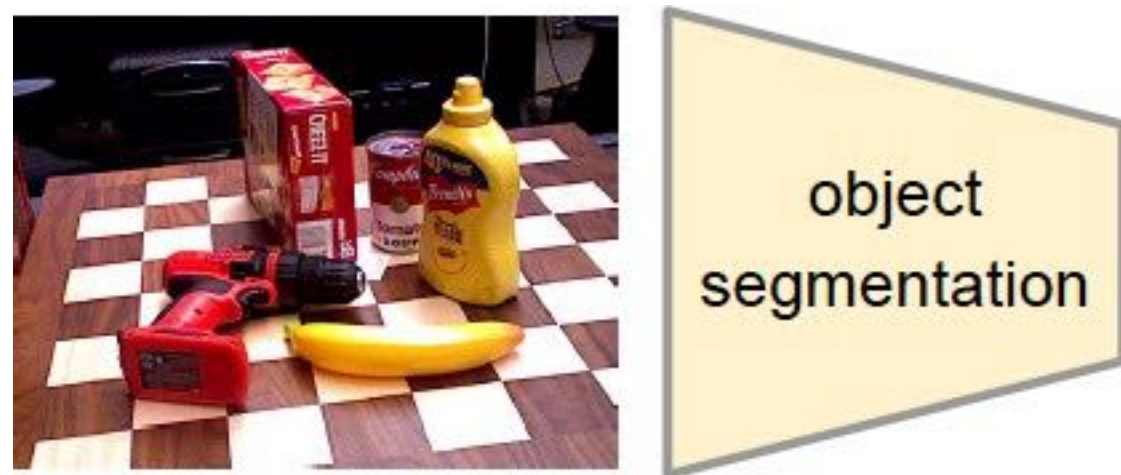
CD=2.02



CD=2.50

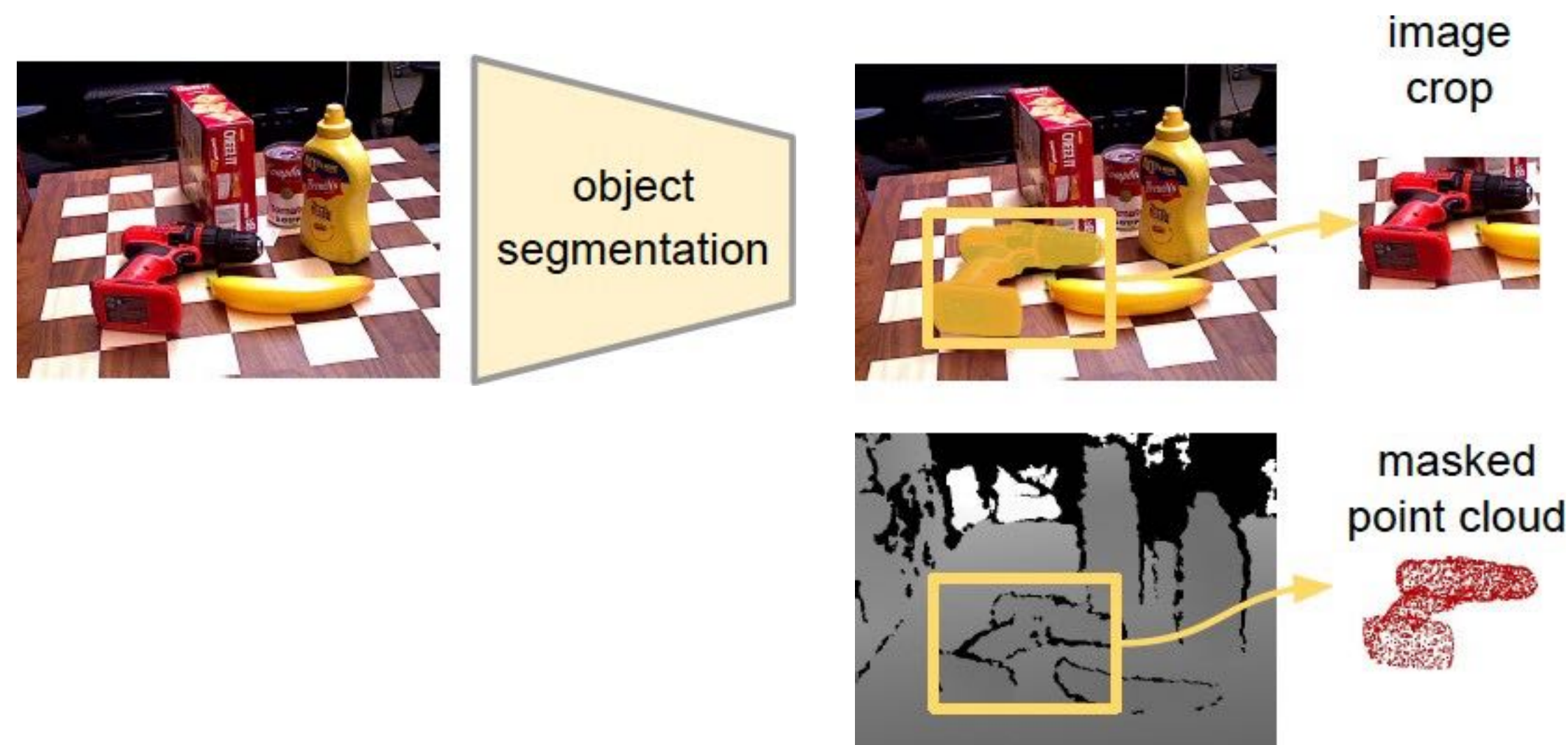
Note: No local minima for Min of N

Example: DenseFusion



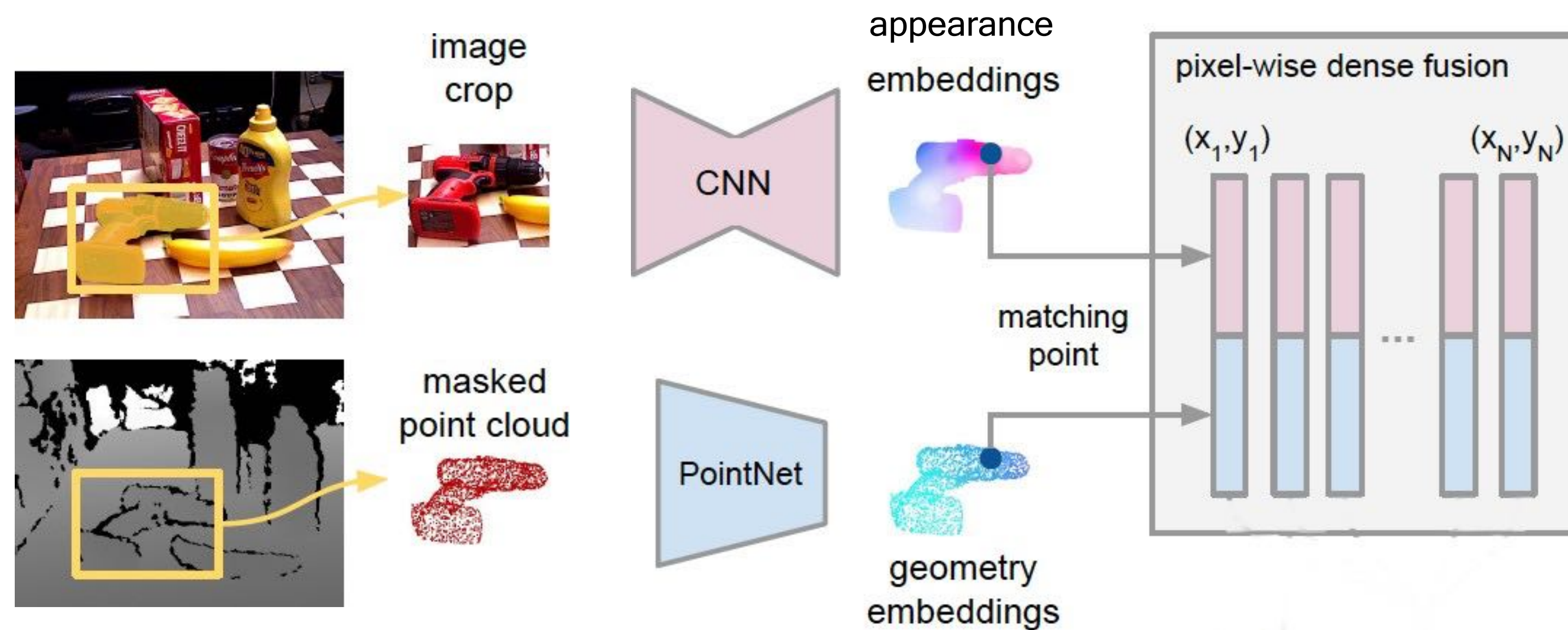
Segment objects

Example: DenseFusion



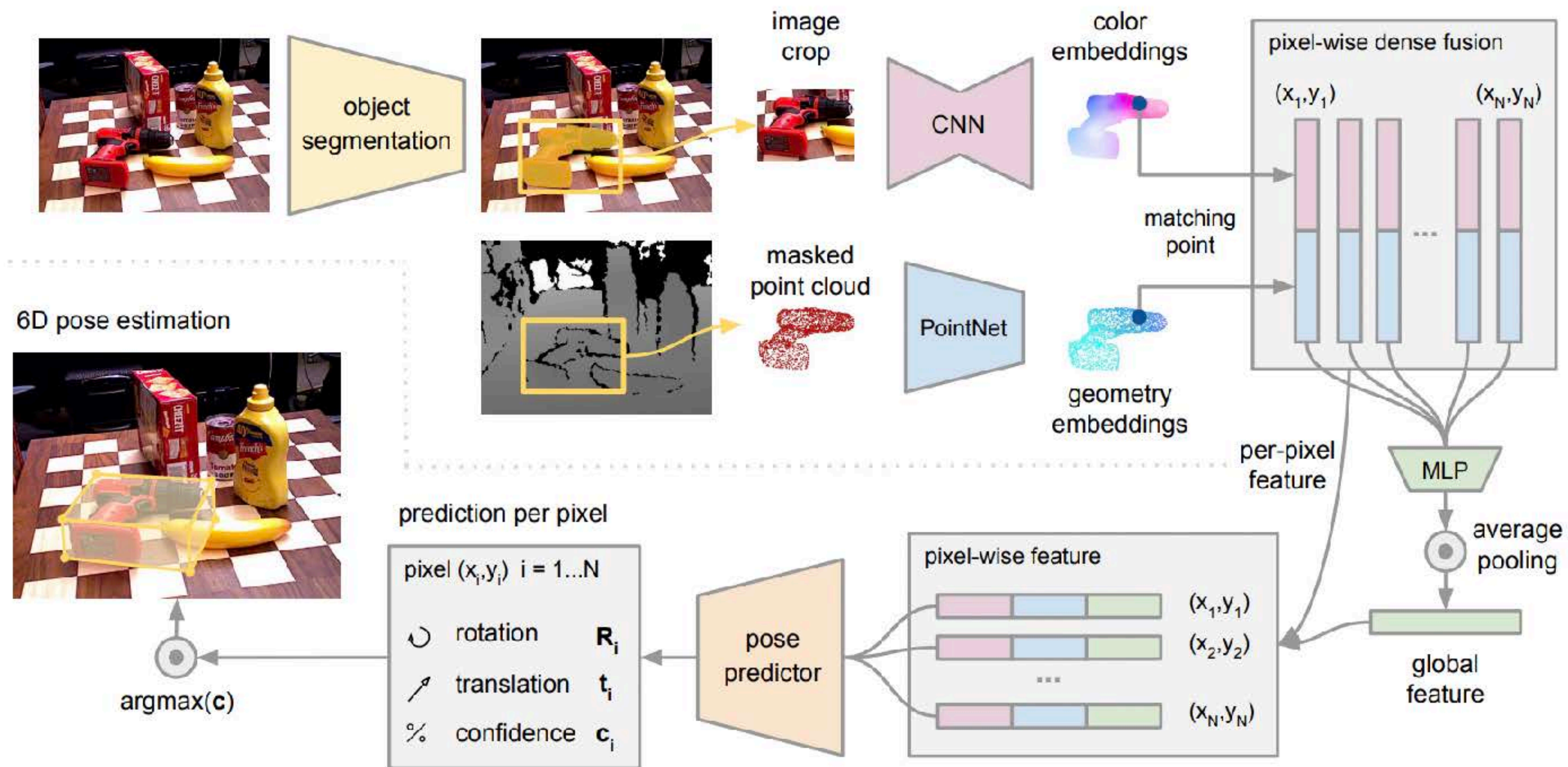
Get the image crop and the point cloud for an object

Example: DenseFusion



Encode appearance (image) with CNN
and geometry (point cloud) with PointNet

Example: DenseFusion



Predict poses from features

Example: DenseFusion

6D pose estimation



non-symmetric objects:
per-point MSE

symmetric objects:
chamfer distance

Min-of-N as alternative?

Compute shape-aware loss

Outline

- Introduction
- Rigid transformation estimation
 - Closed-form solution given correspondence
 - Iterative closet point (ICP)
- Learning-based approaches
 - Direct approaches
 - Indirect approaches

Indirect Approaches

- Input: cropped image/point cloud/depth map of a single object
- Output: corresponding pairs $\{(p_i, q_i)\}$
 - points in canonical frame $\{p_i\}$
 - points in camera frame $\{q_i\}$
 - estimate (R, t) by solving

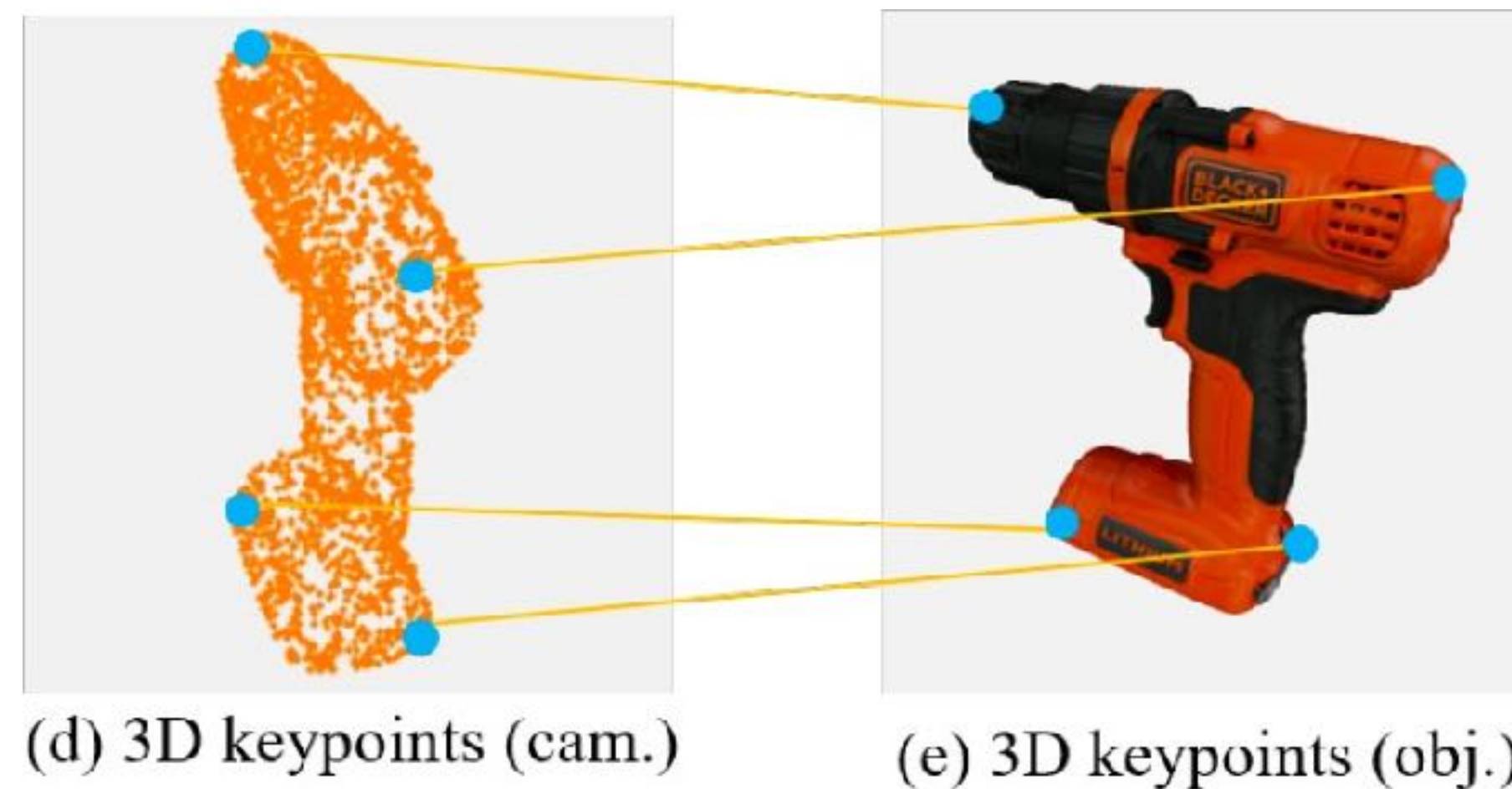
$$\min_{R, t} \sum_{i=1}^n \|Rp_i + t - q_i\|^2$$

Two Categories of Indirect Approaches

- If points in the **canonical frame** are known, predict their corresponding locations in the **camera frame**
- If points in the **camera frame** are known, predict their corresponding locations in the **canonical frame**

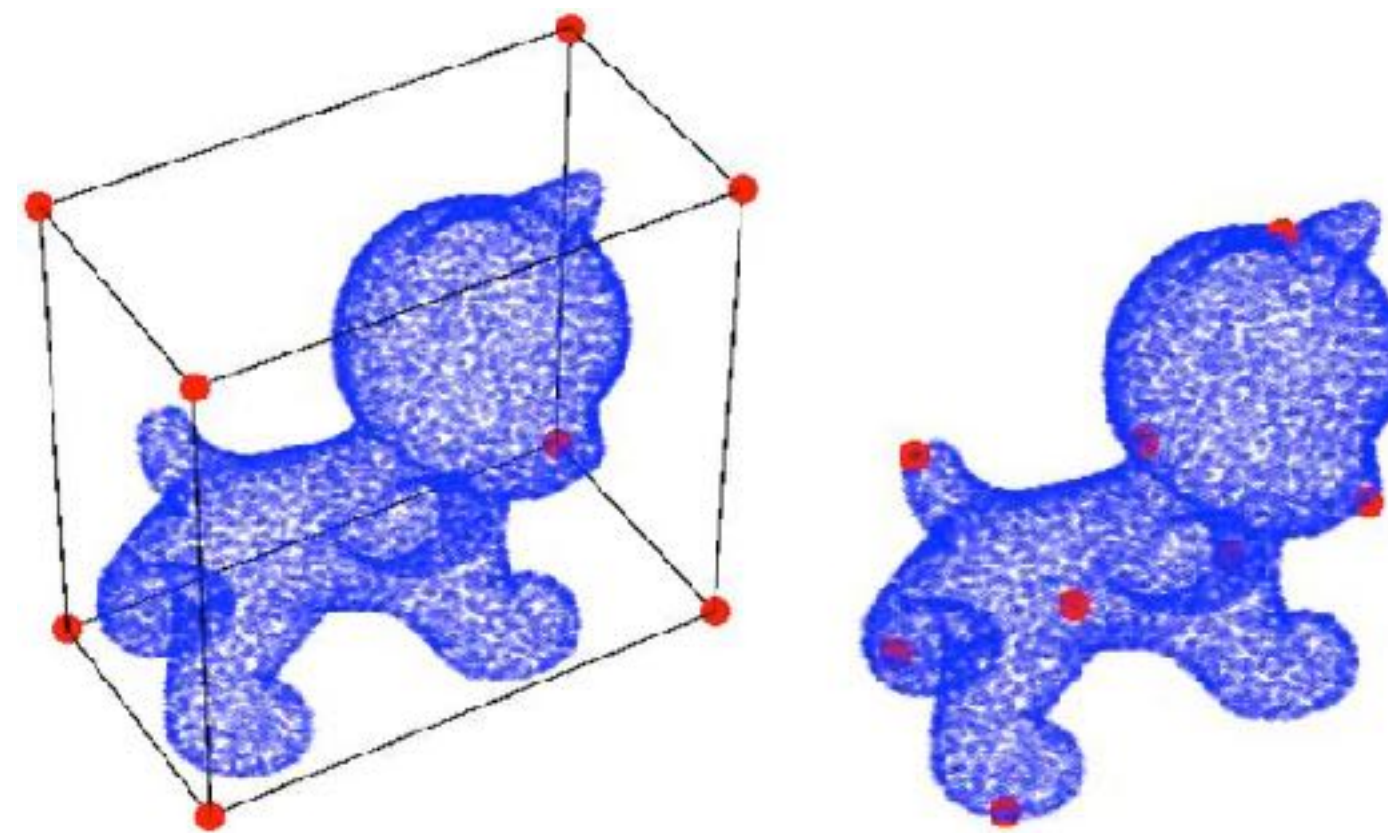
Given Points in the Canonical Frame, Predict Corresponding Location in the Camera Frame

- Recall: Three correspondences are enough
- Which points in the canonical frame should be given?
 - Choice by PVN3D: keypoints in the canonical frame

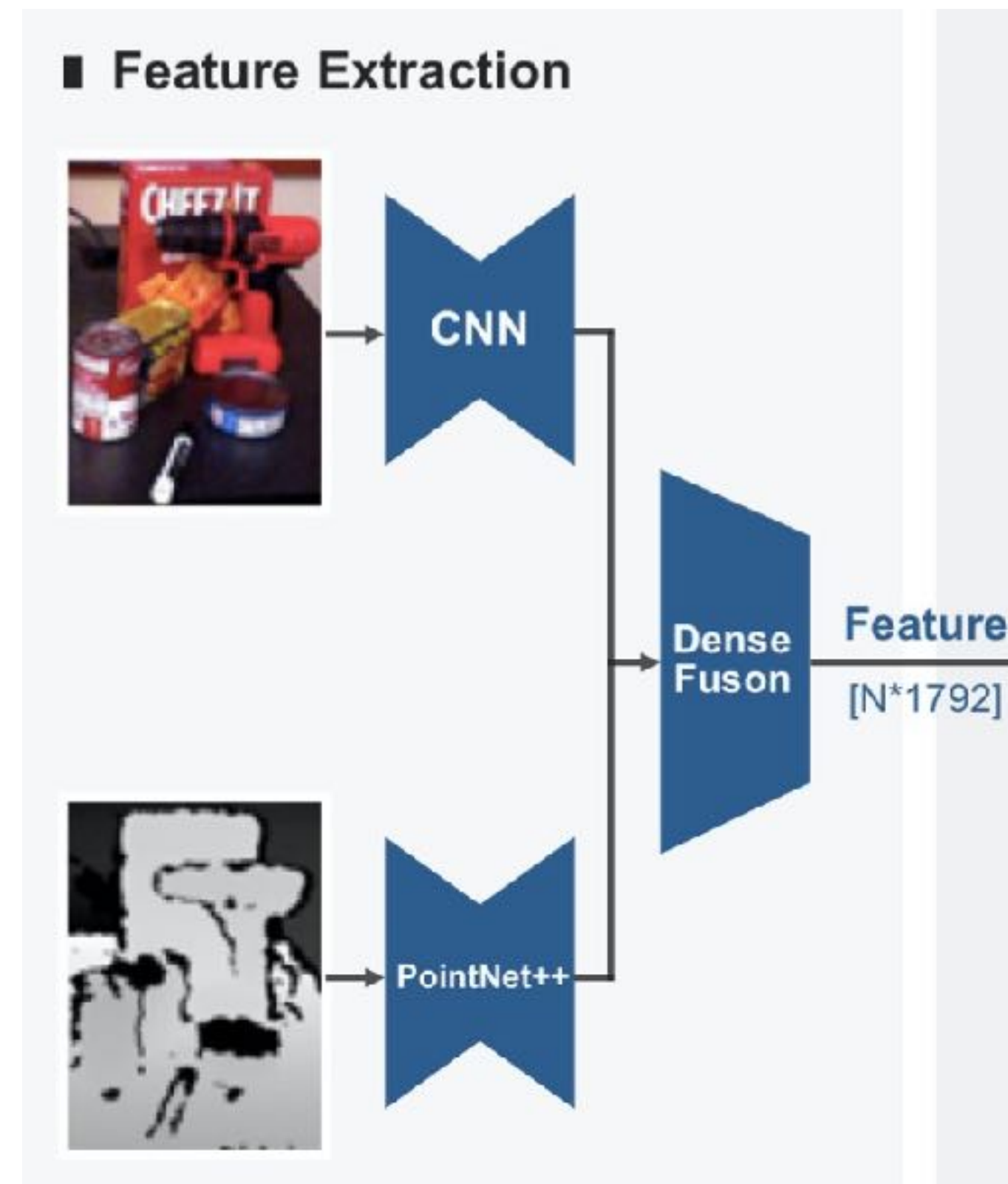


Keypoint Selection

- Option1: bounding box vertices
- Option 2: farthest point sampling (FPS) over CAD object model

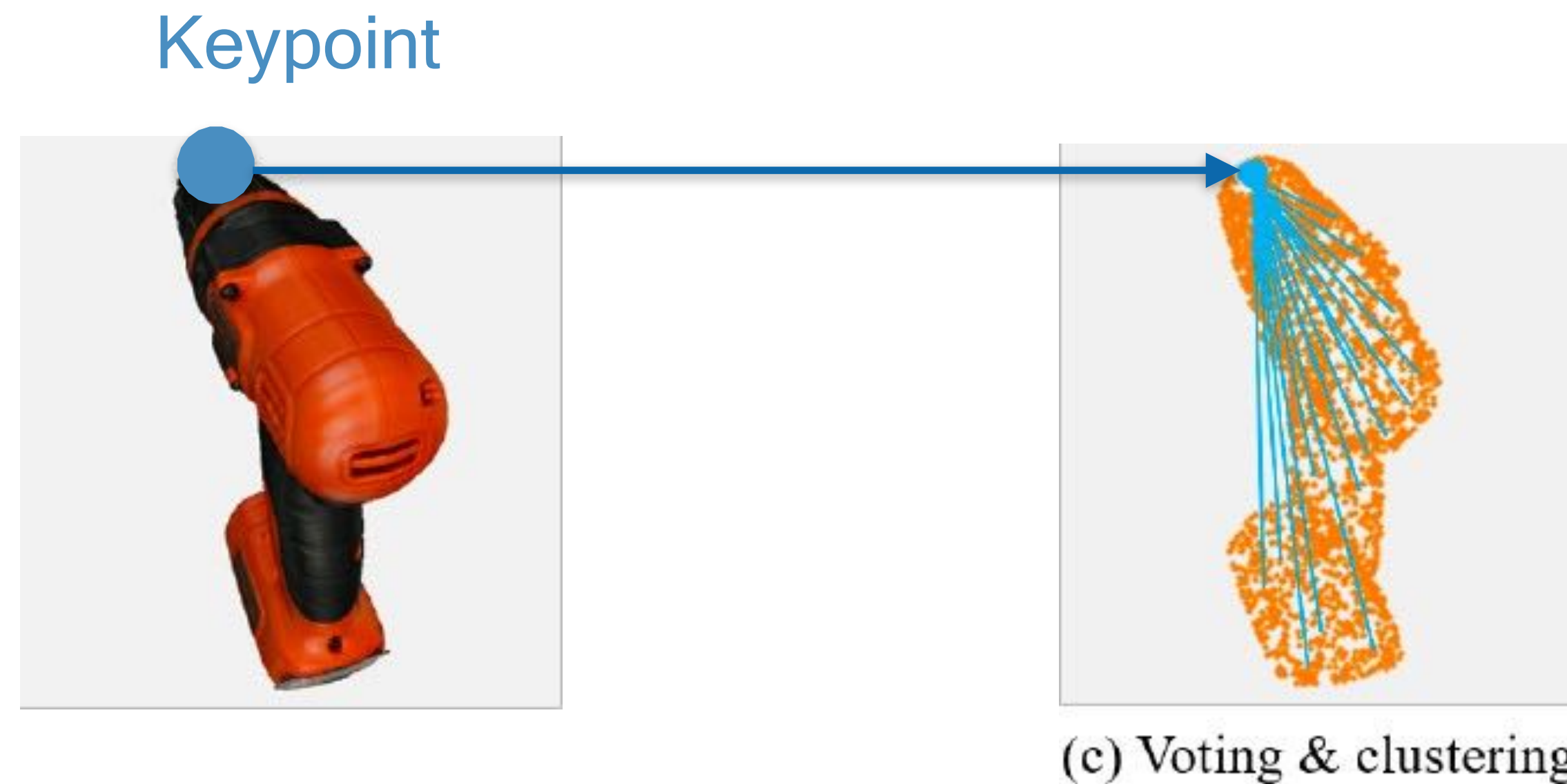


Example: PVN3D



Get point-wise features
by fusing color and geometry features

Example: PVN3D

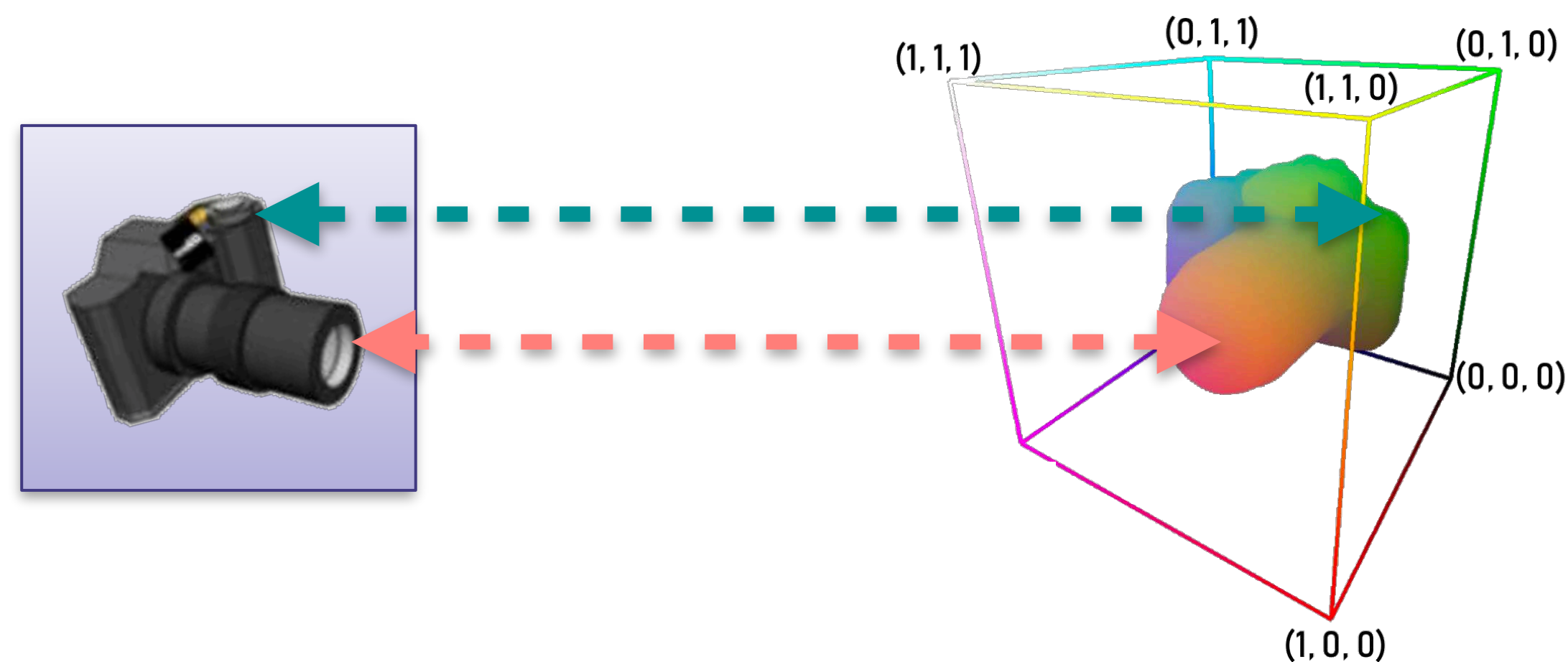


For each keypoint:

- **Voting:** for each point in the camera frame, predict its offset to the keypoint (in the camera frame)
- **Clustering:** find one location according to all the candidates

Given Points in the Camera Frame, Predict Corresponding Location in the Canonical Frame

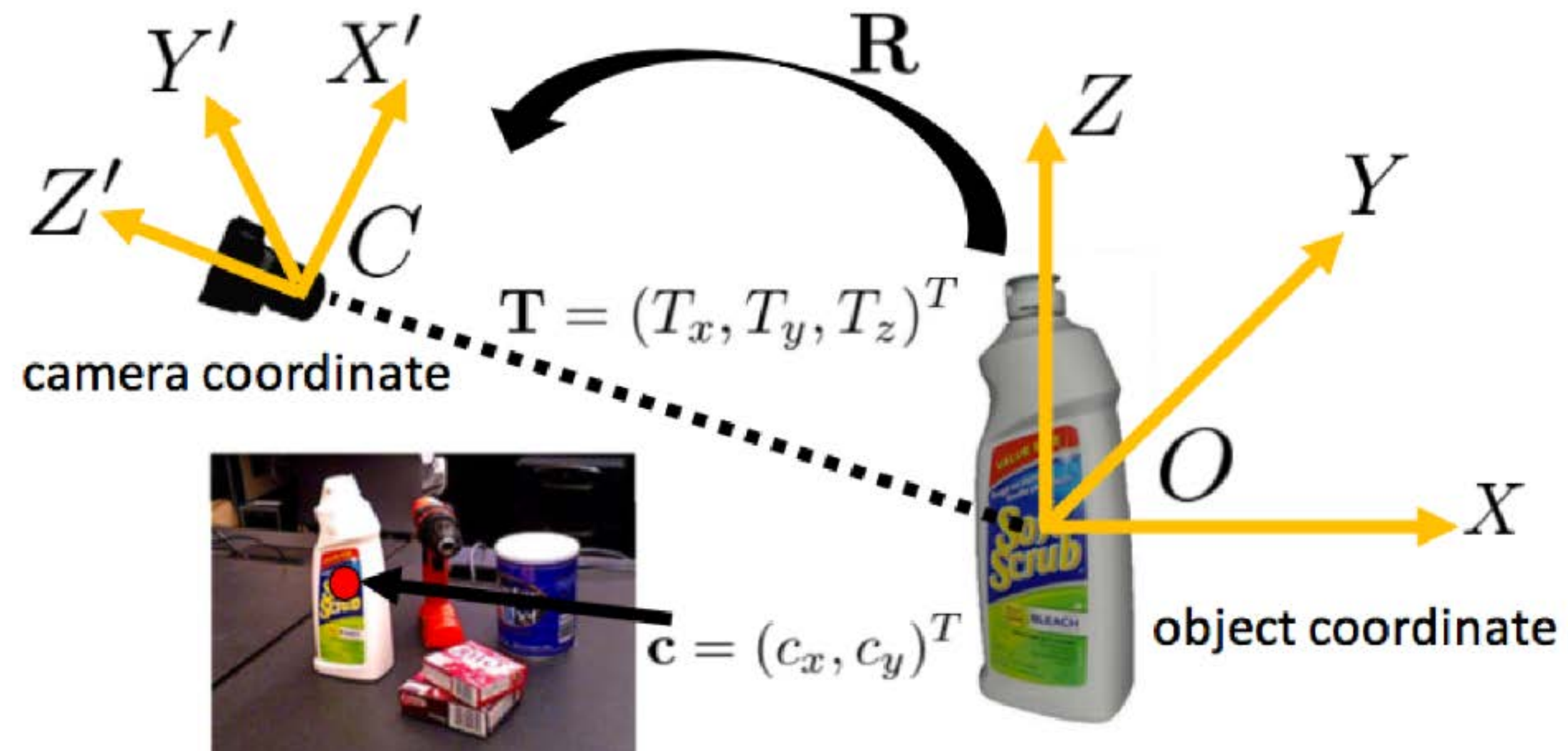
- Which points in the camera frame should be given?
 - Choice by NOCS: every point in the camera frame



3D point in the camera frame
(2D visible pixel with depth)

3D point in the (normalized)
canonical frame

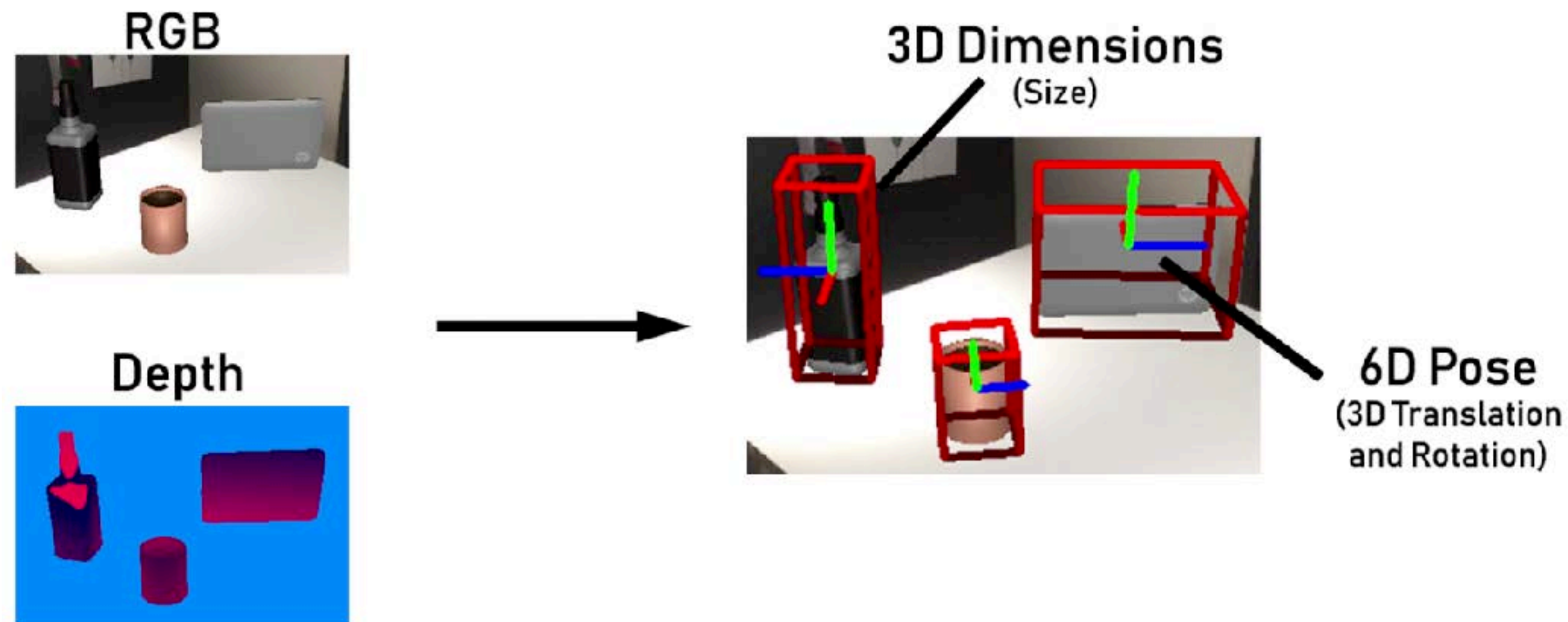
Instance-Level 6D Pose Estimation



- Instance-level: a small set of known instances
- Pose is defined for each instance according to their CAD model.
- Input: RGB/RGBD
- Output: 6D pose(s) of an object or all known objects

Category-Level 6D Pose Estimation

Detecting and estimating **6D pose** and **3D size** of **previously unseen** objects from certain categories from RGBD images.



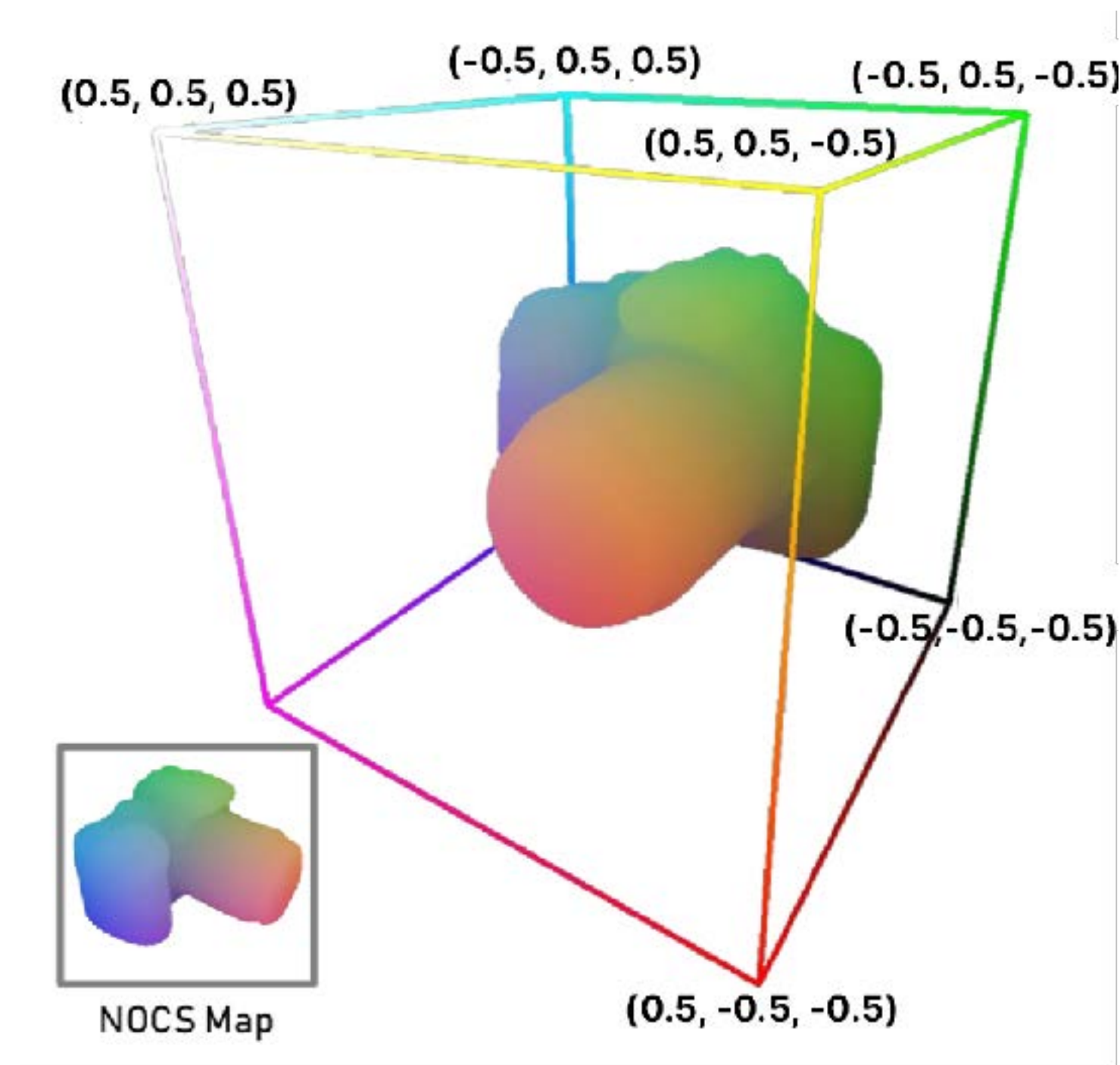
without knowing CAD models

Challenges

- Intra-category shape variations



Representation: Normalized Object Coordinate Space



RGB colors represent XYZ coordinates of shape.

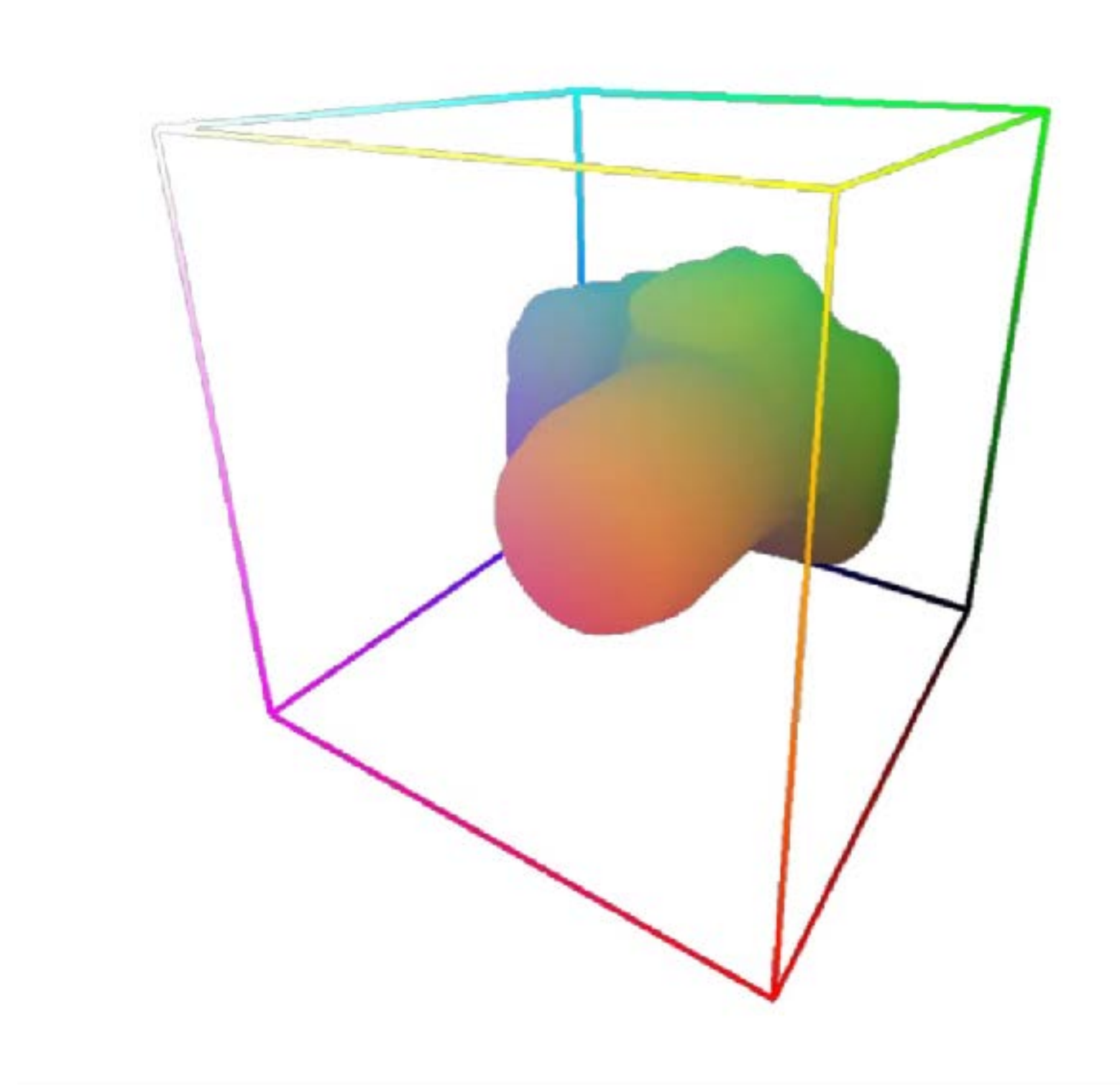
Representation: Normalized Object Coordinate Space

Step 1 (rotation normalization):
align object orientations



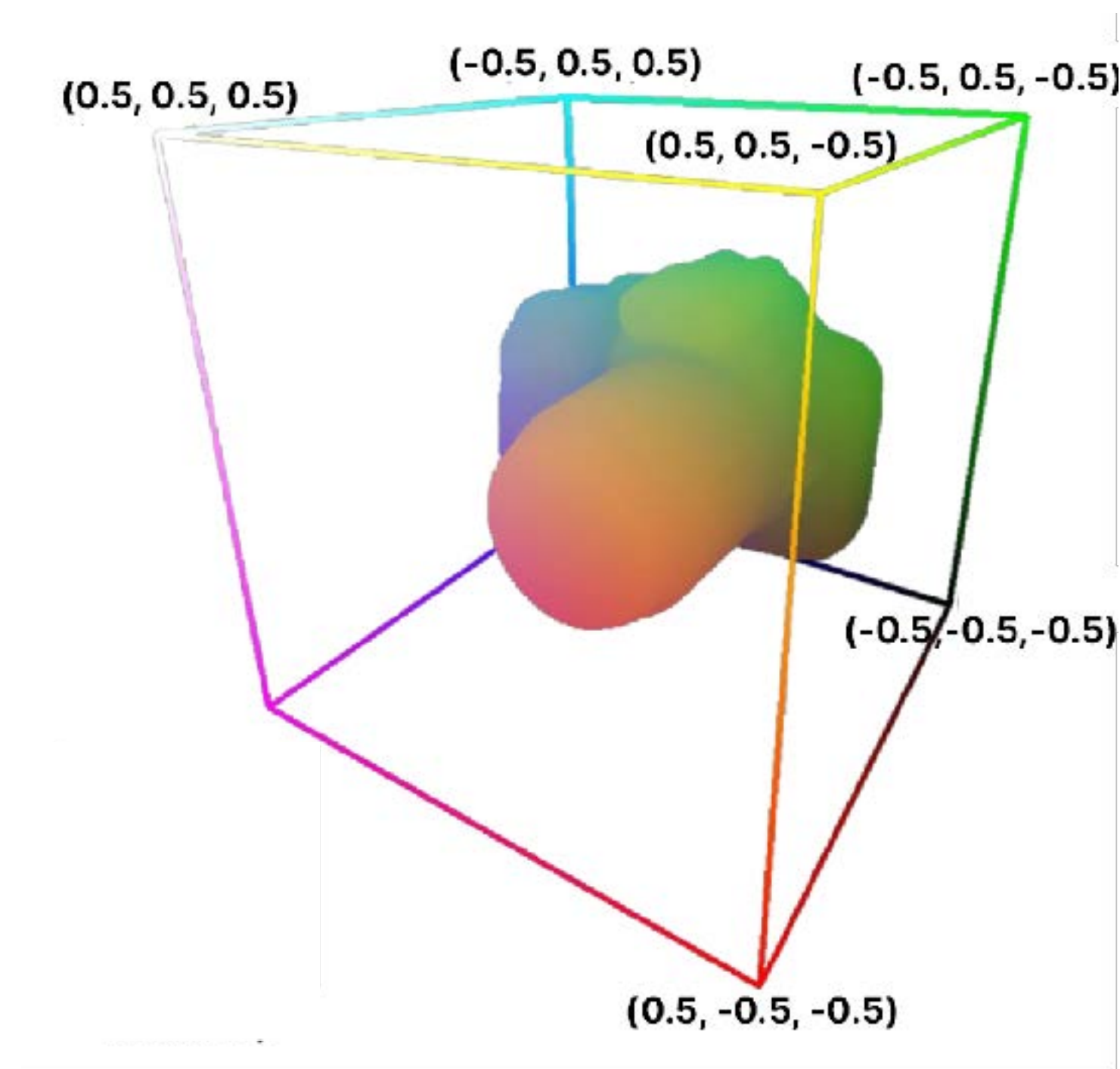
Representation: Normalized Object Coordinate Space

Step 2 (translation normalization):
zero-center the objects



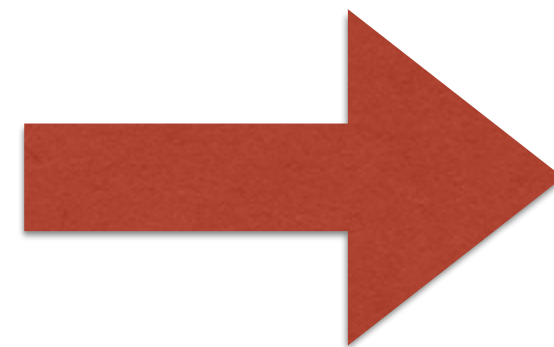
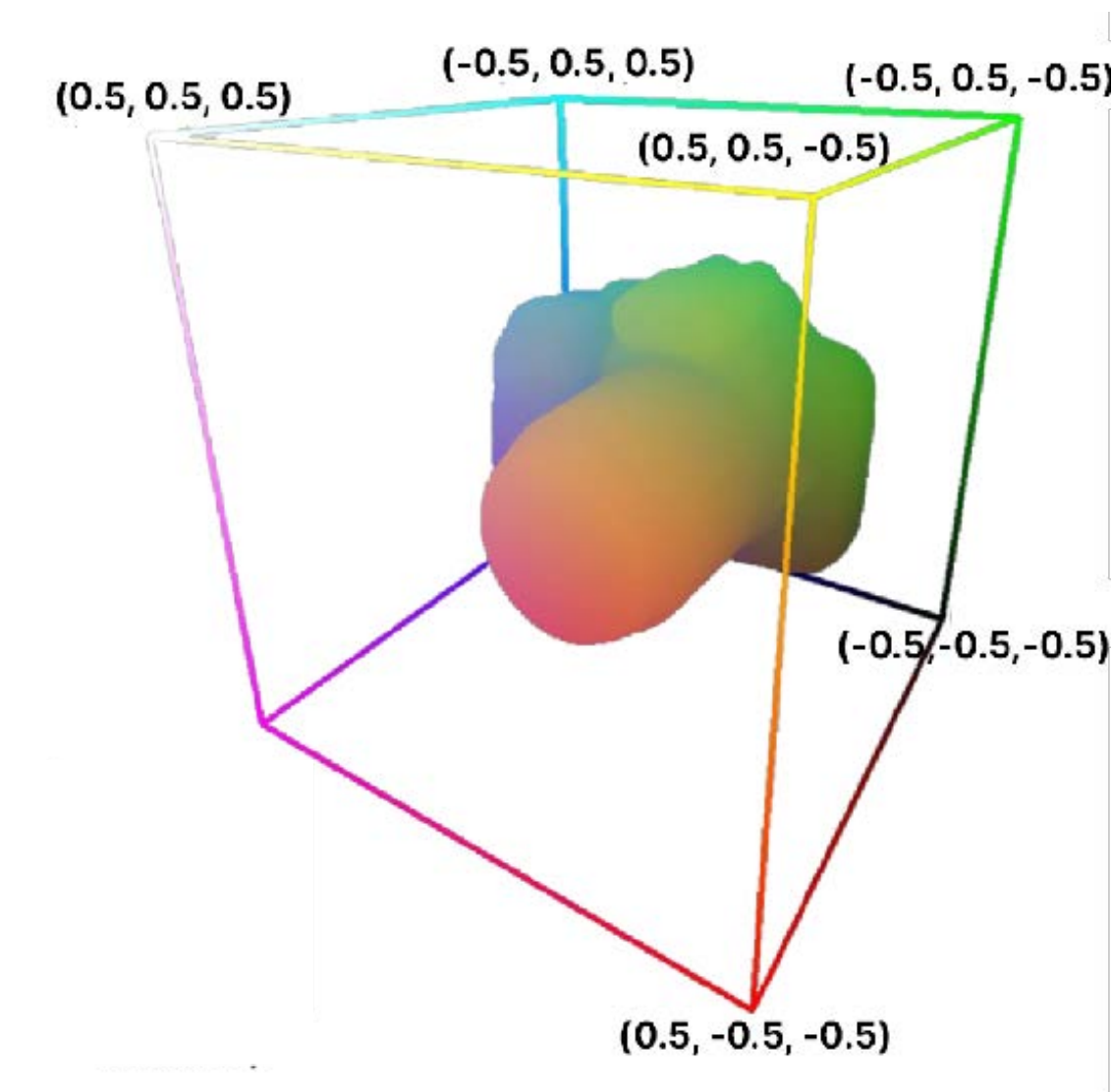
Representation: Normalized Object Coordinate Space

Step 3 (scale normalization):
uniformly normalize the scales

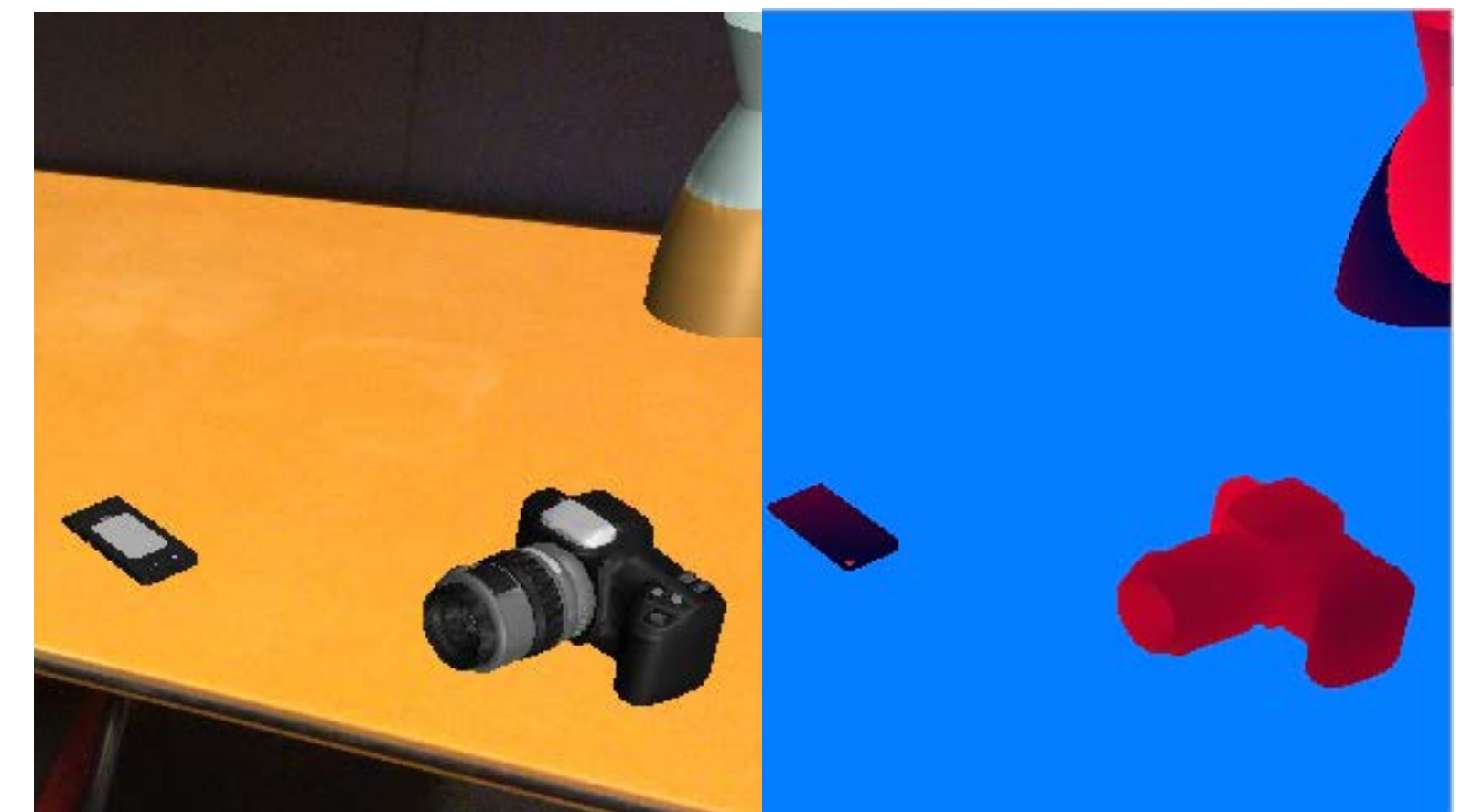


Definition of Category-Level 6D Pose

NOCS = Reference frame



RGBD



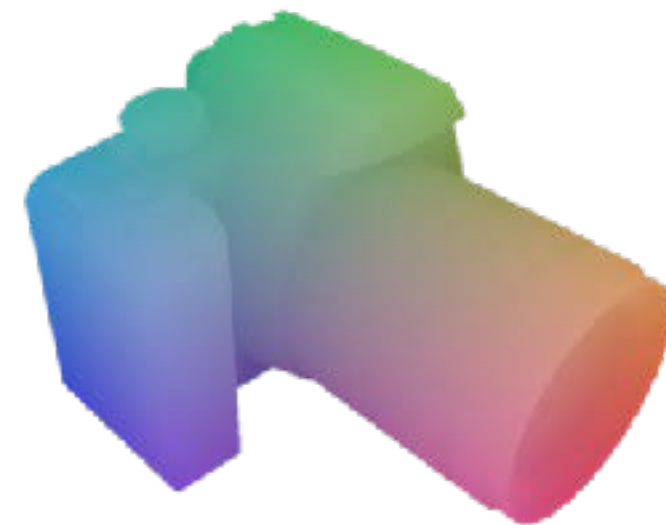
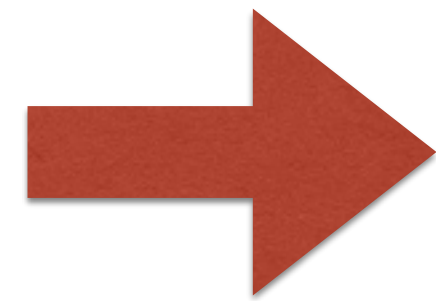
Category-level pose = transformation from NOCS to camera space

From Image to NOCS map to Pose



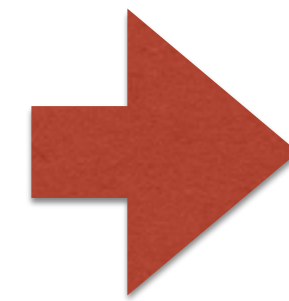
RGB Image

Per-pixel
predict



NOCS map

NOCS to point

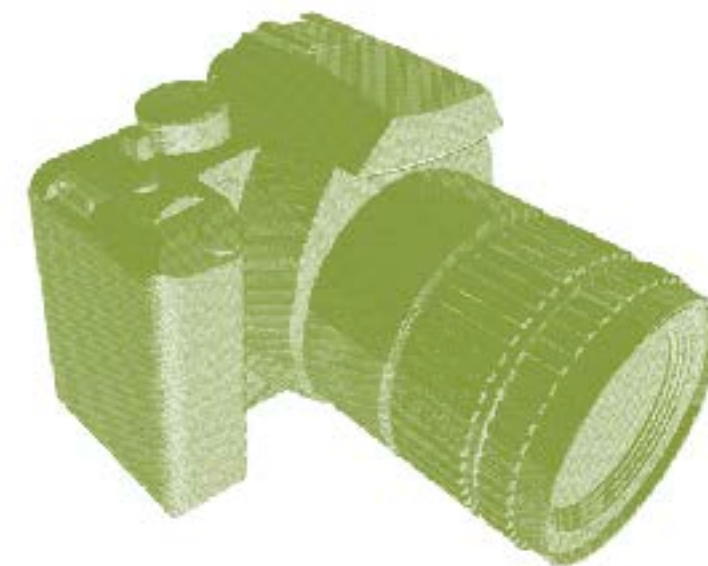
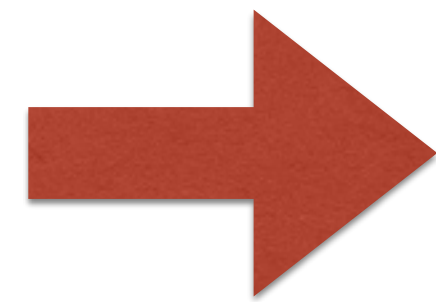


NOCS pts



Depth

Backproject



Depth pts



Pose fitting
w/ RANSAC

NOCS for Symmetric Objects

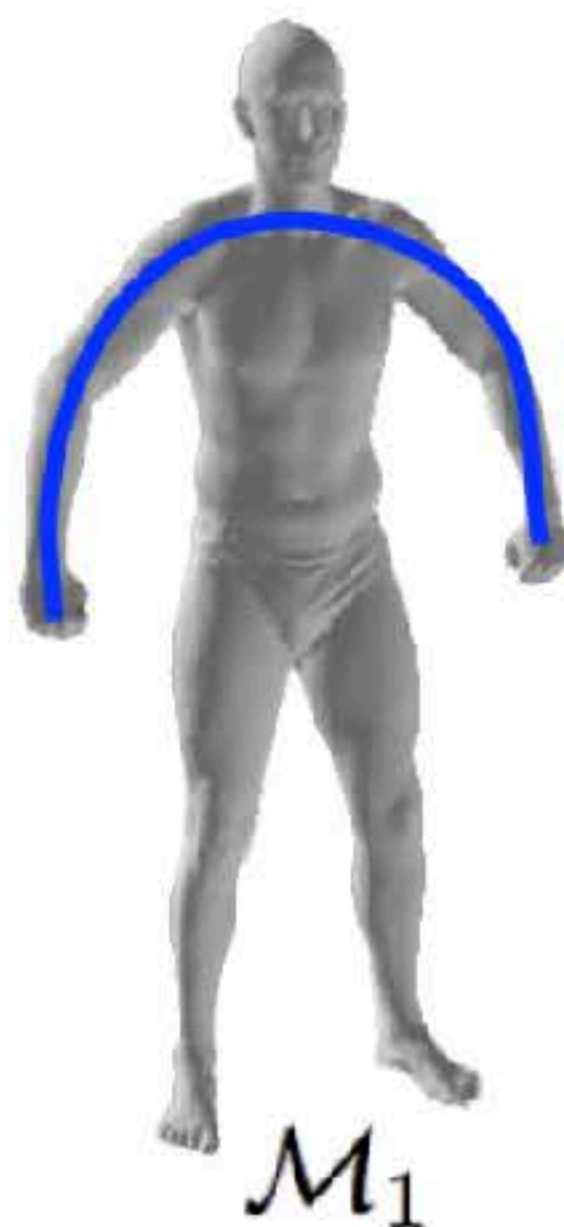
- Given equivalent GT rotations
 $\mathcal{R} = \{R_{GT}^1, R_{GT}^2, \dots, R_{GT}^n\}$ (finite symmetry order n),
we can generate n equivalent NOCS maps
- Similar to shape-agnostic loss in direct approaches,
we can use *Min of N* loss

Umeyama's Algorithm

- However, the target points in the canonical space of NOCS are normalized, and thus we also need to predict the scale factor
- Similarity transformation estimation (rigid transformation + uniform scale factor)
- Closed-form solution
 - Umeyama algorithm: <http://web.stanford.edu/class/cs273/refs/umeyama.pdf>
 - Similar to the counterpart without scale

Today's Focus

- 3D analysis - intrinsic shape understanding



isometry = length-preserving transform



Outline

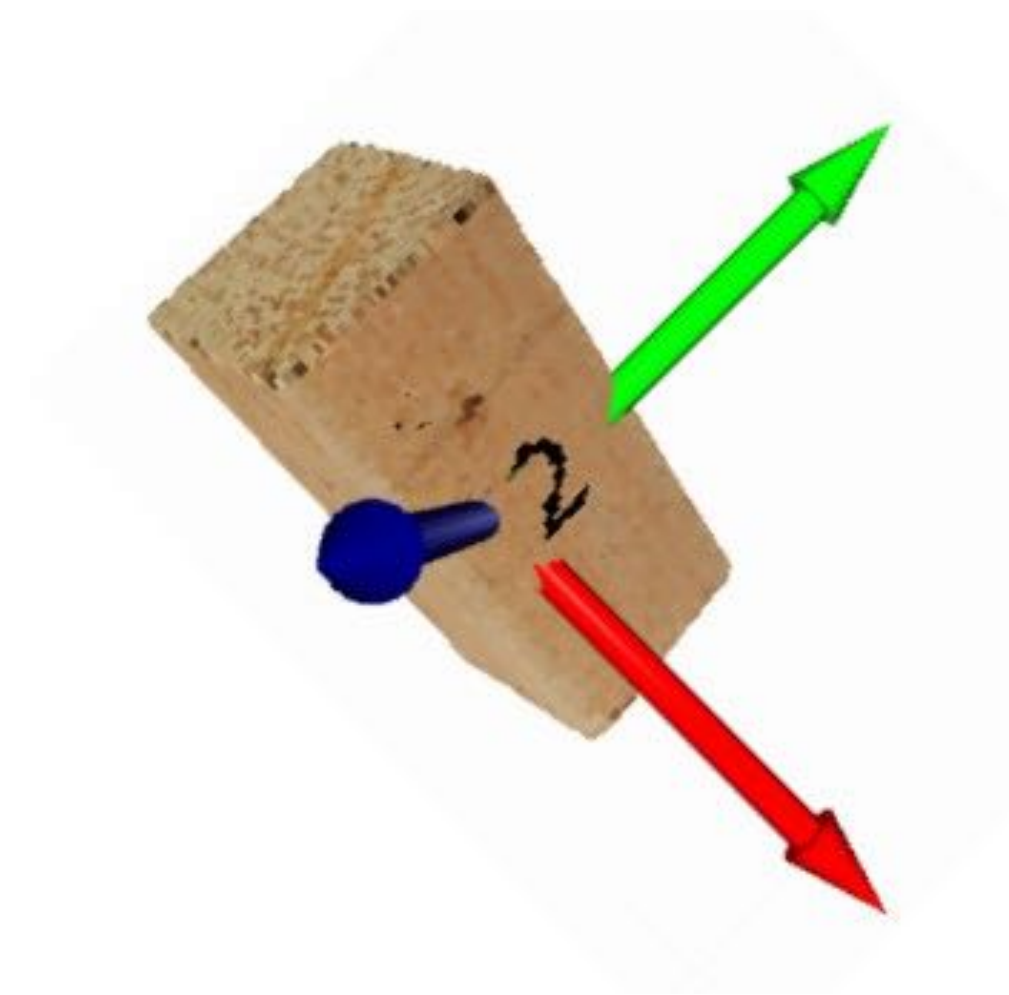
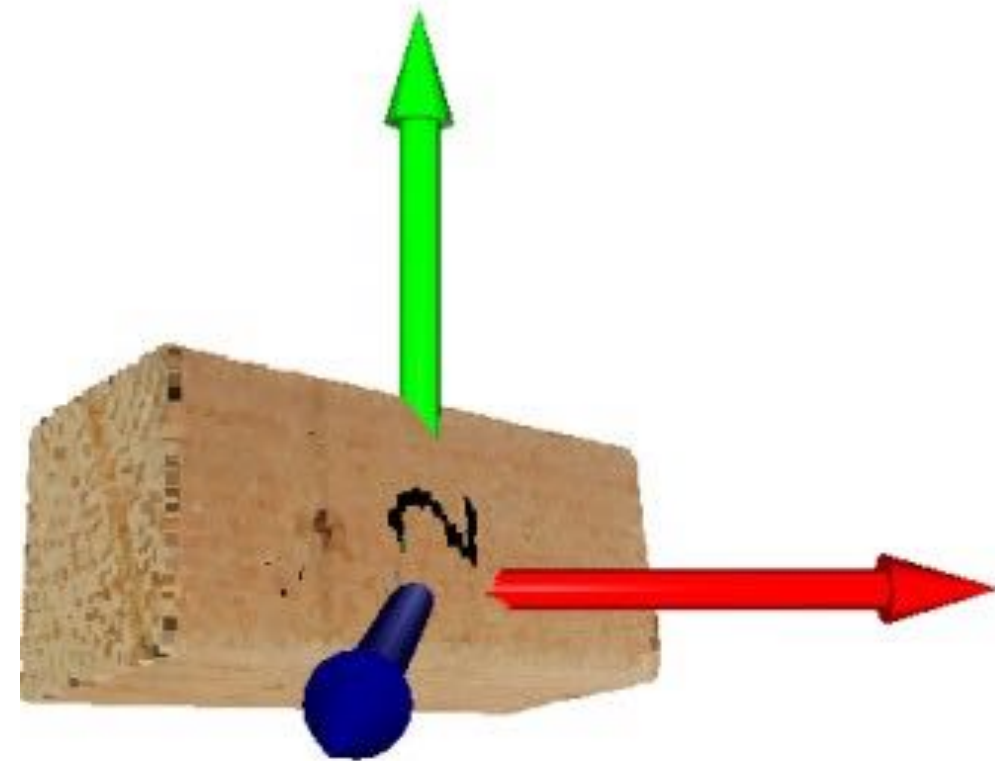
- *Introduction to intrinsic shape analysis*
- Analytic Method for Computing Geodesics
- Learning to Predict Geodesics for Point Cloud
- Applications of Learning Point Cloud Geodesics
 - Normal Estimation
 - Mesh Reconstruction

Extrinsics v.s. Intrinsic

- Extrinsics: cares about the embedding of a surface
 - Looking at the surface **outside** the surface

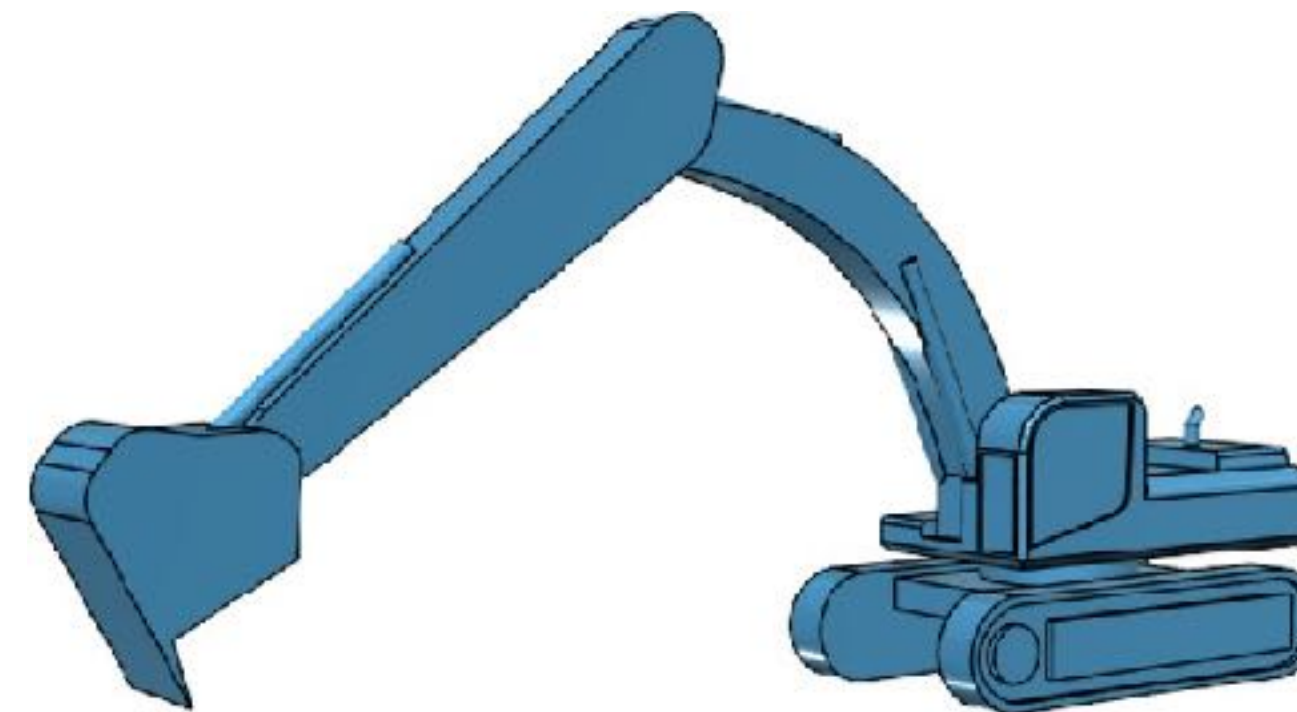
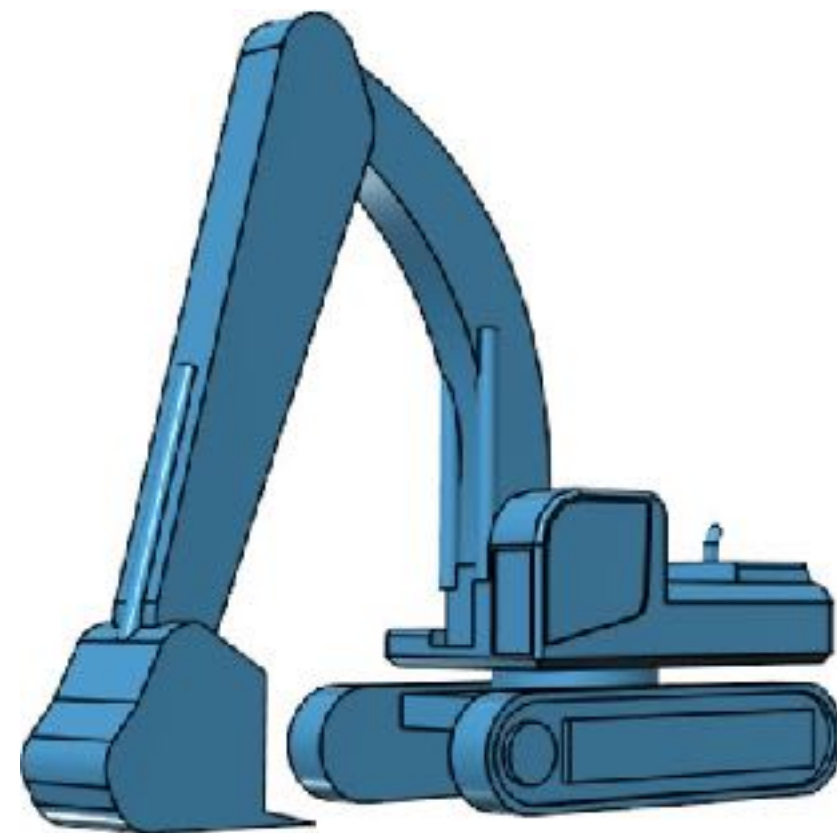
Extrinsics v.s. Intrinsics

- Extrinsics: cares about the embedding of a surface
 - Looking at the surface **outside** the surface
 - For example:
 - est. 6D pose of a rigid body



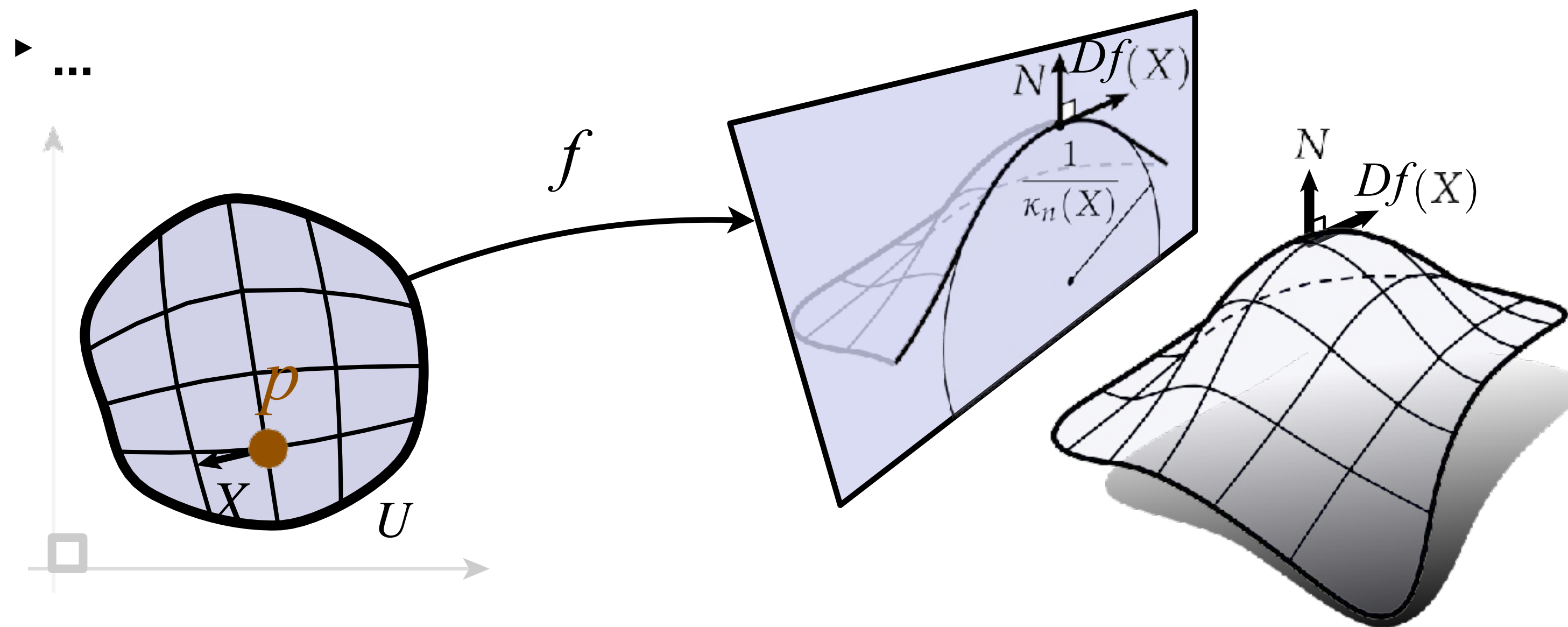
Extrinsics v.s. Intrinsic

- Extrinsics: cares about the embedding of a surface
 - Looking at the surface **outside** the surface
 - For example:
 - est. 6D pose of a rigid body
 - est. Articulation state of joints linking rigid bodies



Extrinsics v.s. Intrinsics

- Extrinsics: cares about the embedding of a surface
 - Looking at the surface **outside** the surface
 - For example:
 - est. 6D pose of a rigid body
 - est. Articulation state of joints linking rigid bodies
 - est. Normal curvature



Extrinsics v.s. Intrinsics

- Extrinsics: cares about the embedding of a surface
 - Looking at the surface **outside** the surface
 - For example:
 - est. 6D pose of a rigid body
 - est. Articulation state of joints linking rigid bodies
 - est. Normal curvature
 - ...
- Intrinsics:
 - Looking at the surface **inside** the surface

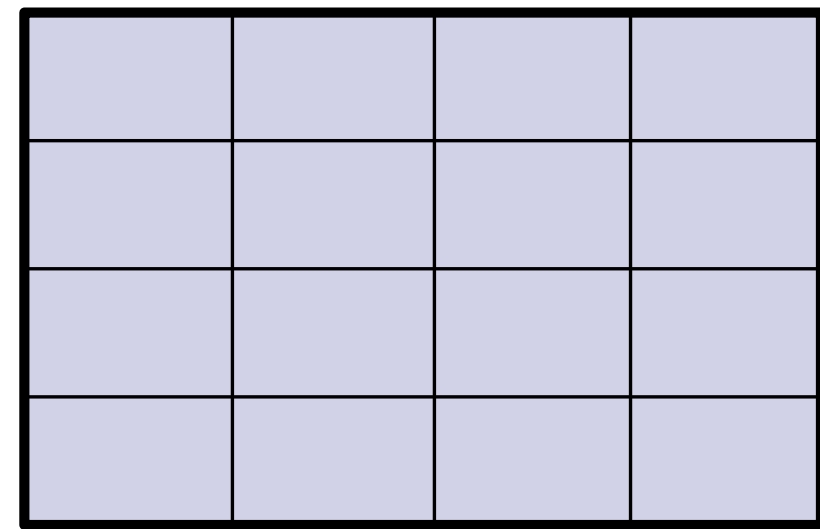
Extrinsics v.s. Intrinsics

- Extrinsics: cares about the embedding of a surface
 - Looking at the surface **outside** the surface
 - For example:
 - est. 6D pose of a rigid body
 - est. Articulation state of joints linking rigid bodies
 - est. Normal curvature
 - ...
- Intrinsics:
 - Looking at the surface **inside** the surface
 - **Geodesic distance** uniquely determines shape intrinsics

Gaussian and Mean Curvatures

Gaussian: $K := \kappa_1 \kappa_2$

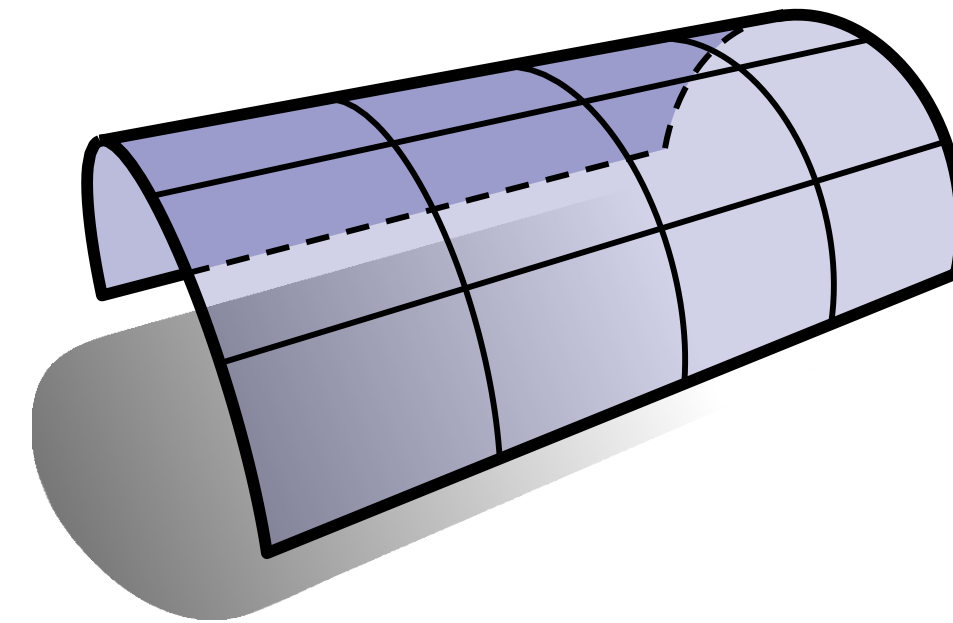
Mean: $H := \frac{1}{2}(\kappa_1 + \kappa_2)$



$$K = 0$$

$$H = 0$$

$$\kappa_n(X) \equiv 0$$

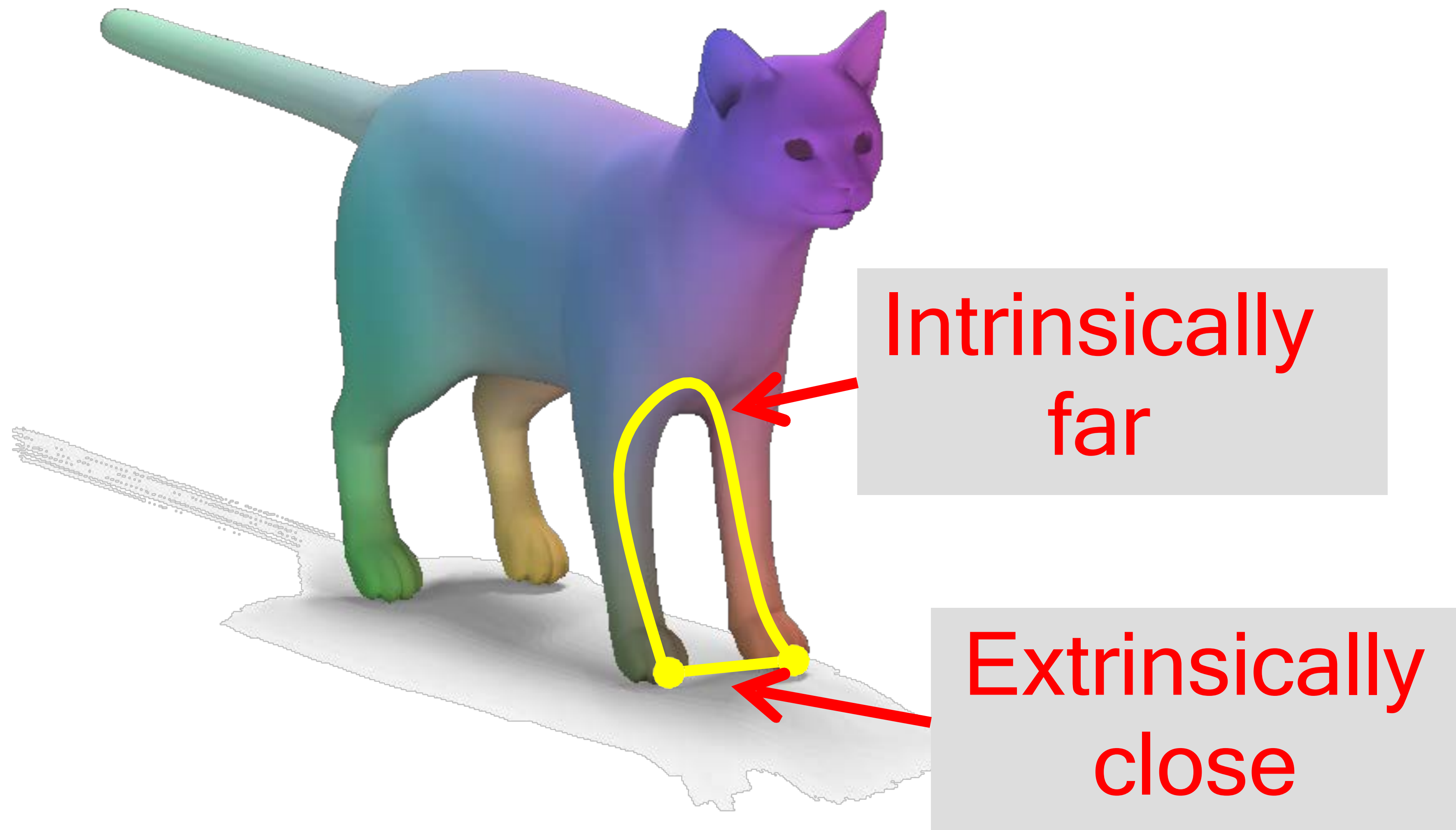


$$K = 0$$

$$H \neq 0$$

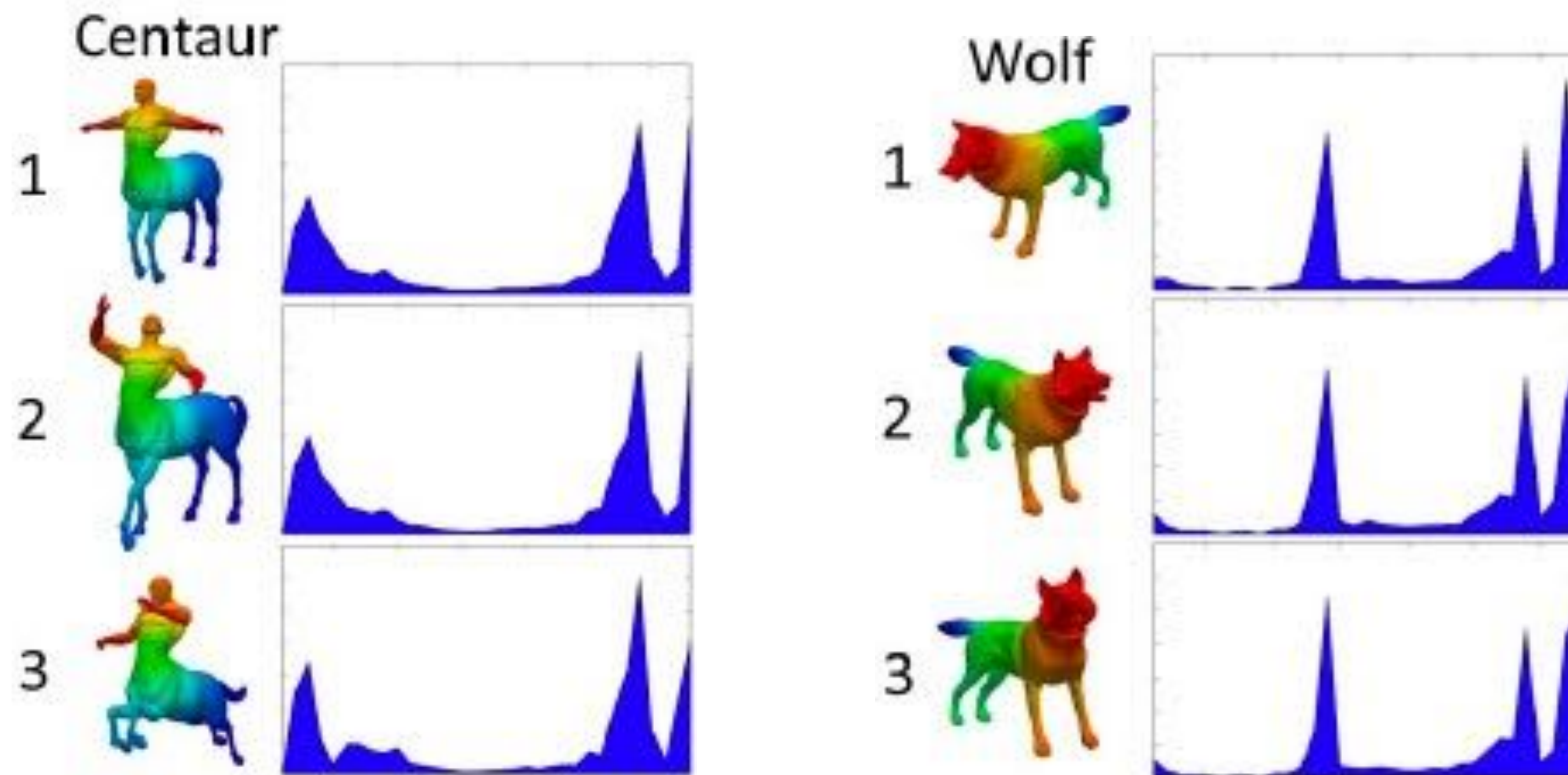
$$\kappa_n(X) \neq 0$$

Geodesic Distances



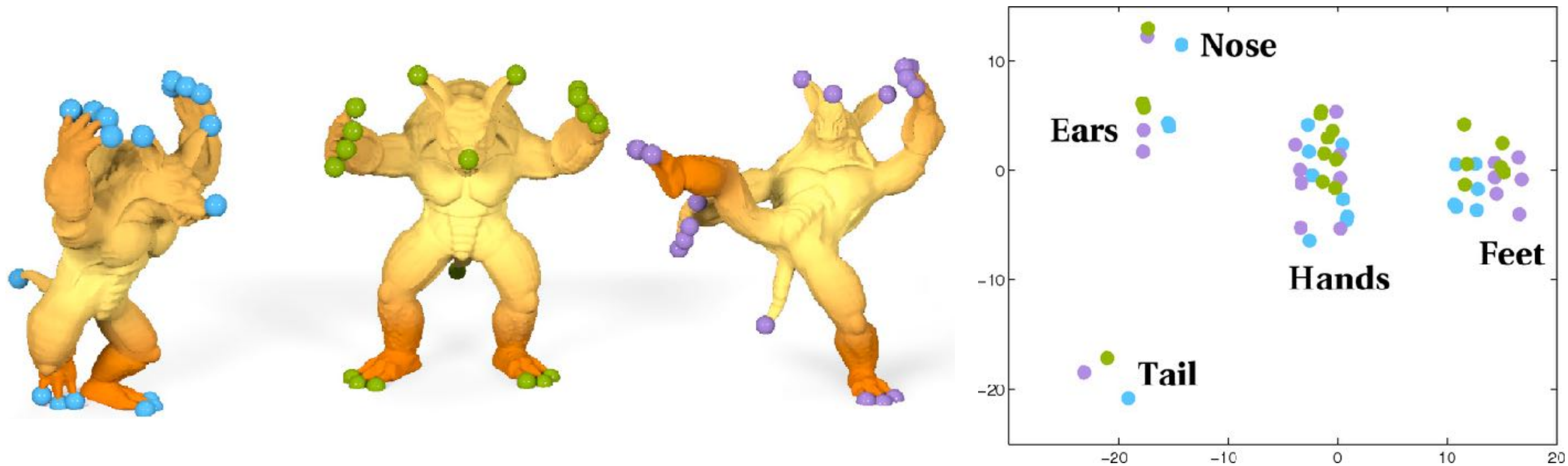
Applications: Shape Classification

- By classical method: Distribution of distances for point pairs randomly picked on the surface



Applications: Deformation-Invariant Point Feature

- By learning-based method: Extract some geodesic distance based features (intrinsic features)



Outline

- Introduction to intrinsic shape analysis
- *Analytic Method for Computing Geodesics*
- Learning to Predict Geodesics for Point Cloud
- Applications of Learning Point Cloud Geodesics
 - Normal Estimation
 - Mesh Reconstruction

Computing Geodesics



Single source

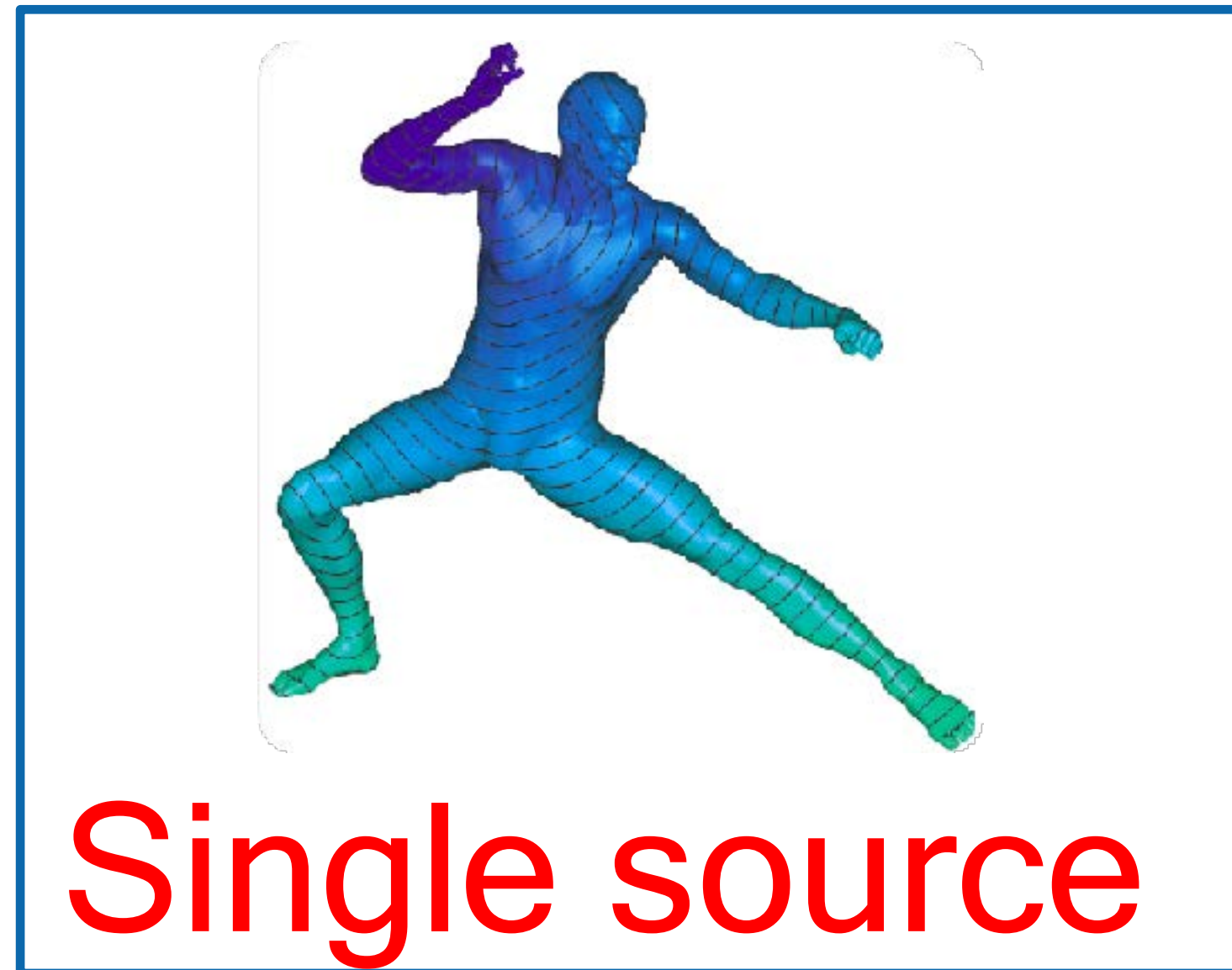


Multi-source



All-pairs

Computing Geodesics

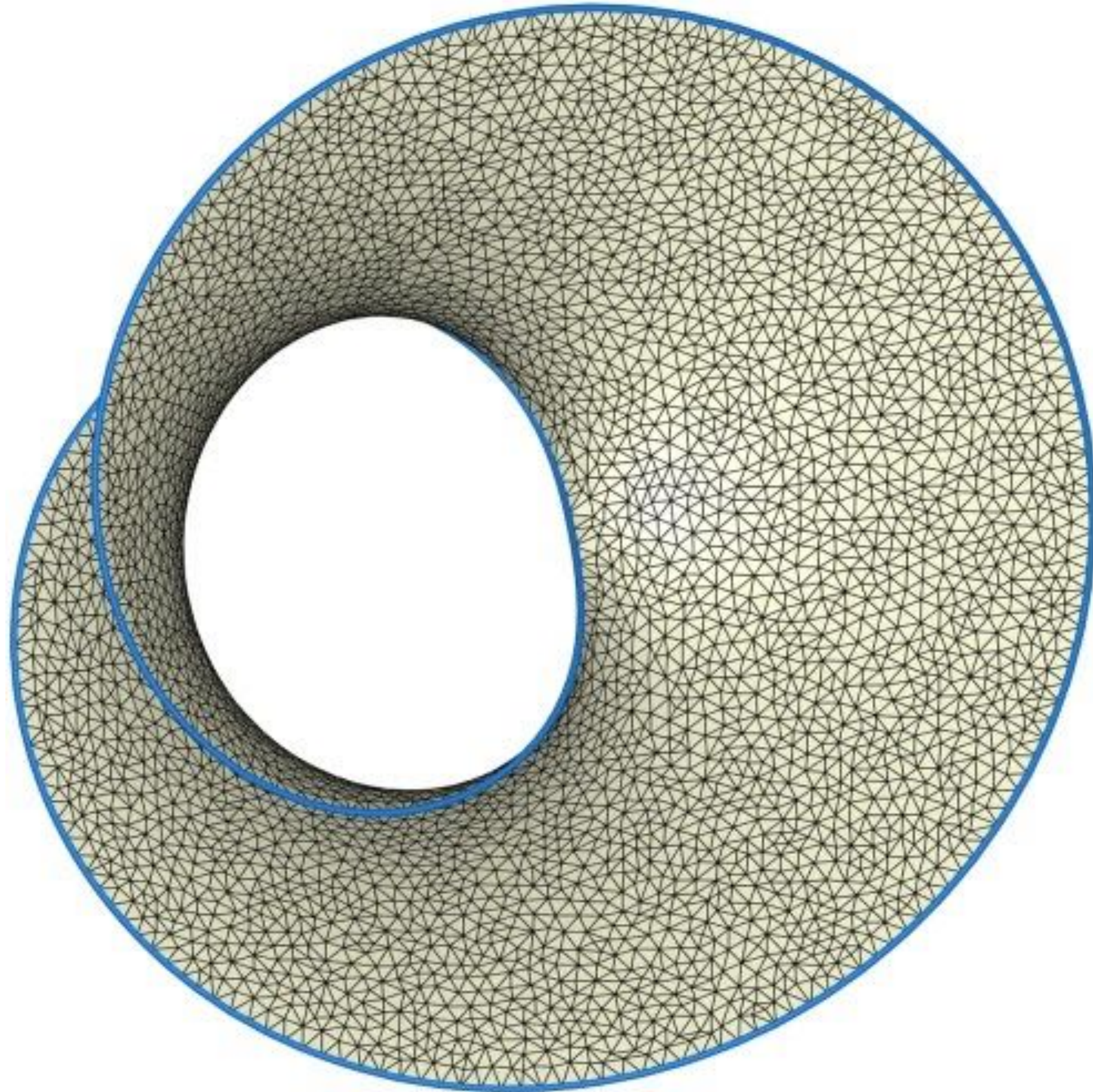


Multi-source



All-pairs

Computing Geodesics

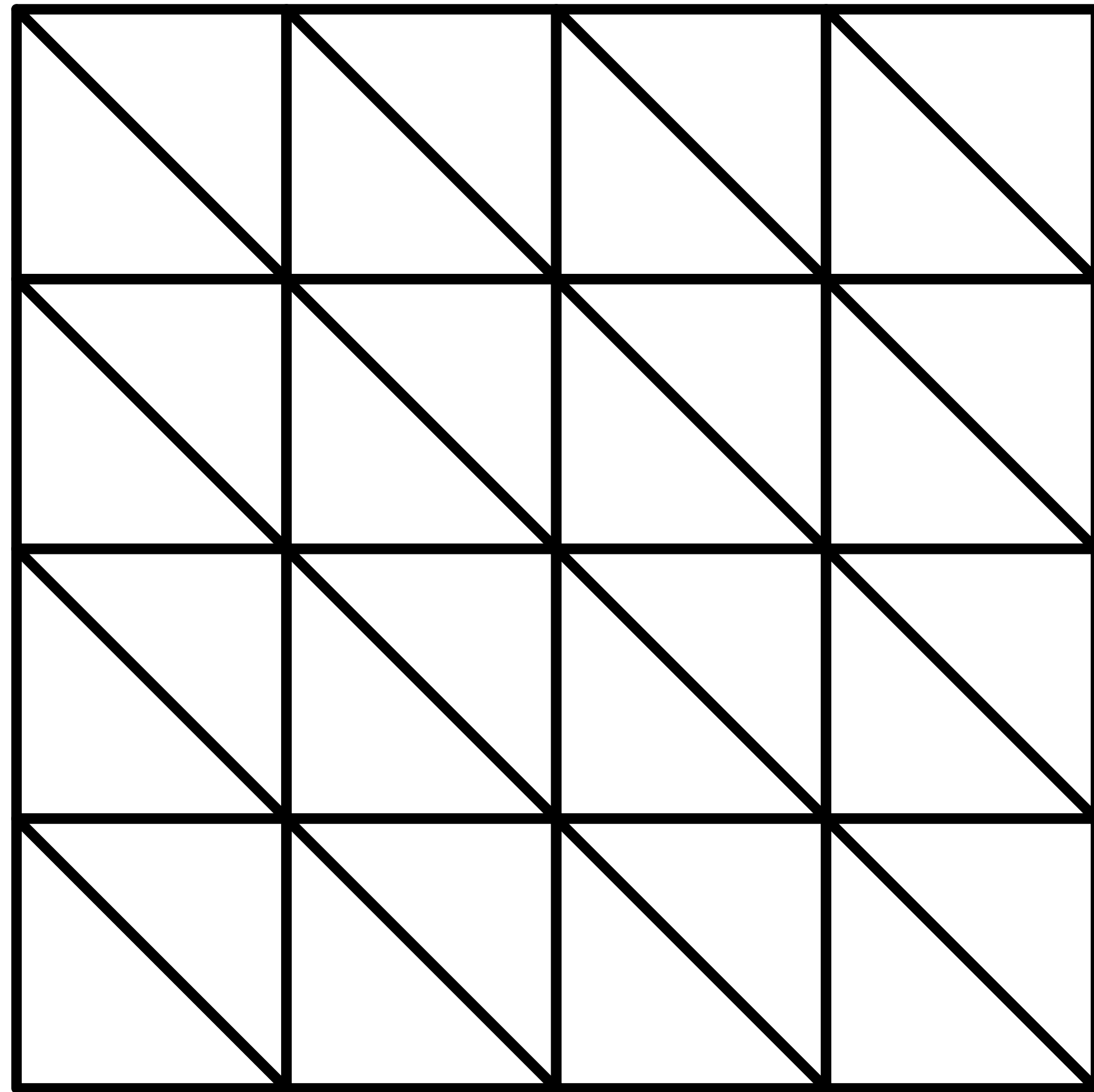


Approximate
geodesics
as paths
along edges

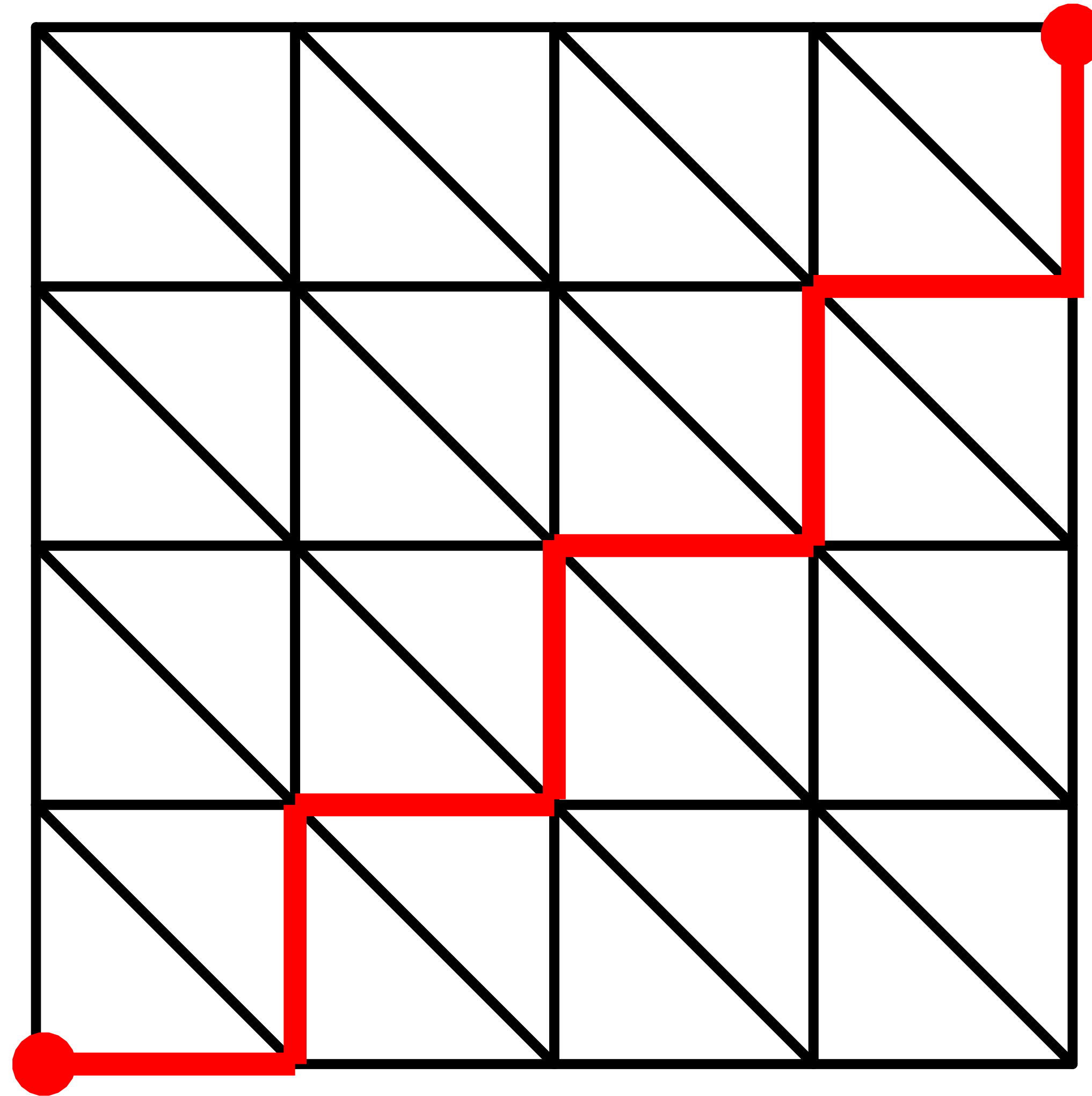
<http://www.cse.ohio-state.edu/~tamaldey/isotopic.html>

Meshes are graphs

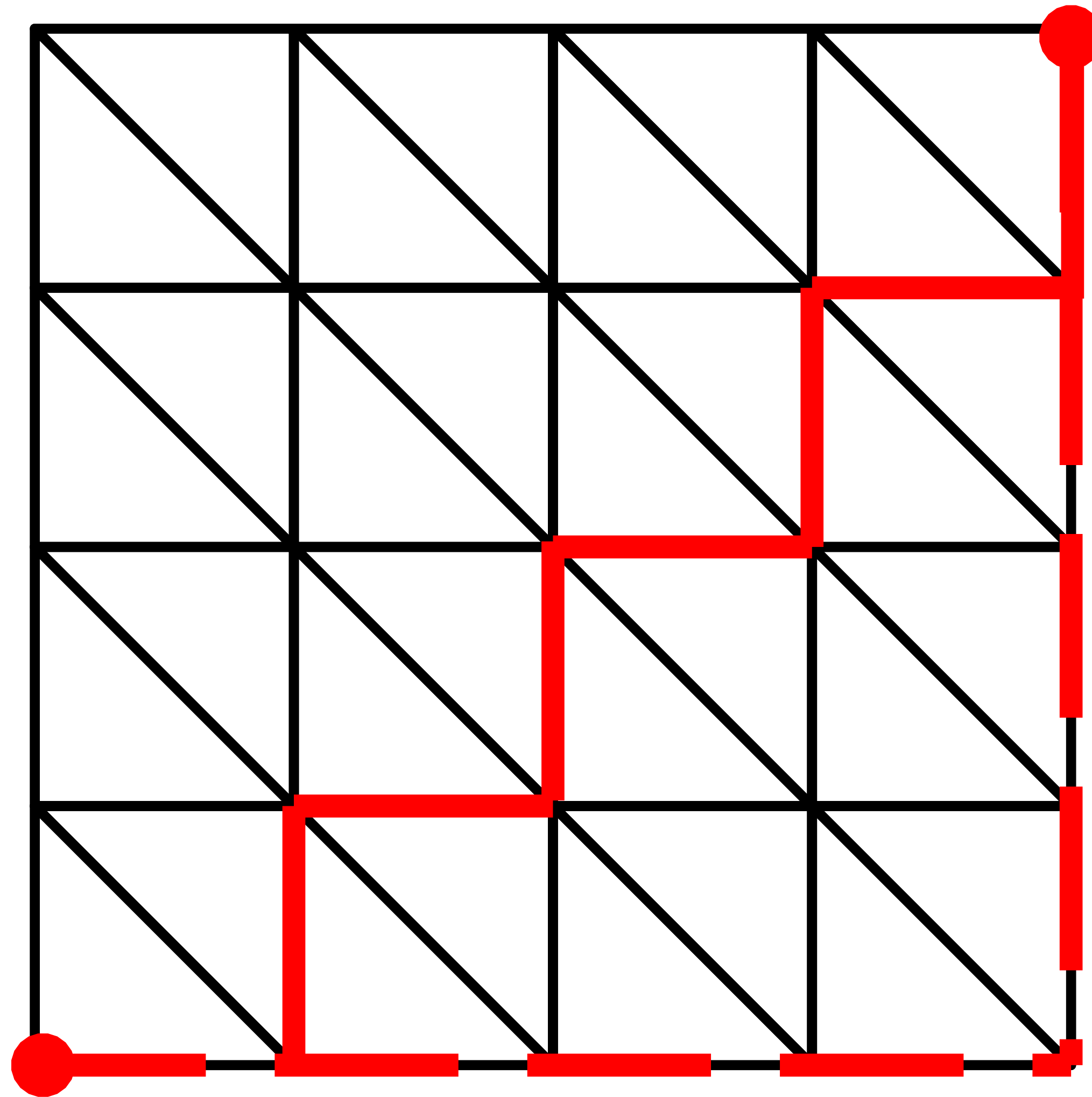
Shortest Path Algorithms for Geodesics Computation?



Shortest Path Algorithms for Geodesics Computation?



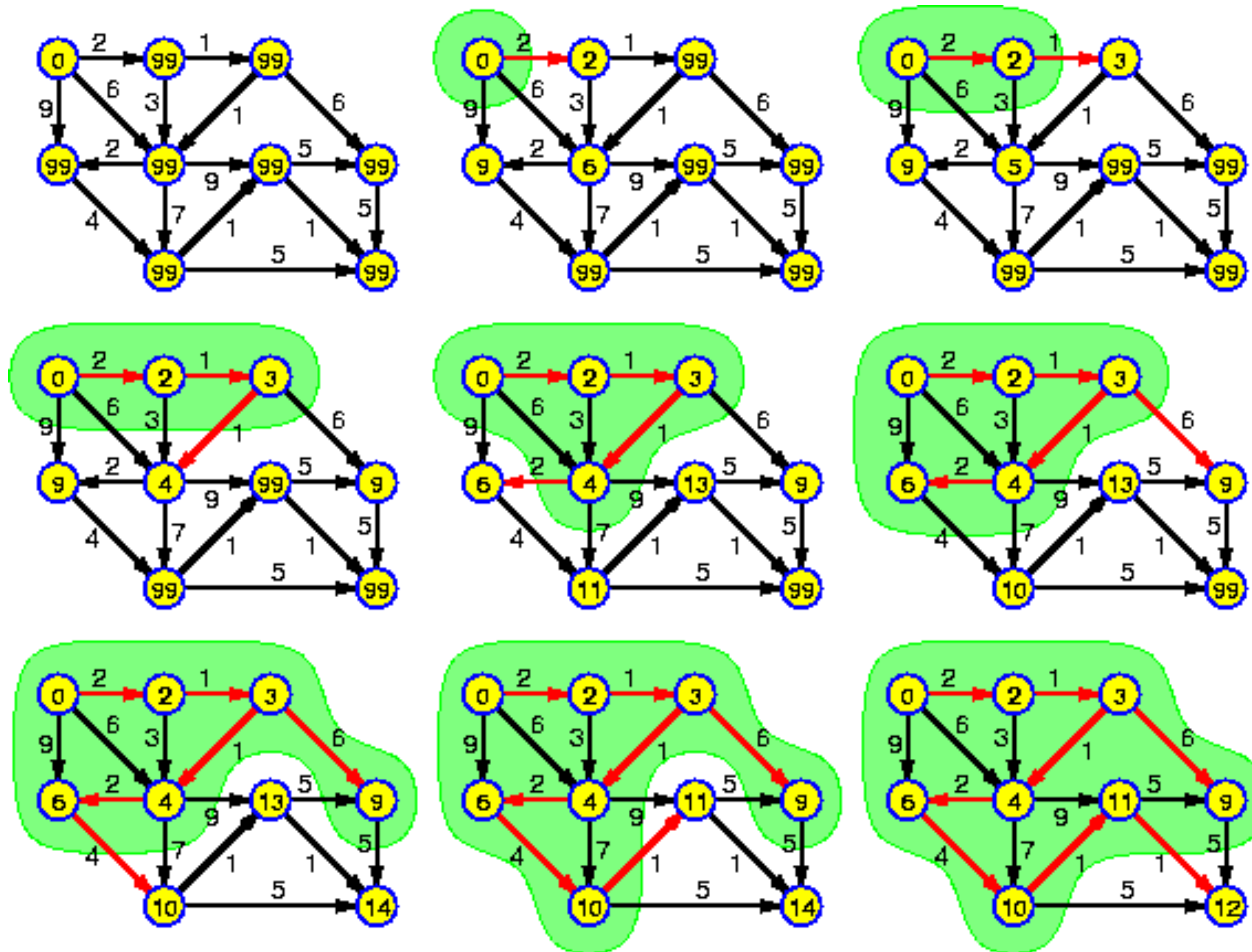
Shortest Path Algorithms for Geodesics Computation?



Fast Marching Algorithm

Dijkstra's algorithm, modified to approximate geodesic distances.

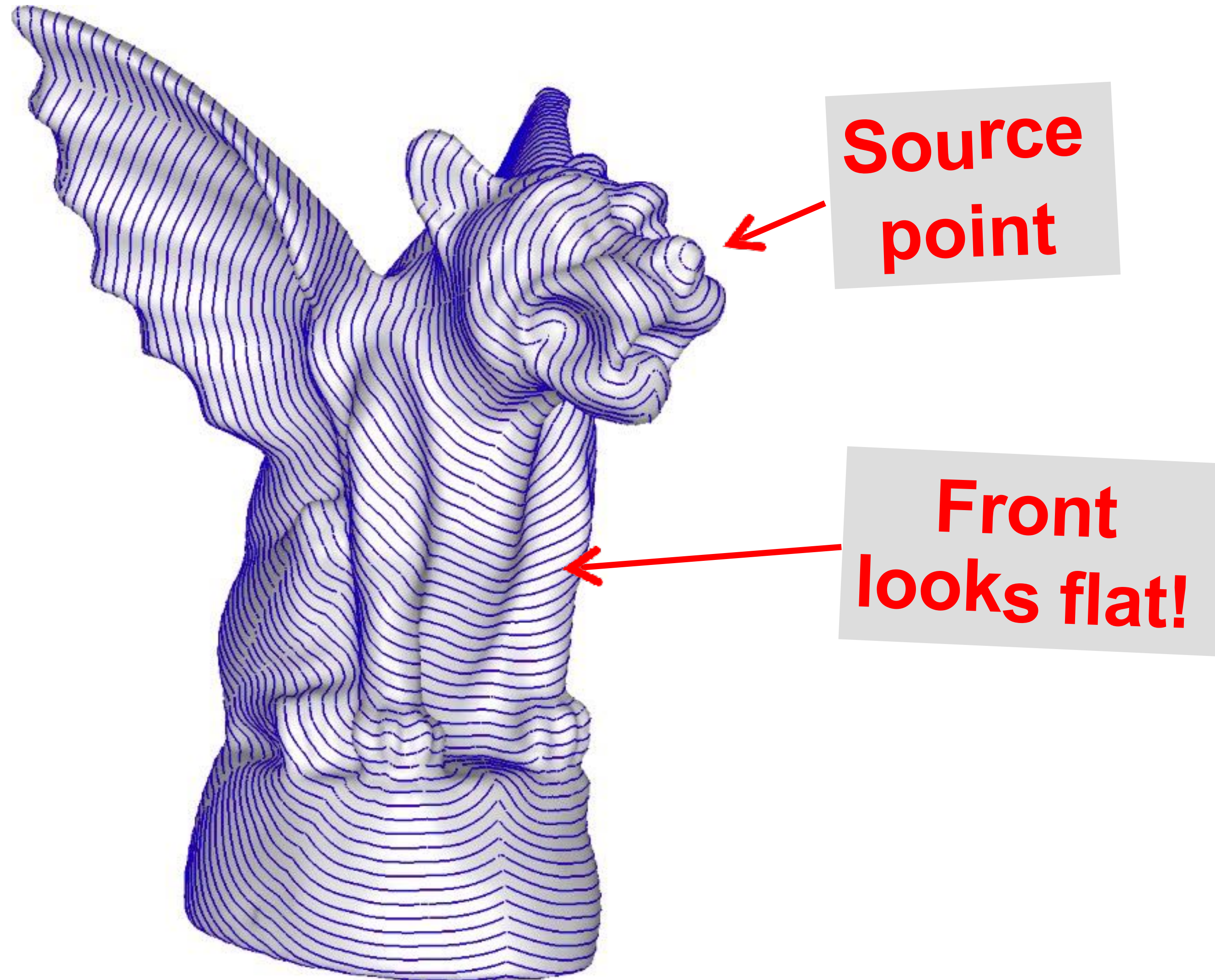
Review of Dijkstra's Algorithm



Key Idea of Dijkstra's Algorithm

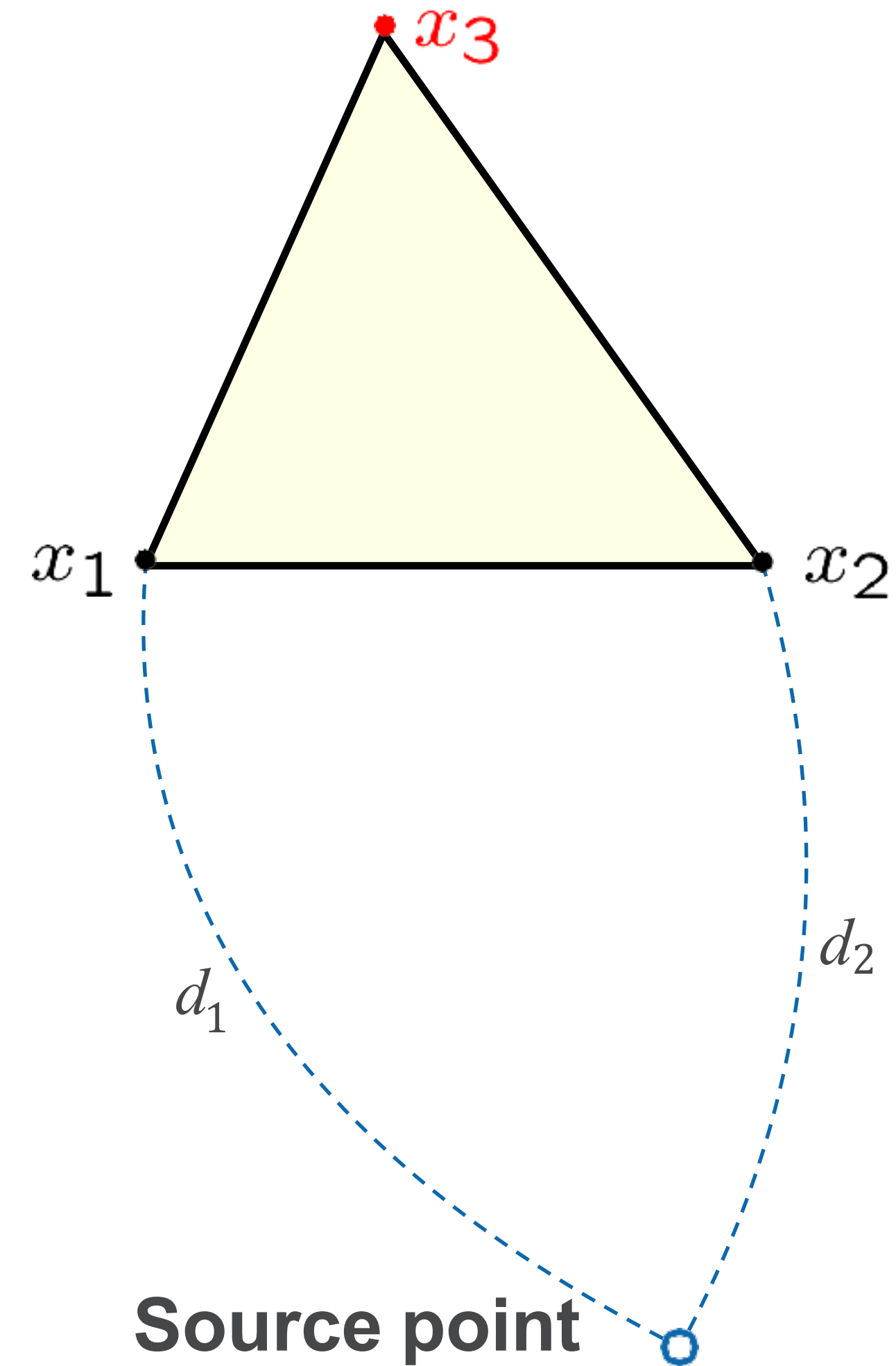
- Maintain a frontier set of vertices with the shortest distance from the source
- Propagate to the neighborhood

Fast Marching Algorithm



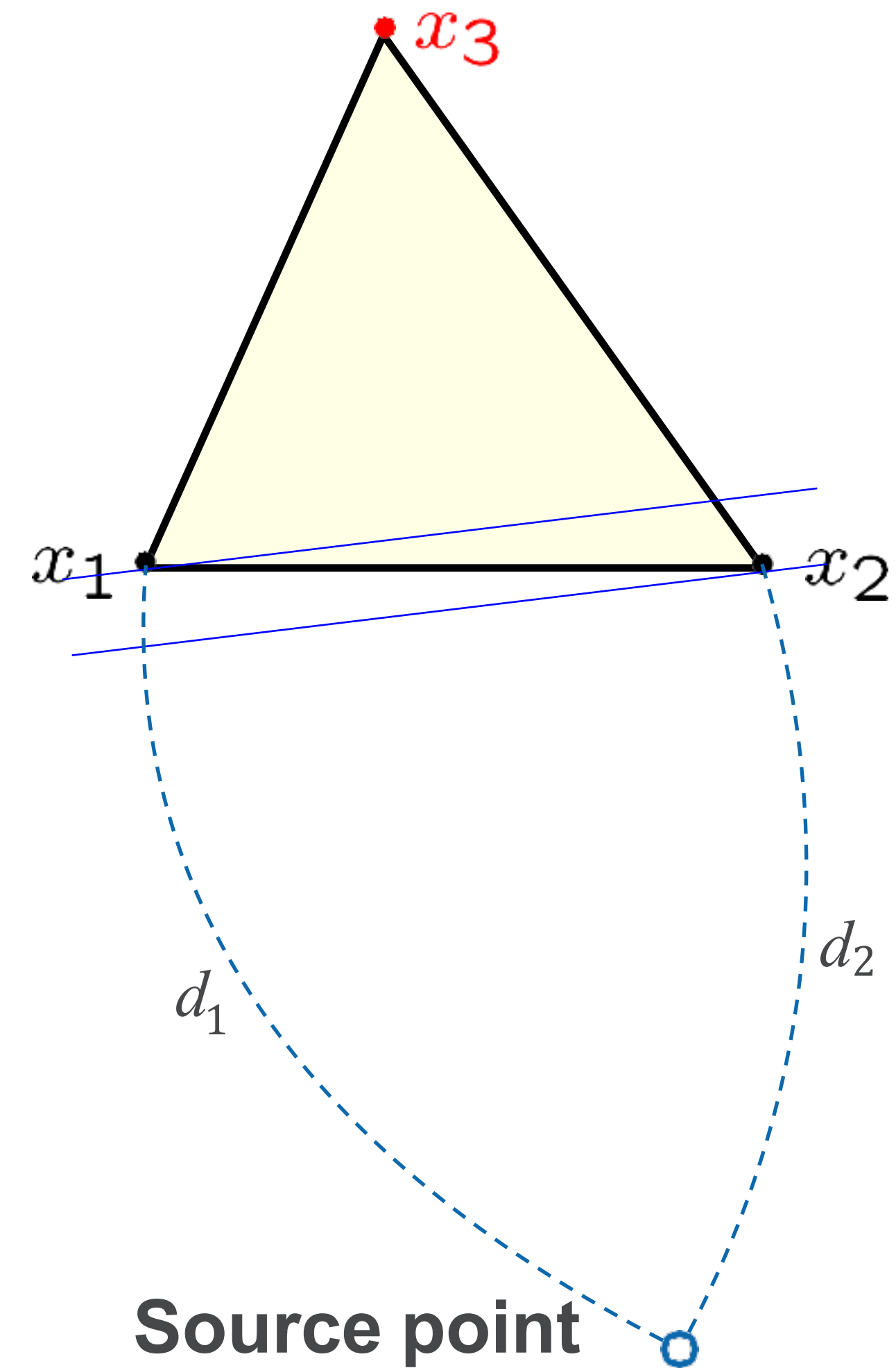
Fast Marching Algorithm

- At x_1 and x_2 stores the shortest paths d_1 and d_2
- Question: shortest path d_3 at x_3 ?

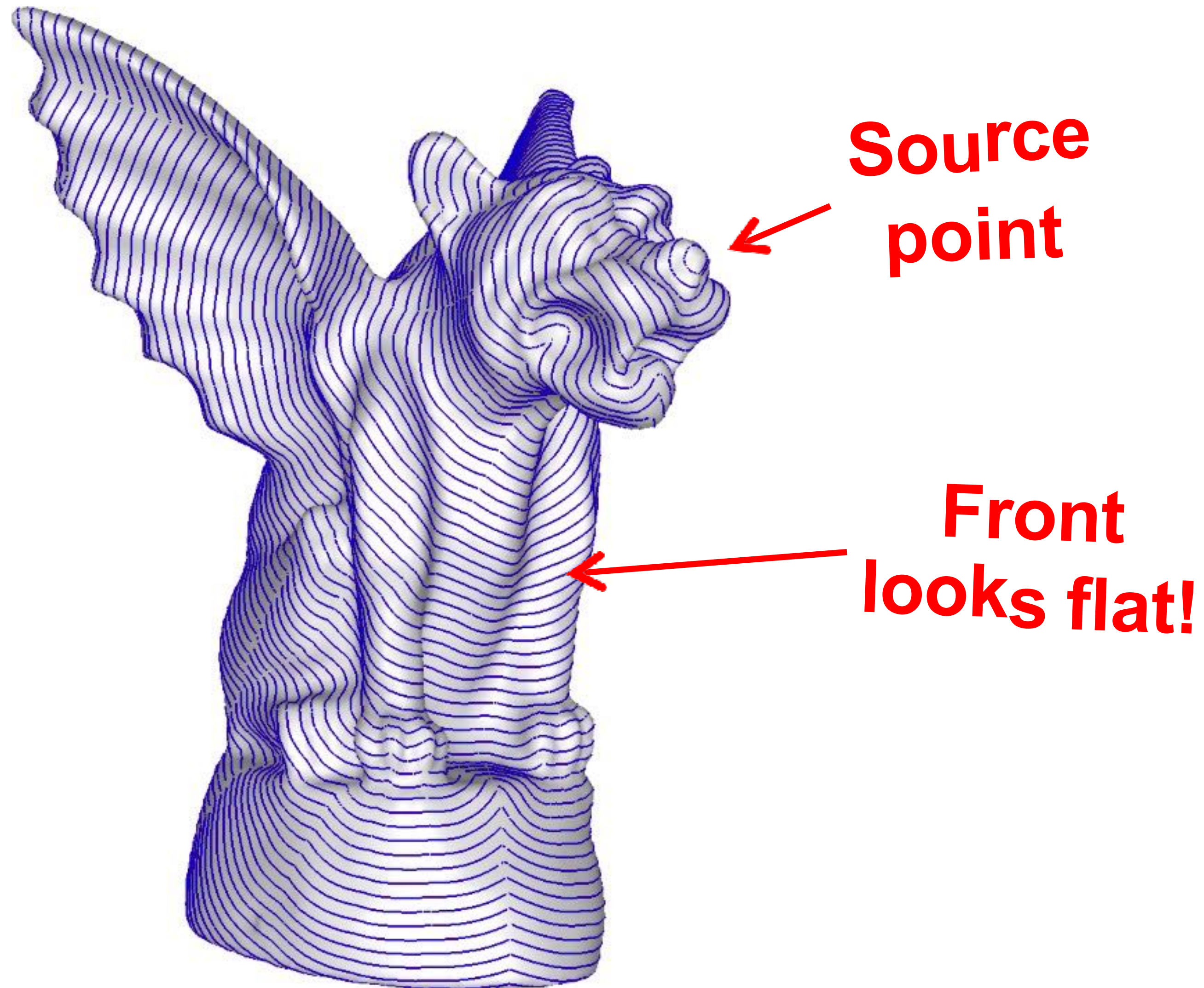


Fast Marching Algorithm

- Idea: Planar front approximation ($\because |x_1 x_2| \approx 0$)

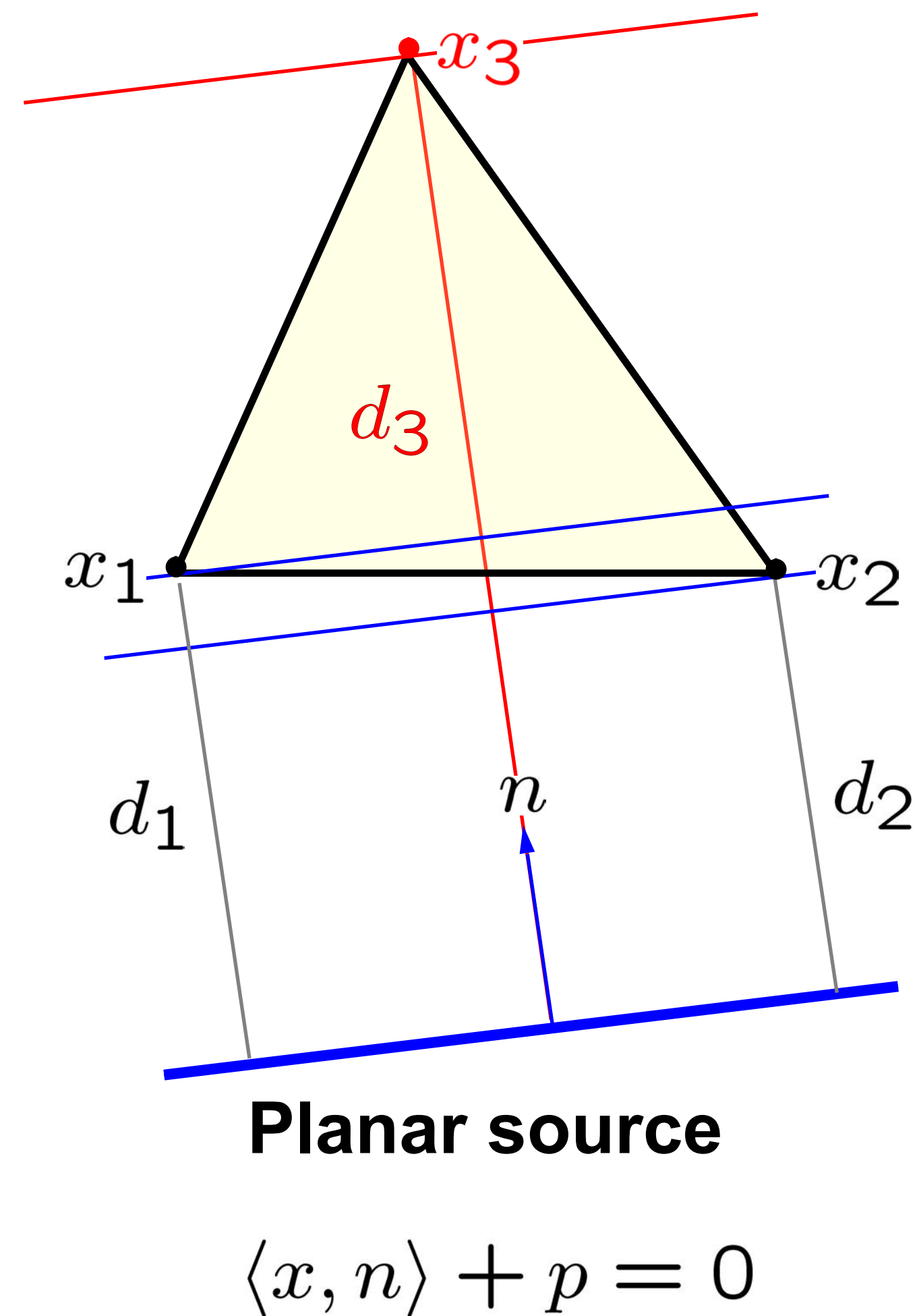
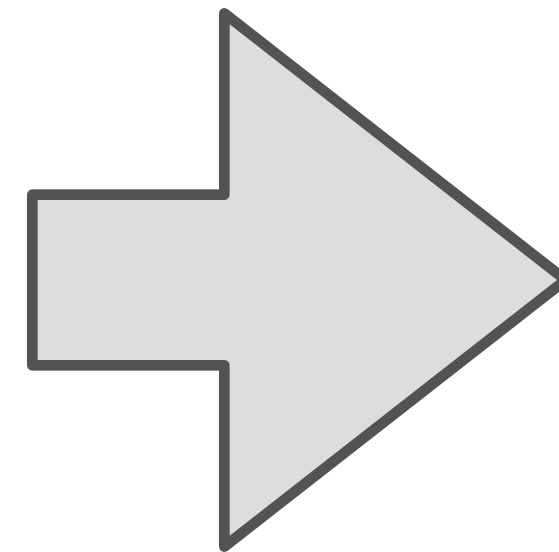
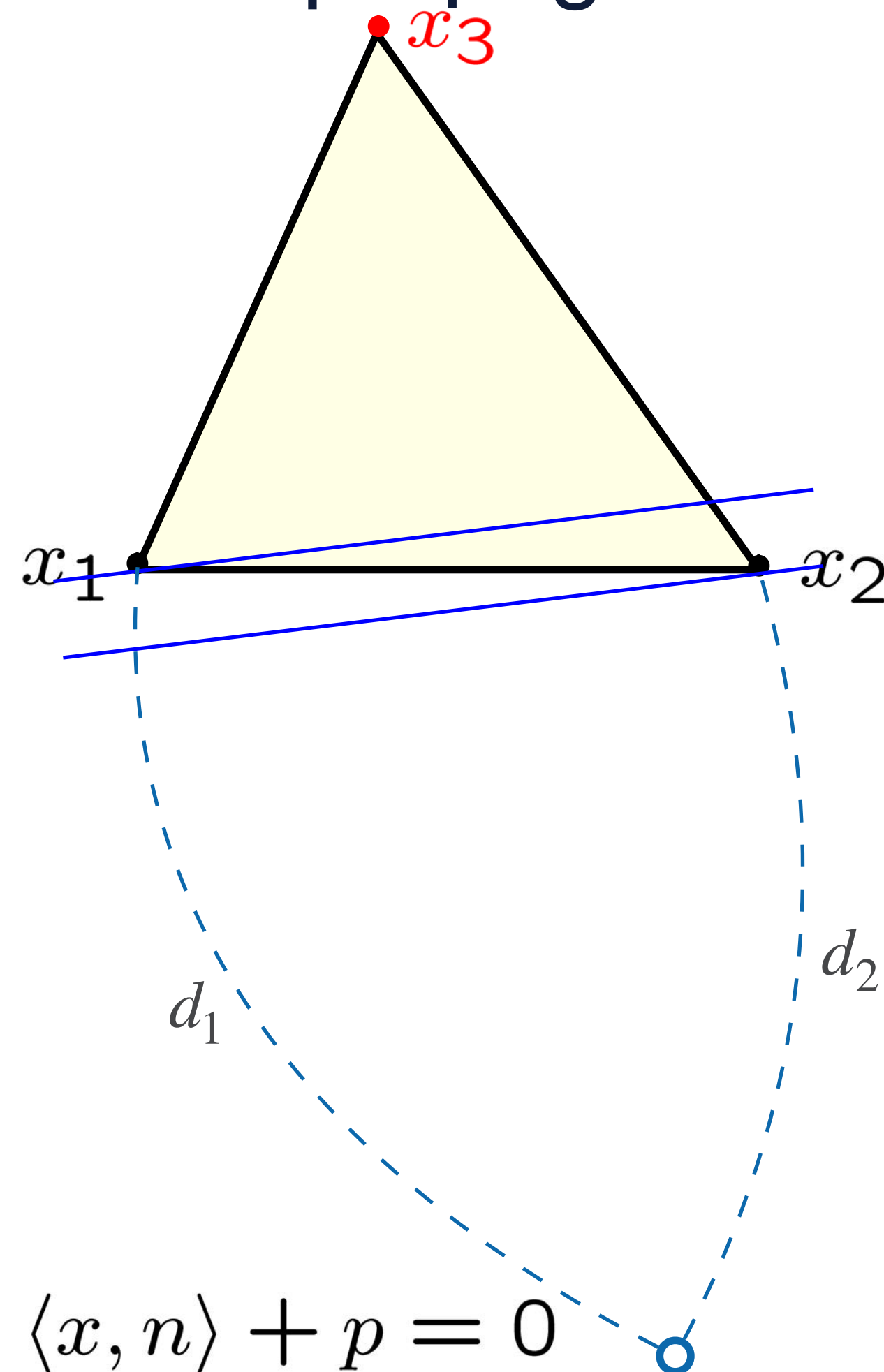


Trick: Every front looks flat!



Fast Marching Algorithm

- Change of view: point source $\rightarrow$ planar source
- Front propagates from **every point** on the source



Fast Marching Update Steps

- Front hits x_1 at time d_1
- Hits x_2 at time d_2
- When does the front arrive x_3 ?

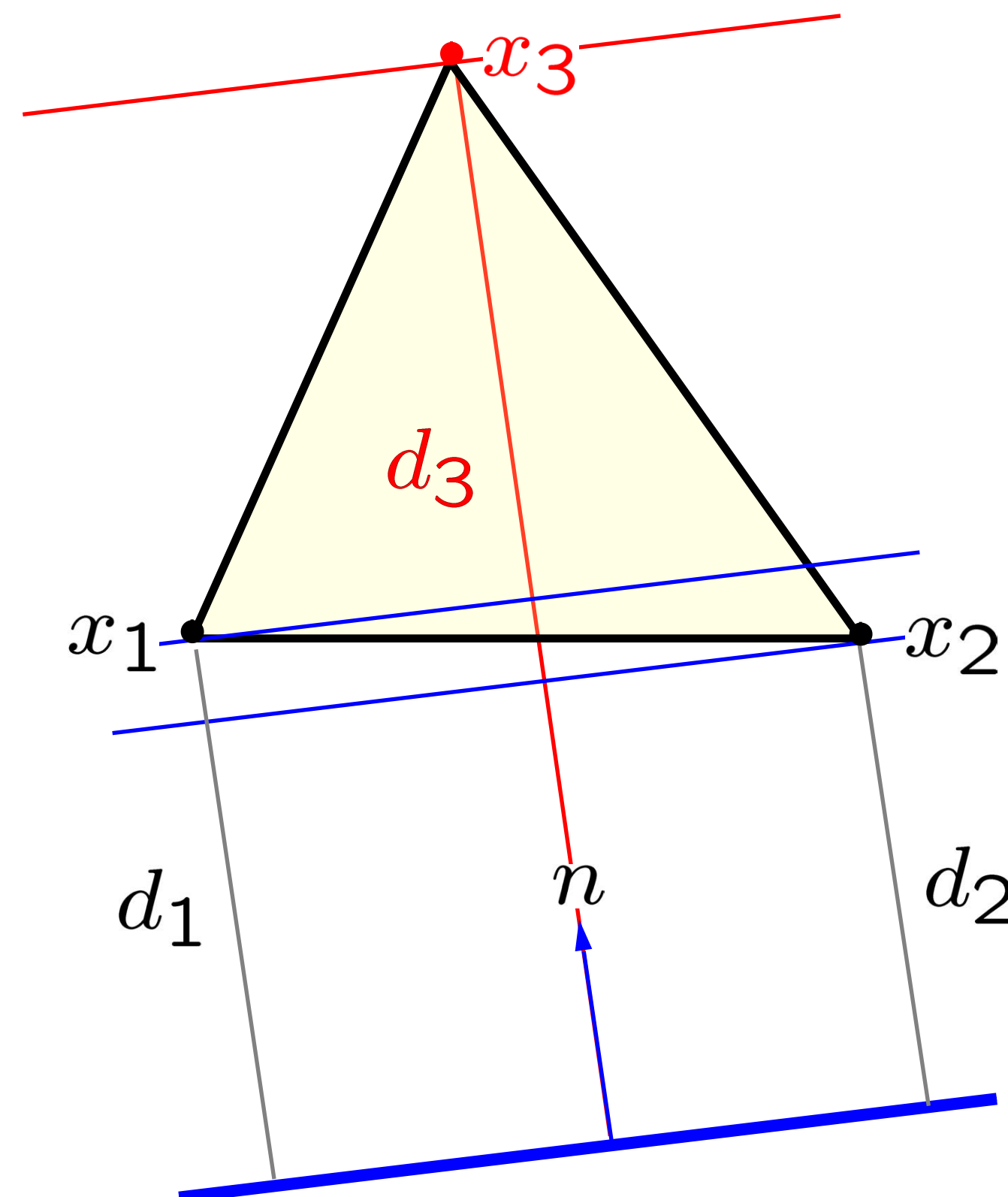
- Step 1: Model **wave front** propagation from a planar source

$$\langle x, n \rangle + p = 0$$

n : unit **propagation direction**

p : source **offset**

- Step 2: Compute the distance from x_3 to the plane



Planar source

$$\langle x, n \rangle + p = 0$$

Practical Implementation

Fast Exact and Approximate Geodesics on Meshes

Vitaly Surazhsky
University of Oslo

Tatiana Surazhsky
University of Oslo

Danil Kirsanov
Harvard University

Steven J. Gortler
Harvard University

Hugues Hoppe
Microsoft Research

Abstract

The computation of geodesic paths and distances on triangle meshes is a common operation in many computer graphics applications. We present several practical algorithms for computing such geodesics from a source point to one or all other points efficiently. First, we describe an implementation of the exact “single source, all destination” algorithm presented by Mitchell, Mount, and Papadimitriou (MMP). We show that the algorithm runs much faster in practice than suggested by worst case analysis. Next, we extend the algorithm with a merging operation to obtain computationally efficient and accurate approximations with bounded error. Finally, to compute the shortest path between two given points, we use a lower-bound property of our approximate geodesic algorithm to efficiently prune the frontier of the MMP algorithm, thereby obtaining an exact solution even more quickly.

Keywords: shortest path, geodesic distance.

1 Introduction

In this paper we present practical methods for computing both exact and approximate shortest (i.e. geodesic) paths on a triangle mesh. These geodesic paths typically cut across faces in the mesh and are therefore not found by the traditional graph-based Dijkstra algorithm for shortest paths.

The computation of geodesic paths is a common operation in many computer graphics applications. For example, parameterizing a mesh often involves cutting the mesh into subparts, as done in [Katz and Tal 2003; Funkhouser et al.



Figure 1: Geodesic paths from a source vertex, and isolines of the geodesic distance function.

tance function over the edges, the implementation is actually practical even though, to our knowledge, it has never been done previously. We demonstrate that the algorithm’s worst case running time of $O(n^2 \log n)$ is pessimistic, and that in practice, the algorithm runs in sub-quadratic time. For instance, we can compute the exact geodesic distance from a source point to all vertices of a 400K-triangle mesh in about one minute.

Approximation algorithm We extend the algorithm with a merging operation to obtain computationally efficient and accurate approximations with bounded error. To compute the shortest path

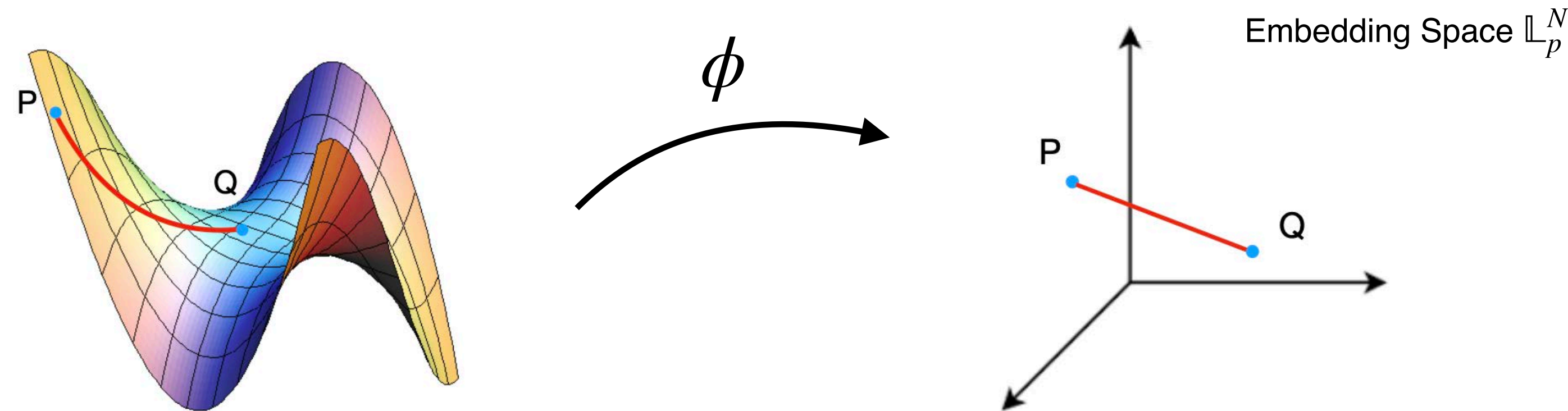
efficiently obtain the exact solution to the “single source, single destination” problem, by using a lower-bound property of our ap

<http://code.google.com/p/geodesic/>

Outline

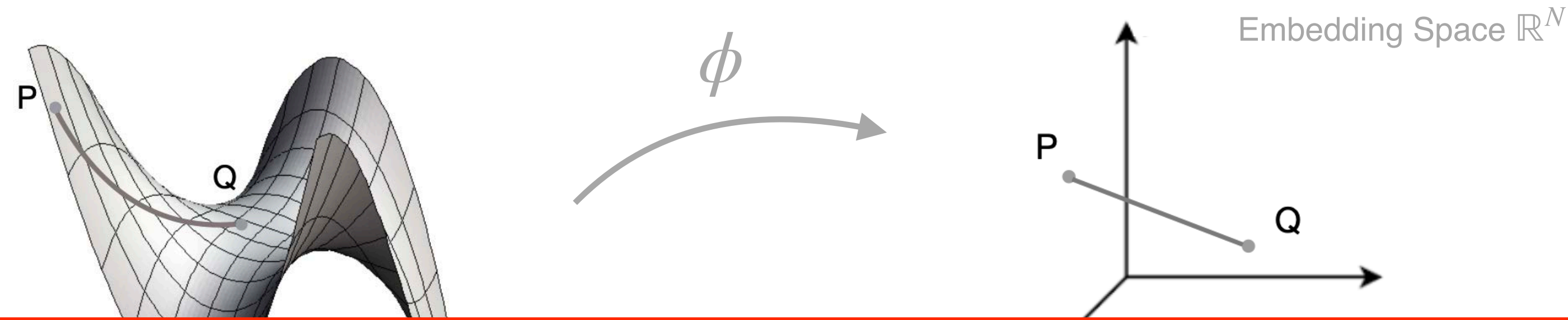
- Introduction to intrinsic shape analysis
- Analytic Method for Computing Geodesics
- *Learning to Predict Geodesics for Point Cloud*
- Applications of Learning Point Cloud Geodesics
 - Normal Estimation
 - Mesh Reconstruction

Metric Learning Formulation



- Given points P, Q on the surface, what is $d_G(P, Q)$?
- Metric learning:
$$d_G(P, Q) \approx \|\phi(P) - \phi(Q)\|_p$$
- ϕ : embedding function (e.g., MLP)
- Usually, $\dim(\phi(\cdot)) \gg 3$ and $p = 2$

Metric Learning Formulation



Q: Could “ $\approx$ ” become “ $=$ ” with $\dim(\phi(\cdot)) < \infty$?

- Metric learning.

$$d_G(P, Q) \approx \|\phi(P) - \phi(Q)\|_p$$

- ϕ : embedding function (e.g., MLP)
- Usually, $\dim(\phi(\cdot)) \gg 3$ and $p = 2$

Counter-Example

- Recall our $SO(2)$ example:
 - Distance preservation (geodesic v.s. ℓ_p) would preserve topological structure
 - Circle has different topology as line segments
 - So $SO(2)$ cannot be embedded into Euclidean space without distortion
- Some results of the embedding for general metric space to come

Distortion of Embedding

- Metrics:
 - $d(x, y) \geq 0$
 - $d(x, x) = 0$ and $\forall y \neq x, d(x, y) > 0$
 - $d(x, y) + d(y, z) \geq d(x, z)$
- Assume $\phi : S \mapsto T$ and $d_S(\cdot, \cdot)$ and $d_T(\cdot, \cdot)$
 - $\text{expansion}(\phi) = \max_{x,y} \frac{d_T(\phi(x), \phi(y))}{d_S(x, y)}$
 - $\text{contraction}(\phi) = \max_{x,y} \frac{d_S(x, y)}{d_T(\phi(x), \phi(y))}$
 - $\text{distortion}(\phi) = \text{expansion} \times \text{contraction}$

A Well-Known Result

Theorem (Bourgain, 1985).

Let (X, d) be a metric space on n points. Then,

$$(X, d) \xrightarrow{O(\log n)} \ell_p^{O(\log^2 n)}$$

Remarks:

- **Any n-point** metric space (X, d) can be embedded in ℓ_p with distortion $O(\log n)$
- To accommodate more points, one needs higher dim space
- This bound can be shown tight (by worse case, w.r.t. distortion)

Influence

- We cannot expect that a finite (even infinite) dimensional ℓ_p space would approximate any metric
- To compromise, we usually require the approximation to have higher precision for closer points:
 - The geometry underlying closer points may be better embeddable (e.g., on the same tangent plane)
 - Far away points just need to be known far away
- Recall the hinge loss for point feature learning in the *bottom-up instance segmentation* lecture:

$$L_{ij} = \max(0, K - \|F_i - F_j\|)$$

Example: GeoNet

- Geodesics for Point Clouds
- No connectivity information

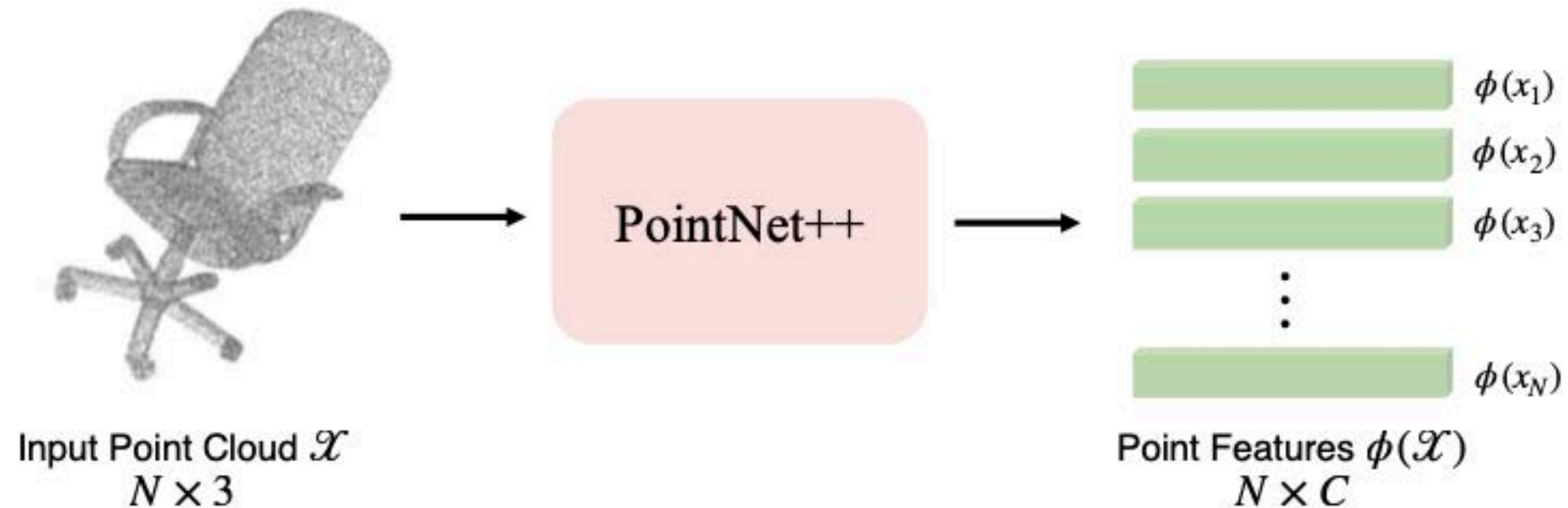


Task Formulation

- Input: Point Cloud $\mathcal{X} = \{x_i\}_{i=1}^N$
- Output: Geodesic distance to K Euclidean neighbors for each point

GeoNet: Geodesic Distance Regression

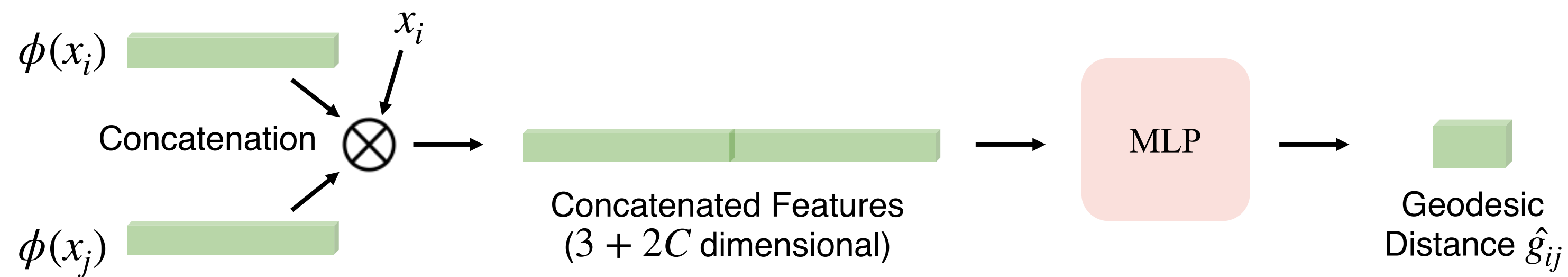
Step 1: Feature Extraction



GeoNet: Geodesic Distance Regression

Step 2: Metric Learning

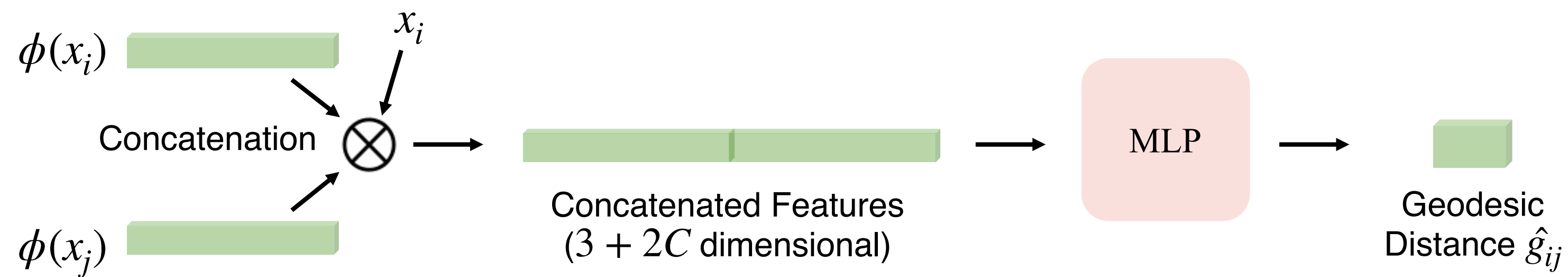
For each x_i and its neighbor x_j :



GeoNet: Geodesic Distance Regression

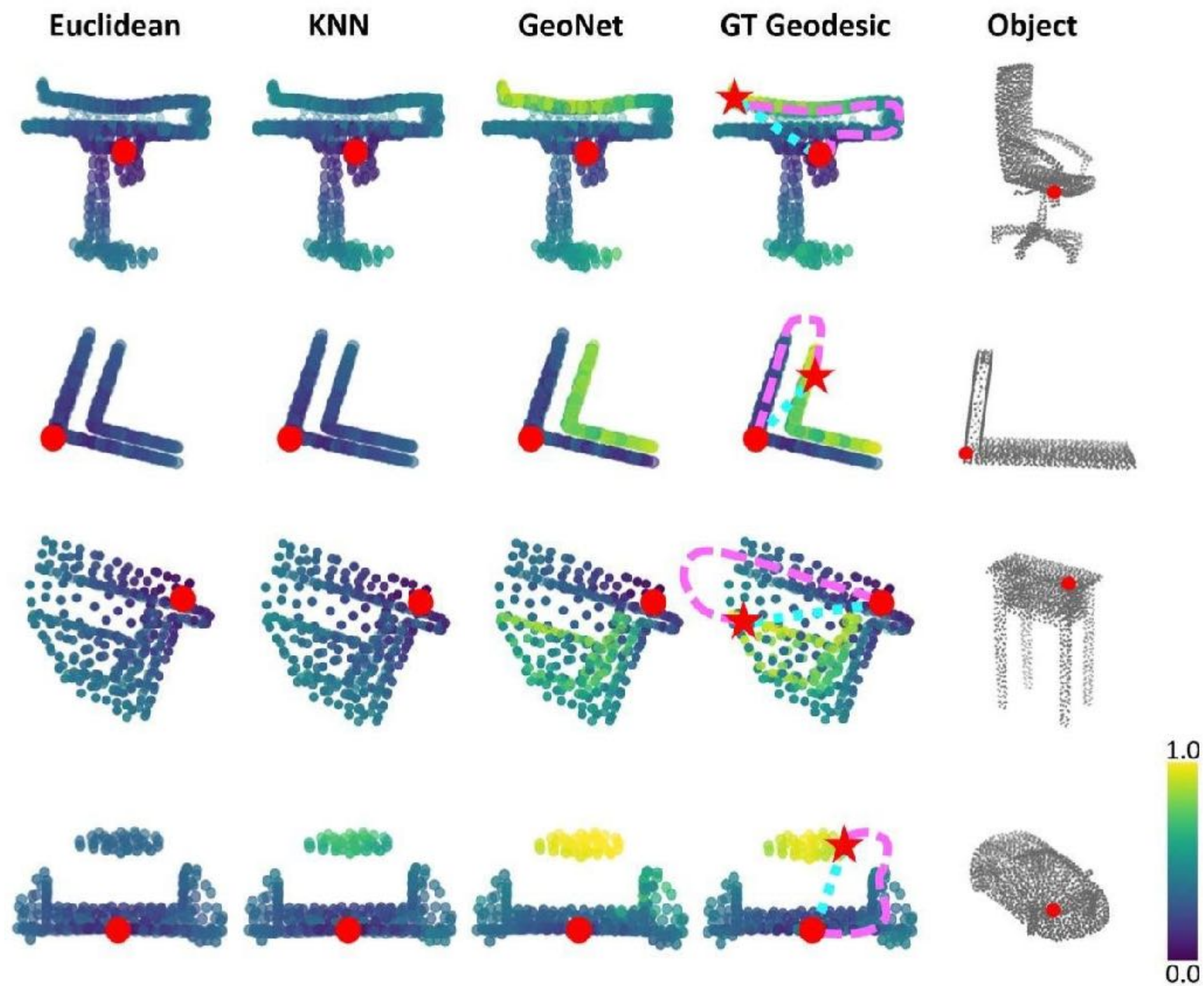
Step 2: Metric Learning

For each x_i and its neighbor x_j :



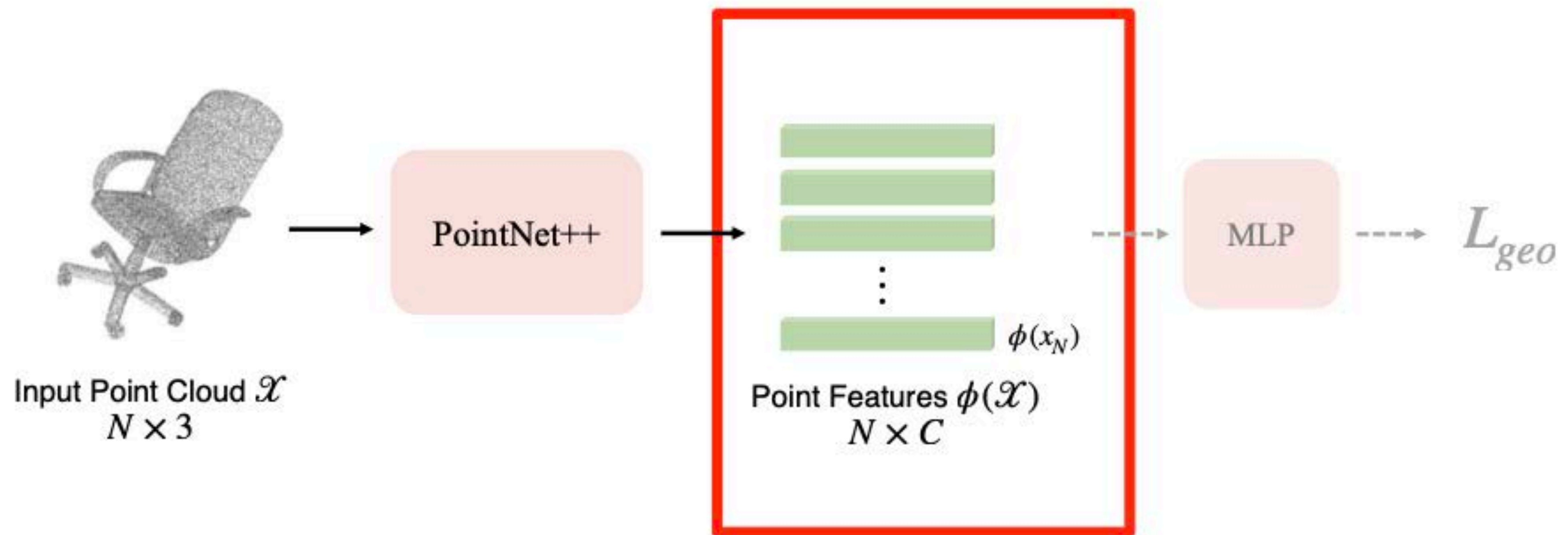
$$L_{geo} = \sum_{x_i \in \mathcal{X}} \sum_{x_j \in NN(x_i)} |g_{ij} - \hat{g}_{ij}|$$

Geodesics Regression



Geodesics Induced Features

- Geodesics induced features are useful for downstream tasks



Outline

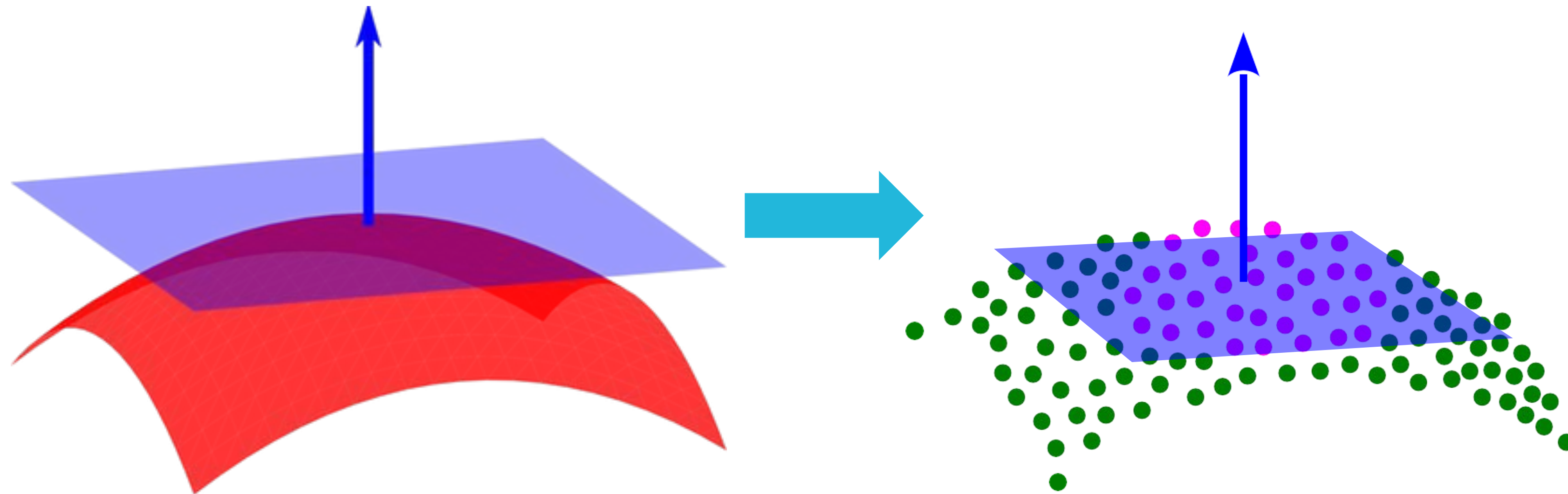
- Introduction to intrinsic shape analysis
- Analytic Method for Computing Geodesics
- Learning to Predict Geodesics for Point Cloud
- Applications of Learning Point Cloud Geodesics
 - *Normal Estimation*
 - Mesh Reconstruction

Recall: Normal Estimation

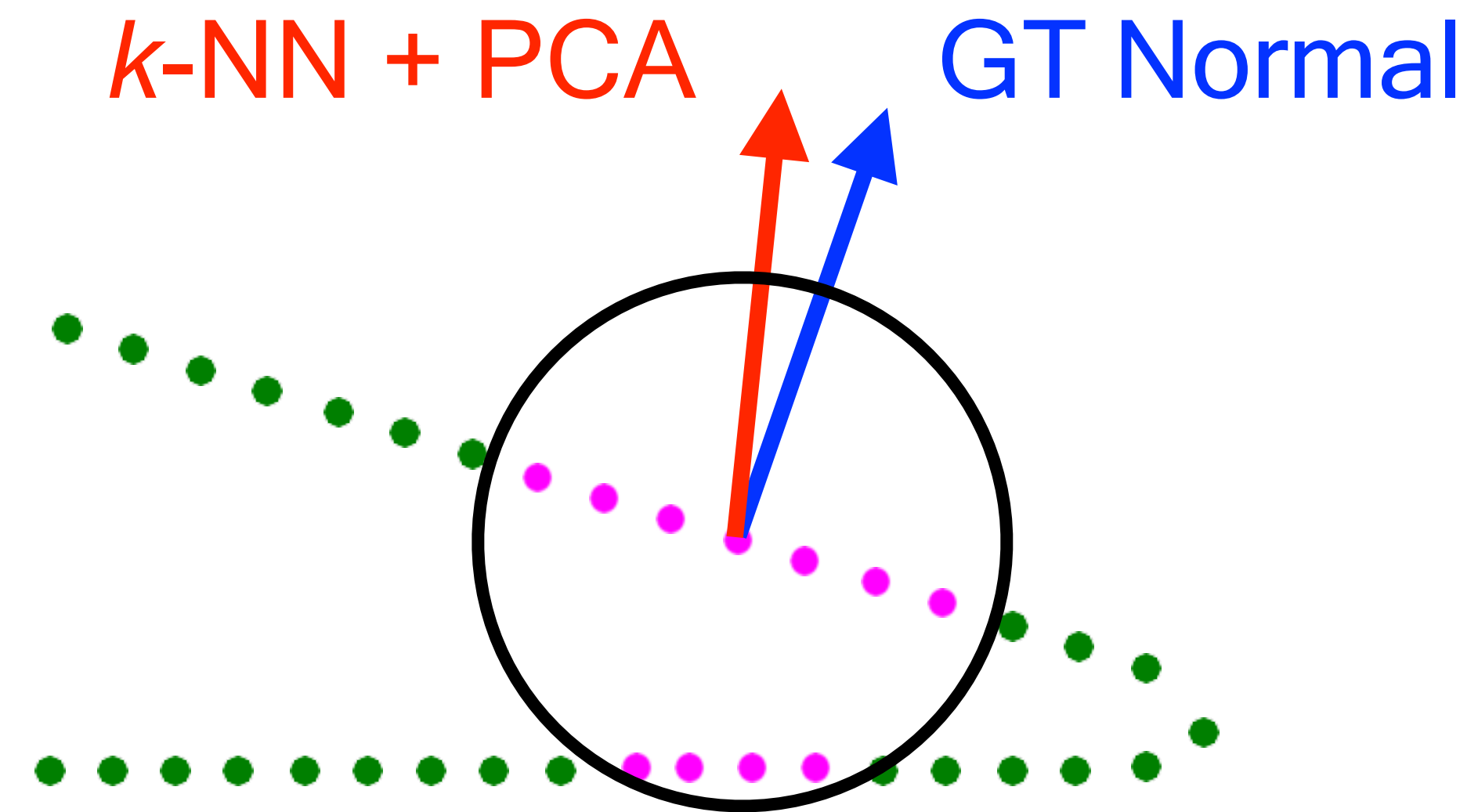
- Plane-fitting: find the plane that best fits the neighborhood of a point of interest

- Let $M = \sum_i (x_i - \bar{x})(x_i - \bar{x})^T$ and $\bar{x} = \frac{1}{n} \sum_i x_i$,

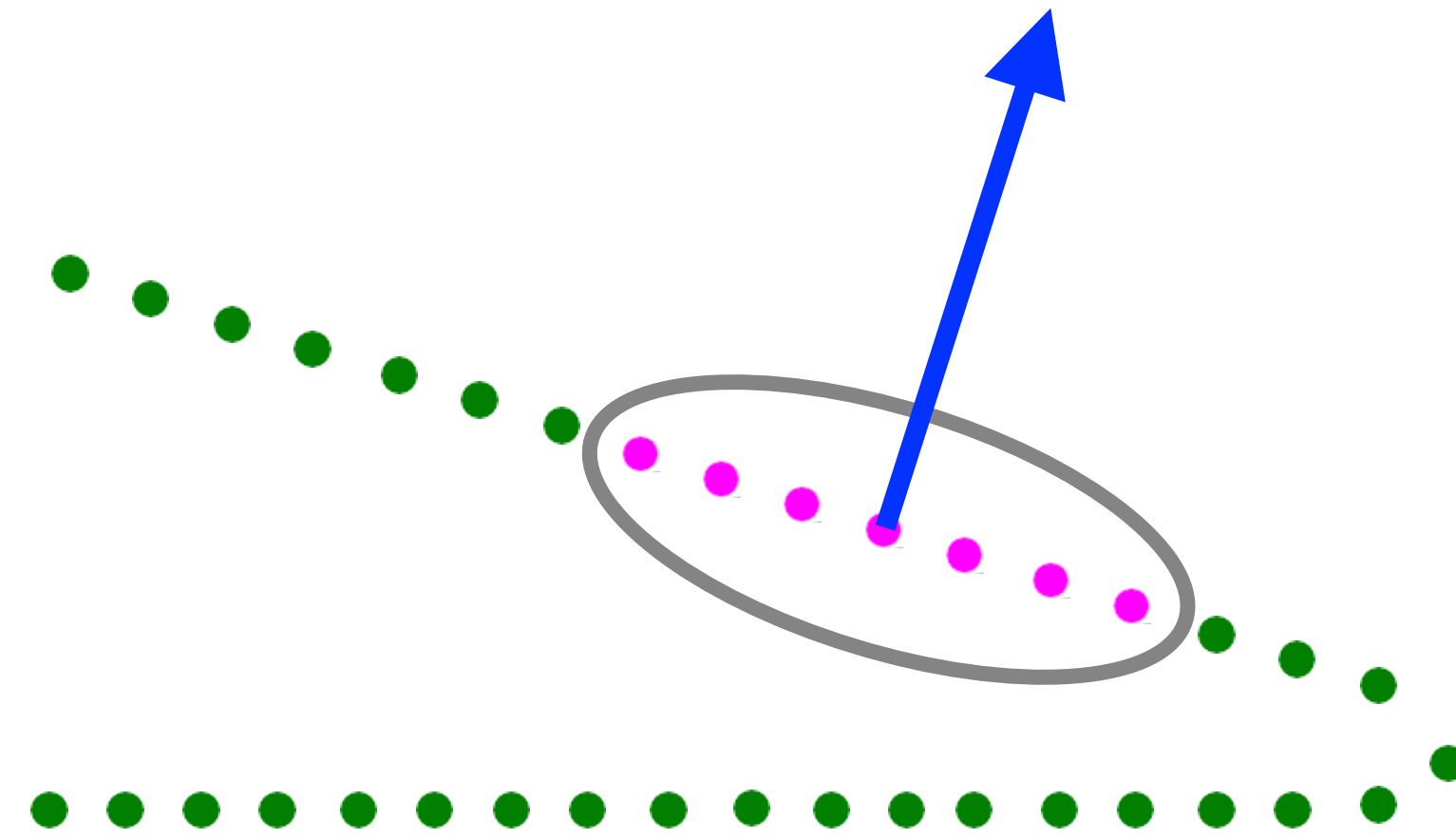
- n : the last principal component of M



Normal Estimation



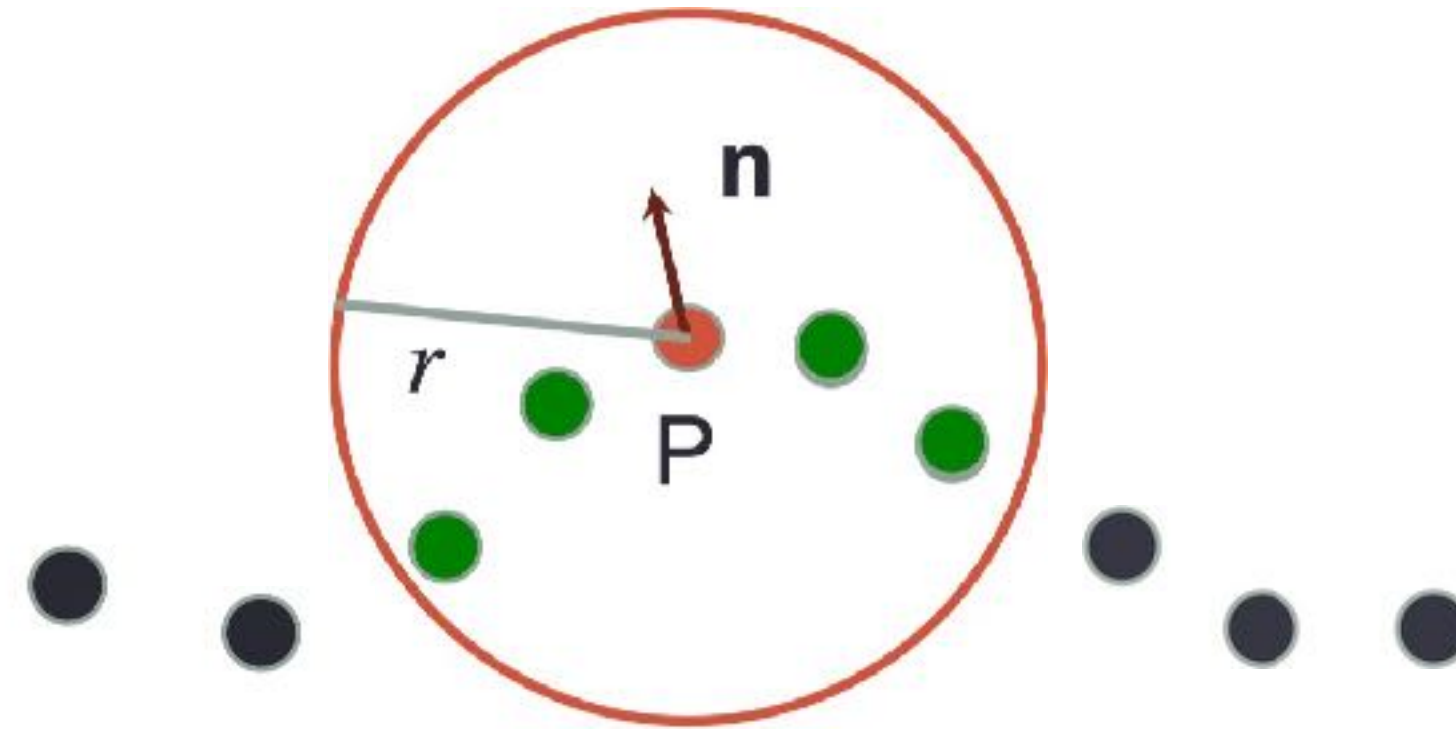
Normal Estimation



$d_G(u, v)$ is large although $d_E(u, v)$ is small!

$\Rightarrow$ Use geodesic neighborhoods for PCA

Normal Estimation

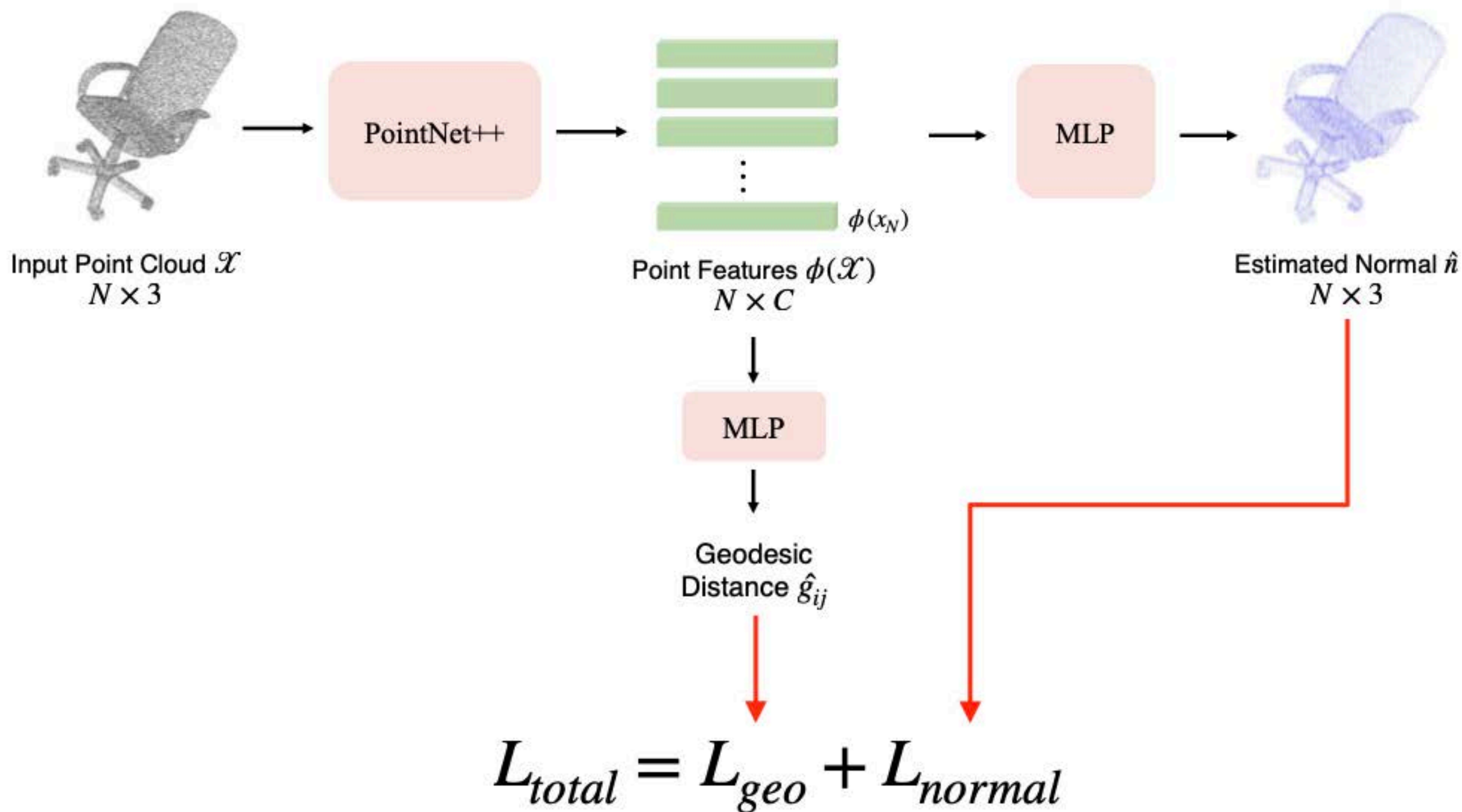


Critical parameter: number of neighbors k .
For unevenly sampled point cloud, we typically use
all points inside a ball of radius r

Tricky to choose in practice!

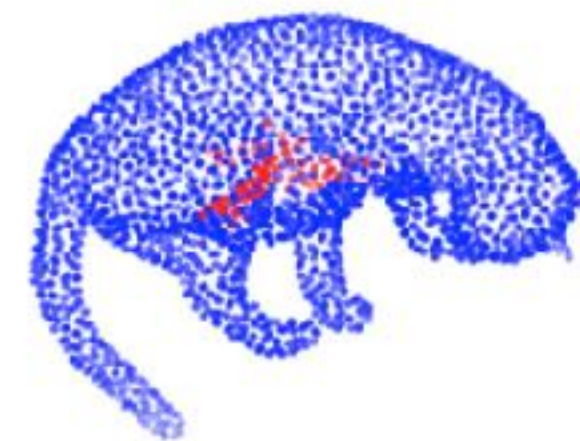
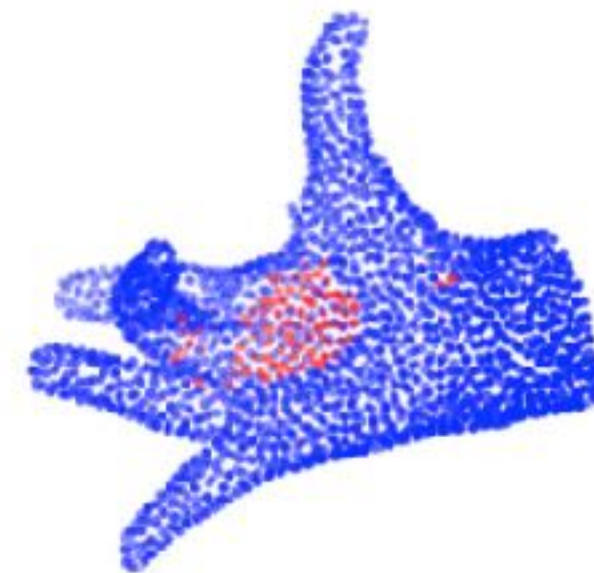
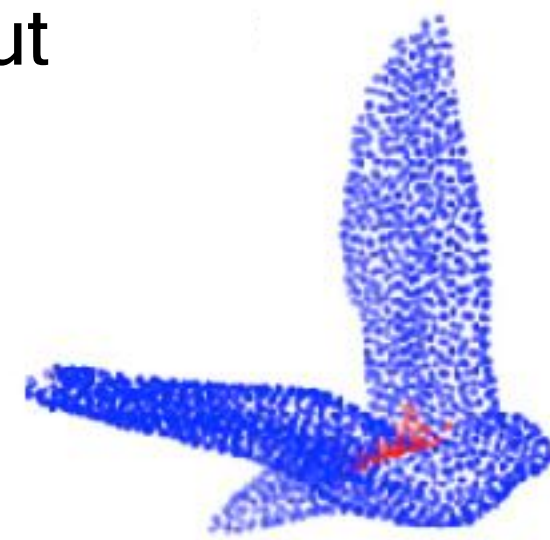
How about **predicting the normal**,
without PCA and estimation of r or k ?

Learning to Regress Normals

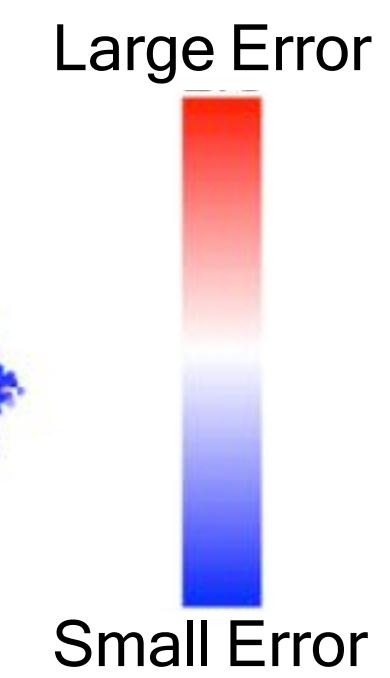


Error Patterns

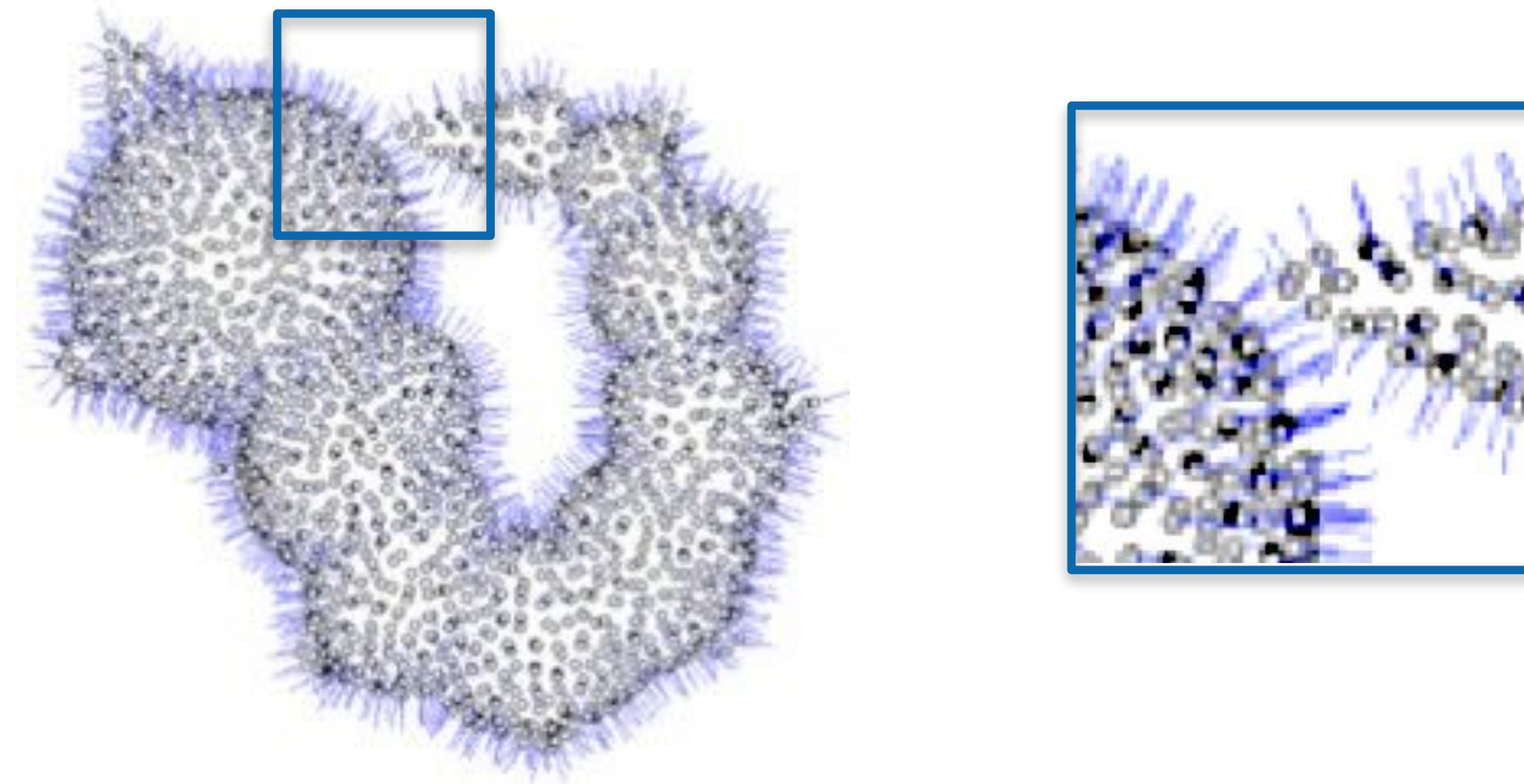
PointNet++ without
GeoNet features



PointNet++ with
GeoNet features



Learning to Regress Normals



Advantage of Learning-based Normal Estimation

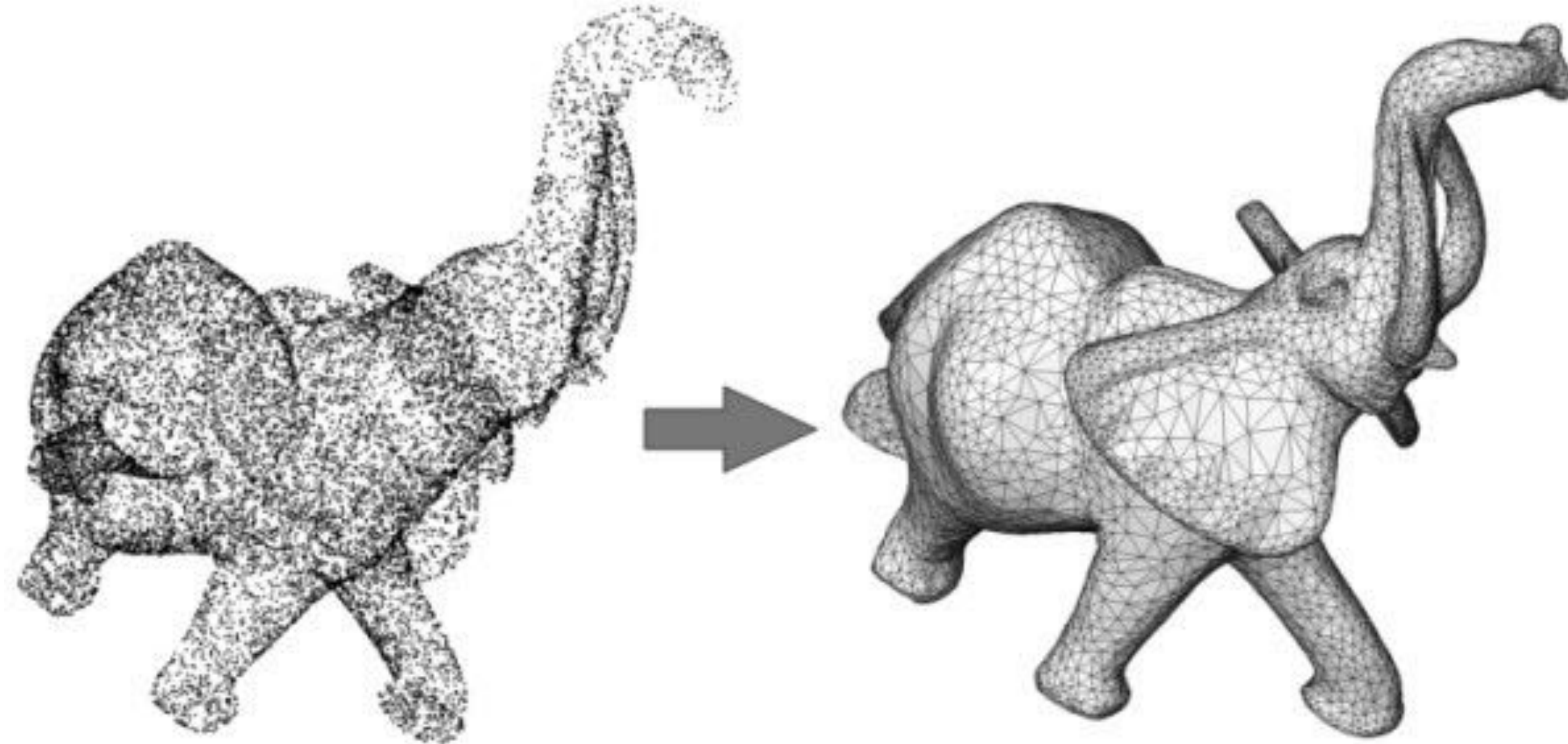
- Can be robust to point cloud sampling strategy
- Without the need to determine neighborhood size
- Get orientation consistent normals (in classical methods, this step is highly non-trivial)

Outline

- Introduction to intrinsic shape analysis
- Analytic Method for Computing Geodesics
- Learning to Predict Geodesics for Point Cloud
- Applications of Learning Point Cloud Geodesics
 - Normal Estimation
 - ***Mesh Reconstruction***

Mesh Reconstruction

- Input: Point Cloud
- Output: Polygon Mesh



A Simple K-NN Approach

- For each point x_i we try to greedily add all triangles formed by x_i and its neighbors:

$\chi = \{x_i\}^n$ is the input point cloud
 $S = \{\}$ is the output mesh triangles

for $i = 1$ to n :

find K nearest neighbors $\{p_l\}_{l=1}^K$ of x_i :

for $j = 1$ to K :

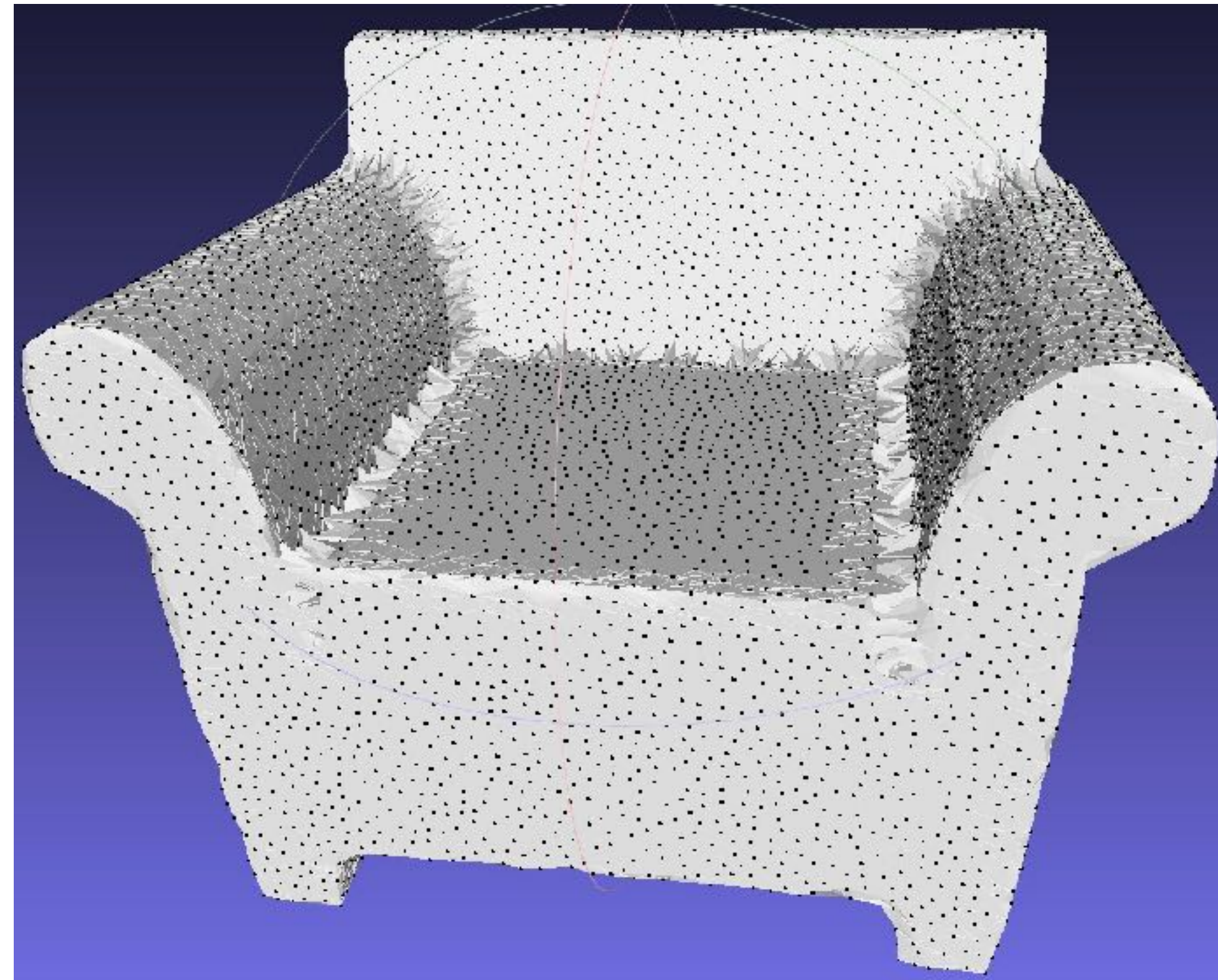
for $k = j + 1$ to K :

triangle $\triangle = (x_i, N_j, N_k)$

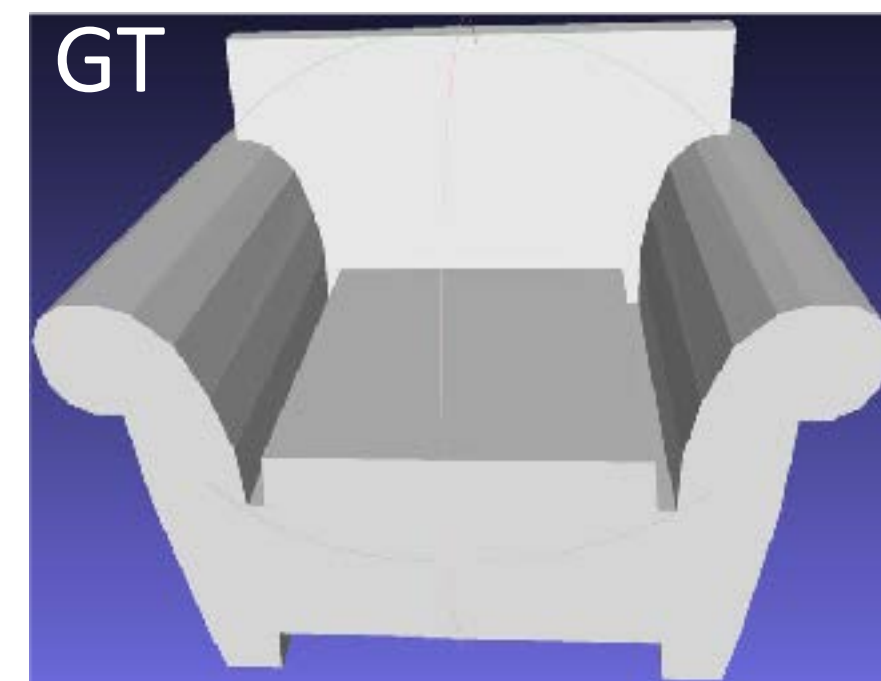
if $\triangle$ is not in S and S is manifold after adding $\triangle$:

add $\triangle$ to S

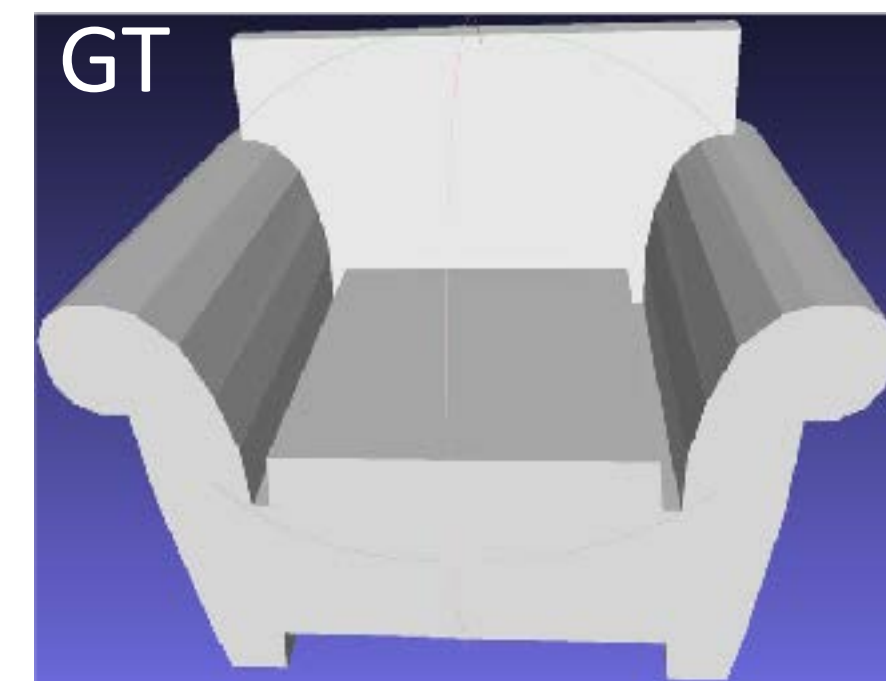
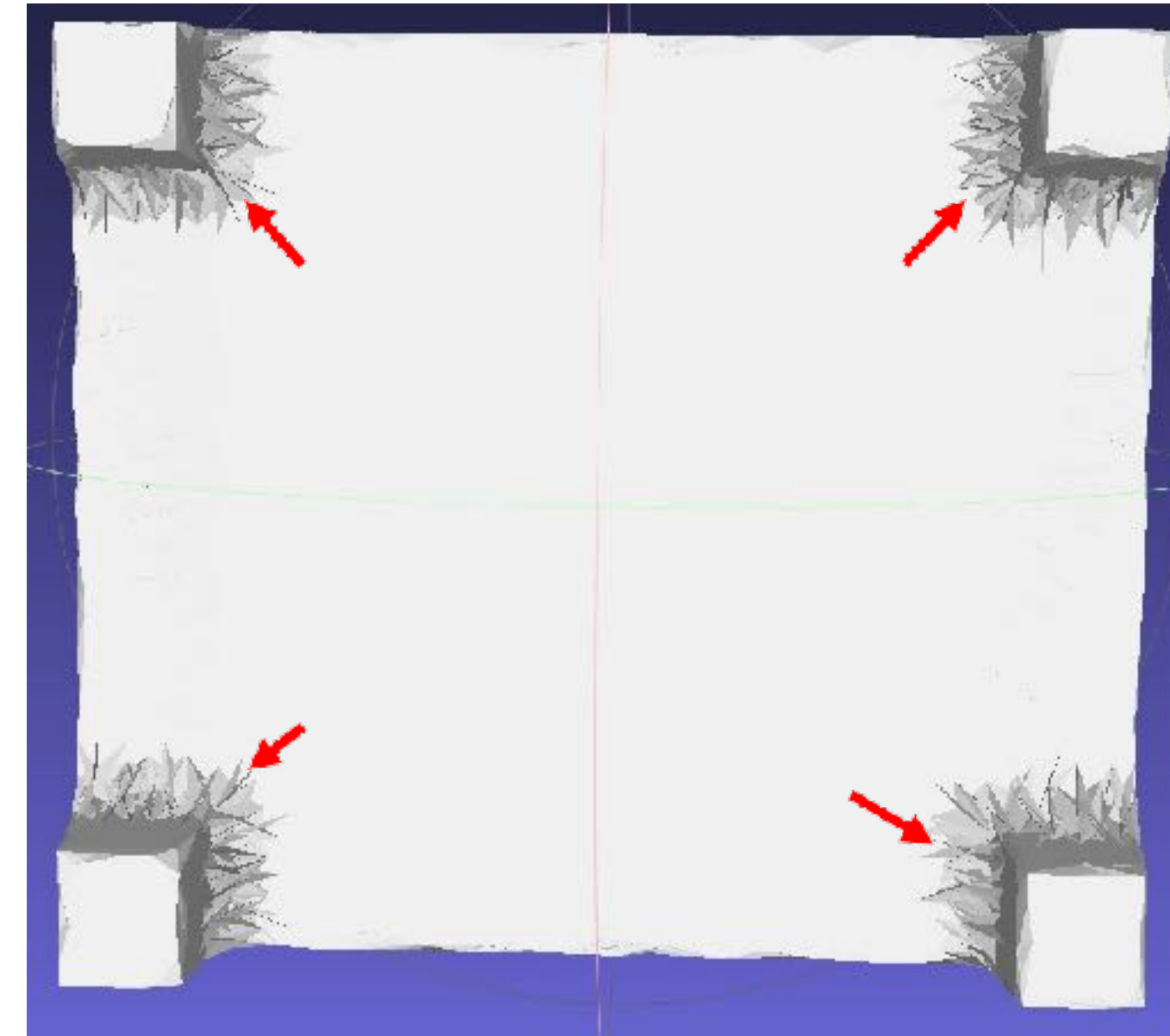
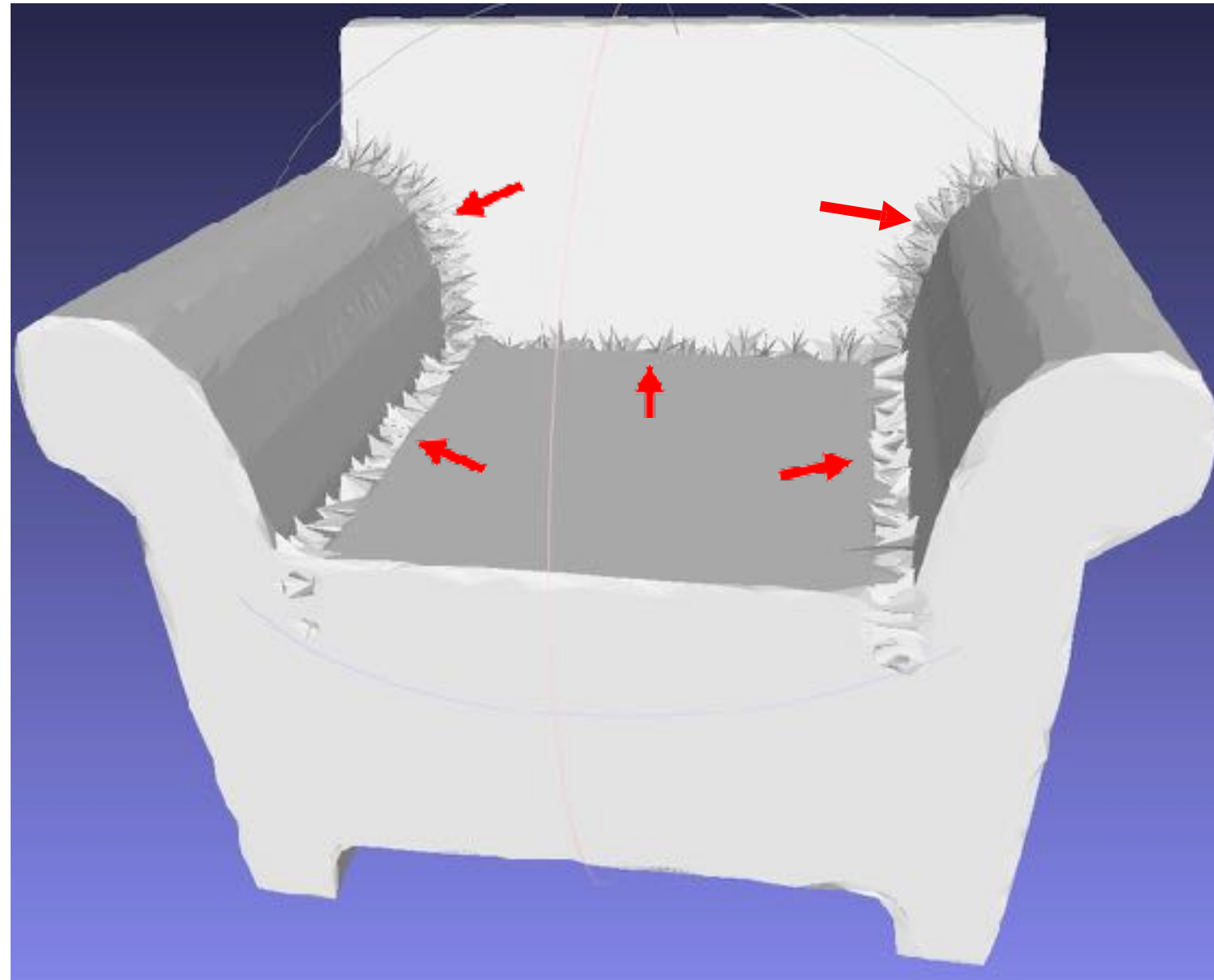
Results not bad...



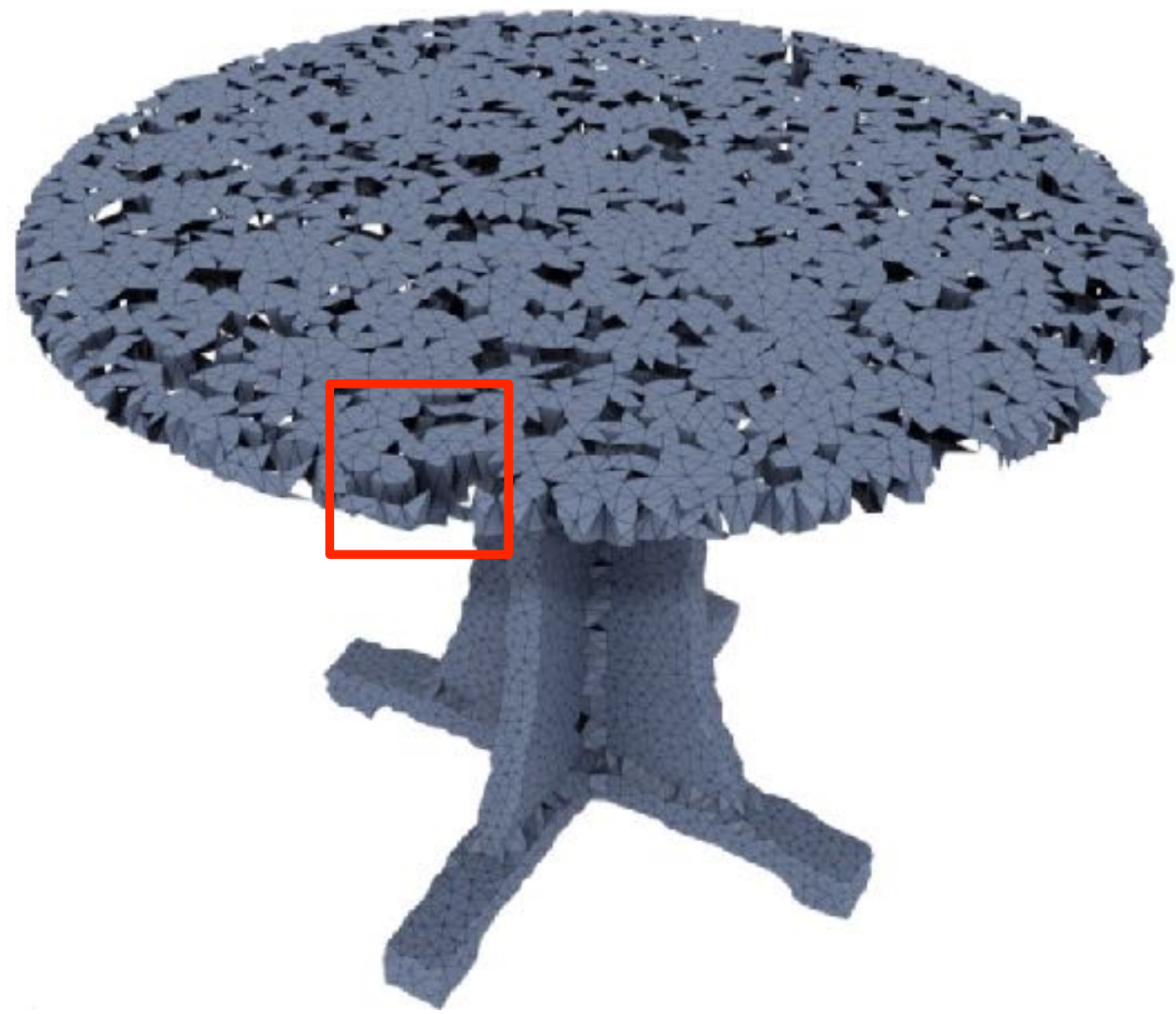
$K = 10$



But has issues at boundaries



And some bad triangles



- Triangles formed by connecting the upper surface and the bottom surface of the tabletop (intrinsically far)

And some bad triangles



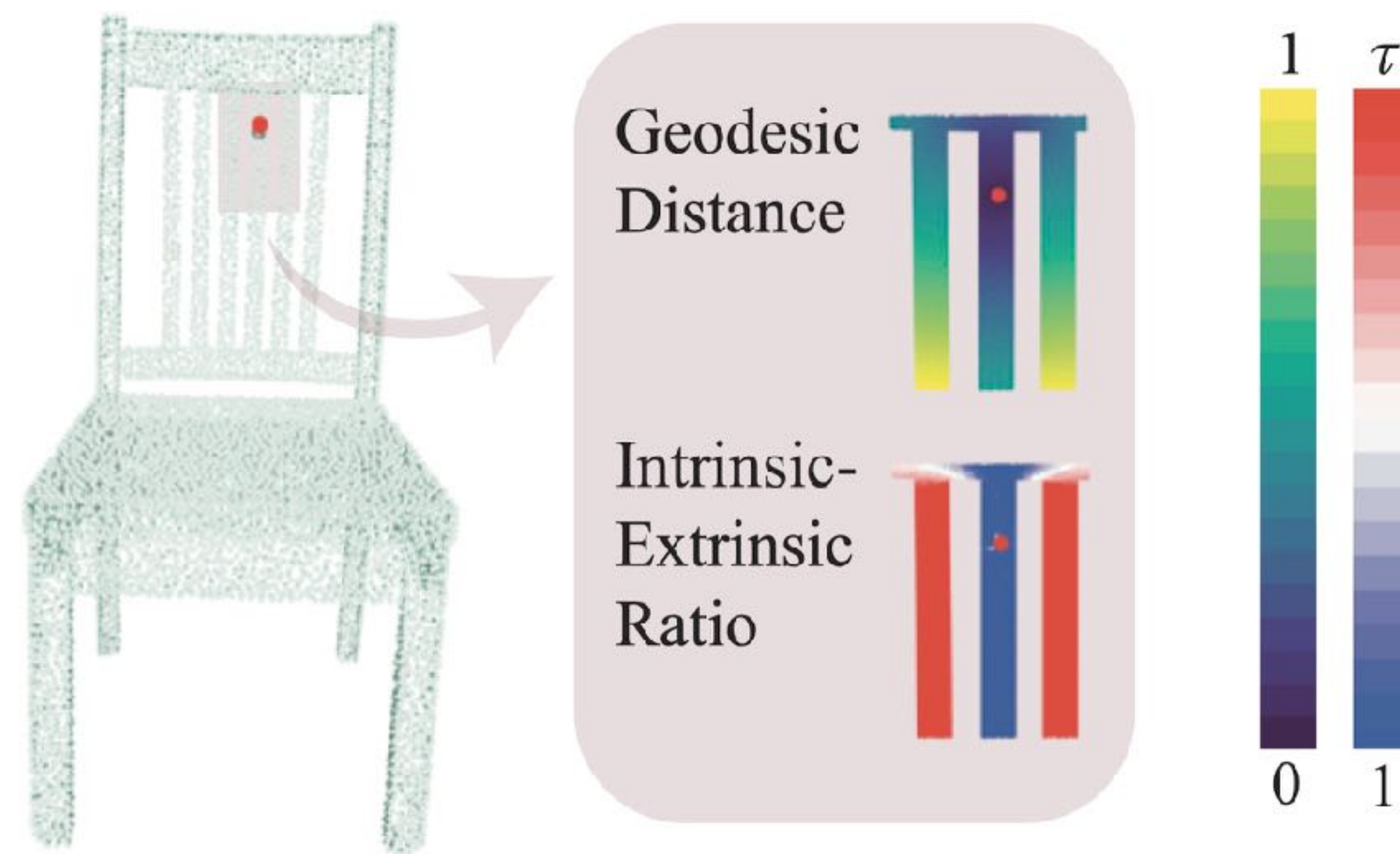
- Triangles formed by connecting the upper surface and the bottom surface of the tabletop (intrinsically far)

Intrinsic-Extrinsic Ratio

Given two vertices u, v on the surface, the intrinsic-extrinsic ratio is defined as:

$$\text{IER}(u, v) = \frac{d_G(u, v)}{d_E(u, v)}$$

Visualization



Intrinsic-Extrinsic Ratio for Triangles

For a triangle $\triangle uvw$:

$$\text{IER}(\triangle uvw) = \frac{d_G(u, v) + d_G(v, w) + d_G(w, u)}{d_E(u, v) + d_E(v, w) + d_E(w, u)}$$

If point u, v, w is **close enough**, $\text{IER}(\triangle uvw) \leq 1 + \varepsilon$
 $\Rightarrow$ (very likely) Triangle is on the surface

Intrinsic-Extrinsic Ratio for Triangles

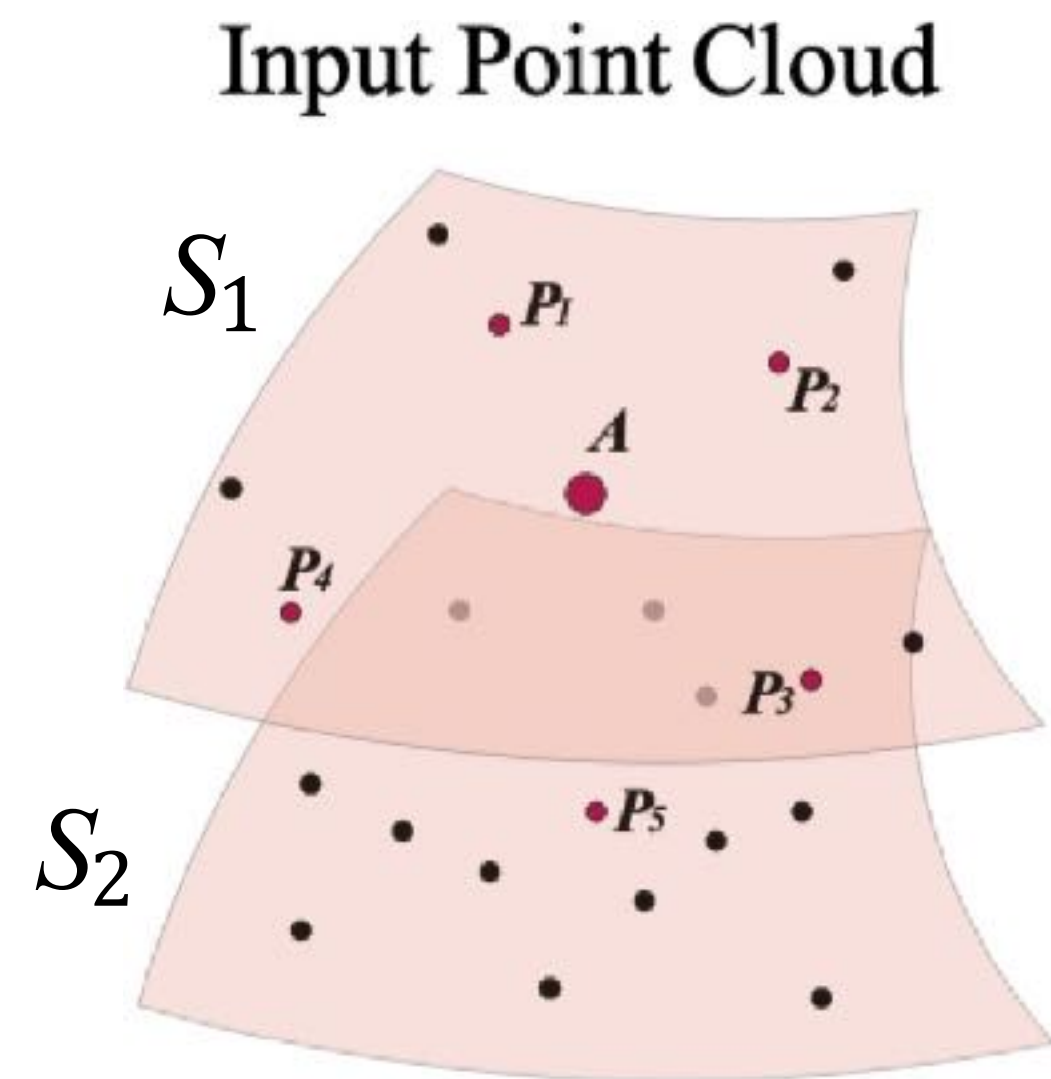
For a triangle $\triangle uvw$:

$$\text{IER}(\triangle uvw) = \frac{d_G(u, v) + d_G(v, w) + d_G(w, u)}{d_E(u, v) + d_E(v, w) + d_E(w, u)}$$

If point u, v, w is **close enough**, $\text{IER}(\triangle uvw) \leq 1 + \varepsilon$
 $\Rightarrow$ (very likely) Triangle is on the surface

We learn to predict this
quantity!

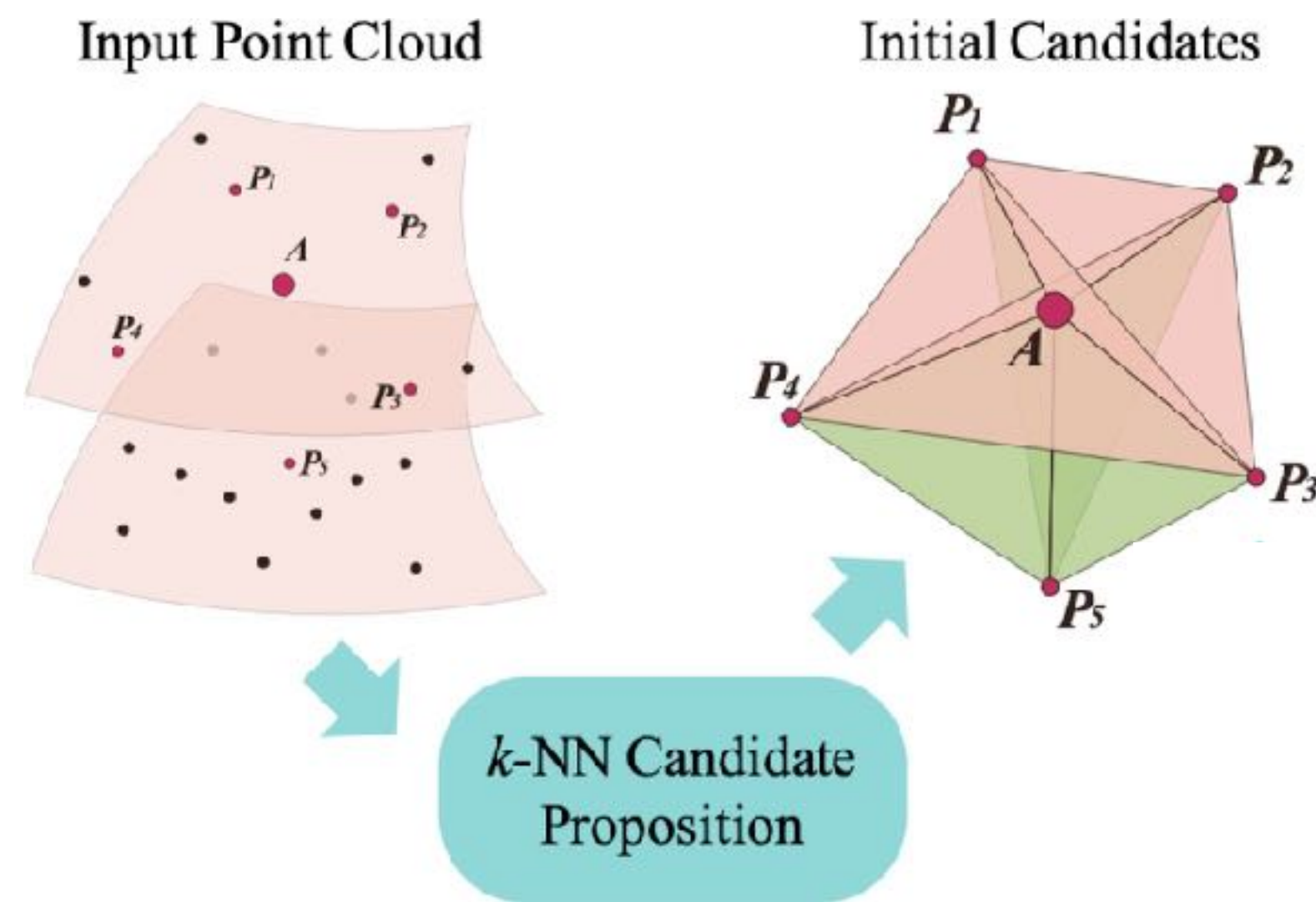
Meshing Point Clouds with Intrinsic-Extrinsic Ratio Guidance



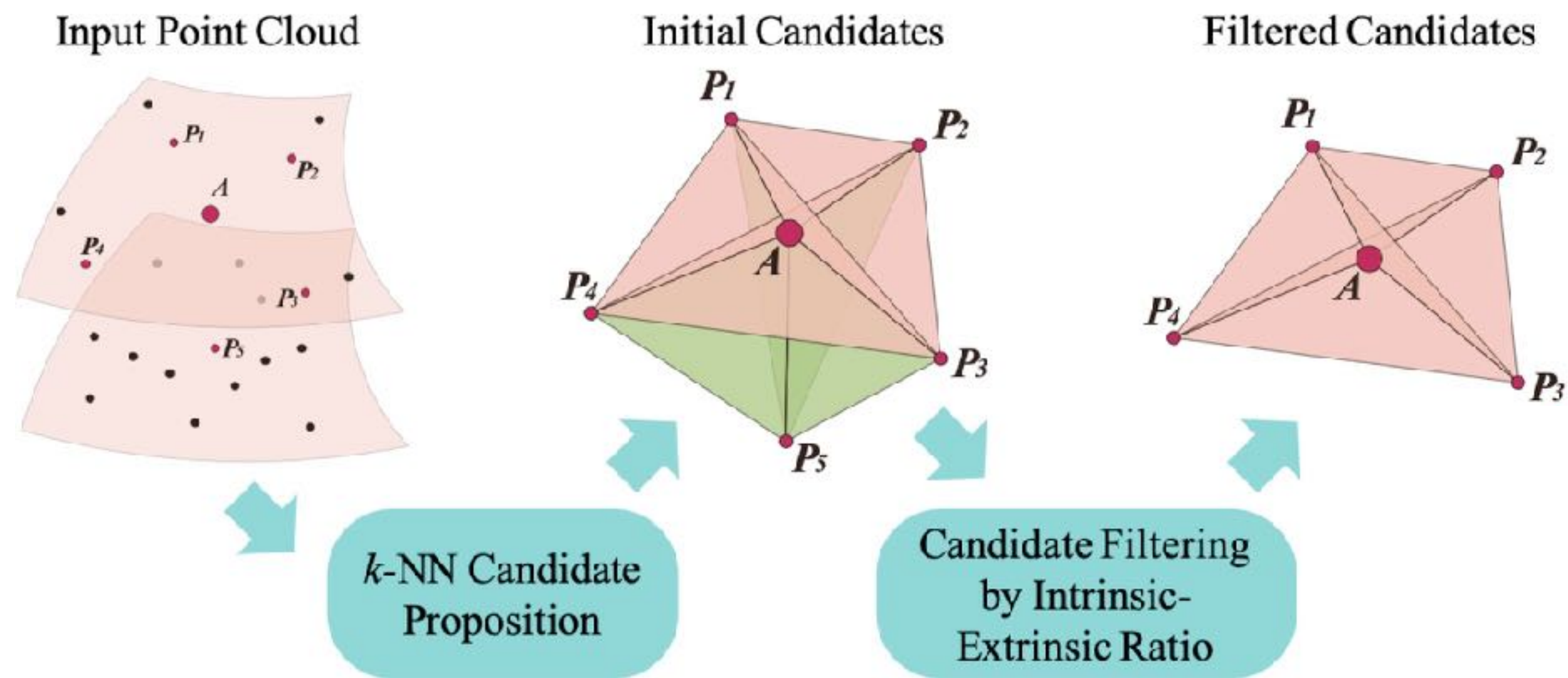
Case Study:

- Input point cloud is sampled from two closed thin surfaces S_1 and S_2
- $\{P_i\}$ are Euclidean nearest neighbors of
 - $\{A, P_1, P_2, P_3, P_4\} \subset S_1$
 - $P_5 \in S_1$

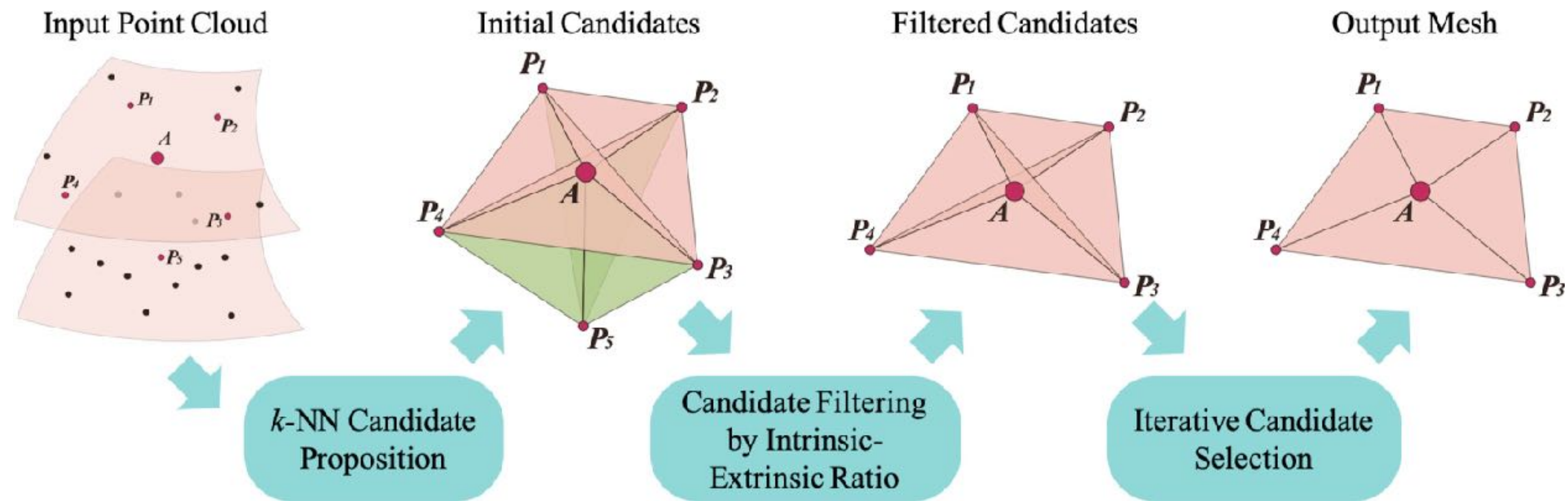
Meshing Point Clouds with Intrinsic-Extrinsic Ratio Guidance



Meshing Point Clouds with Intrinsic-Extrinsic Ratio Guidance

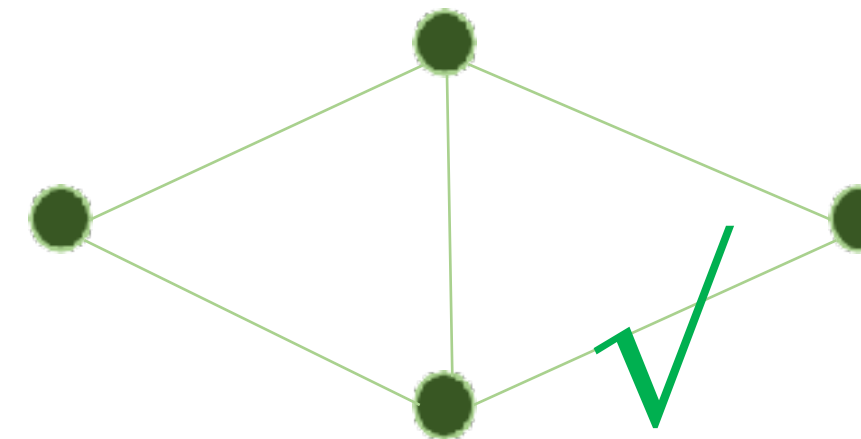
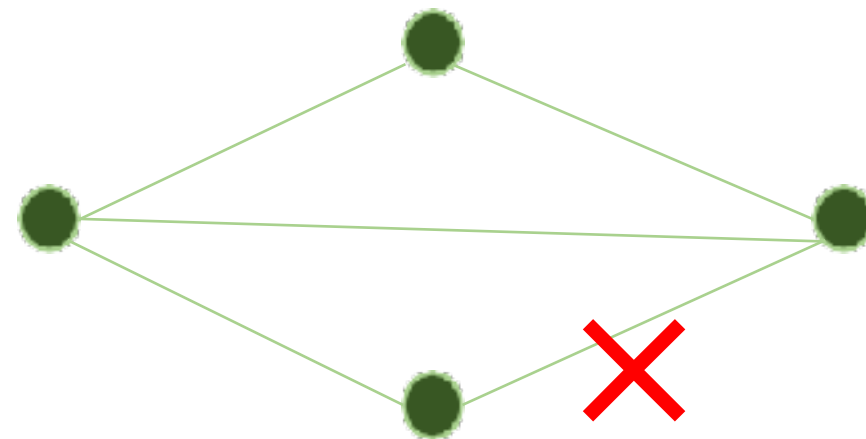


Meshing Point Clouds with Intrinsic-Extrinsic Ratio Guidance



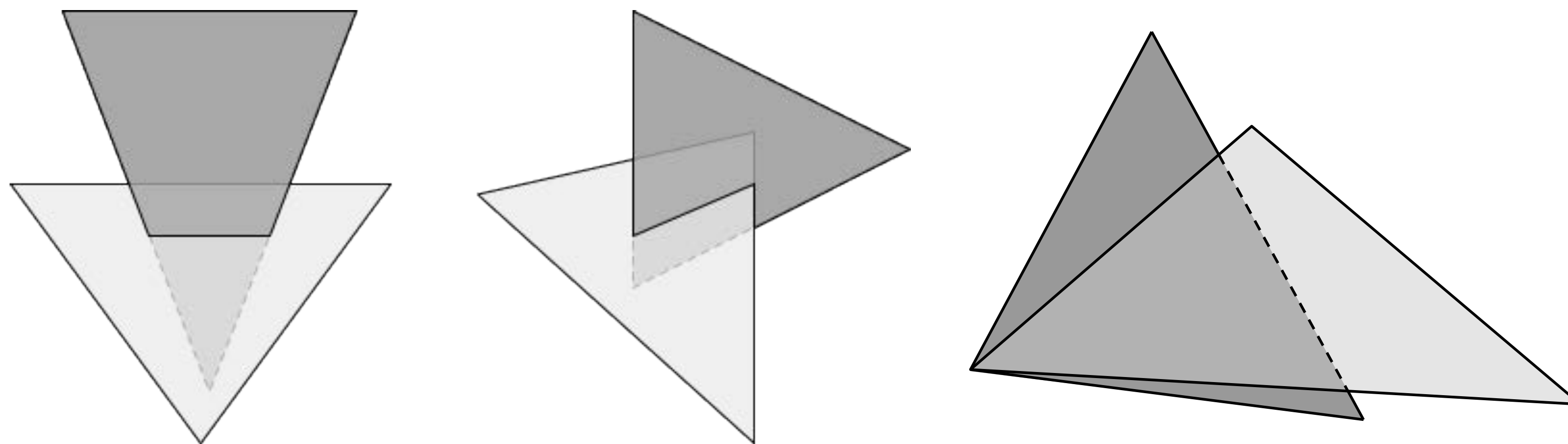
Iterative Candidate Selection

- The added triangle should not break manifold properties
 - No edge has more than two incident faces
 - No intersection between triangles
- Prefer equilateral triangles:



Measured by the ratio of longest edge to shortest edge

Triangle Collision Detection



Guigue P, Devillers O. Fast and robust triangle-triangle overlap test using orientation predicates. Journal of Graphics Tools. 2003 Jan 1;8(1):25-32.

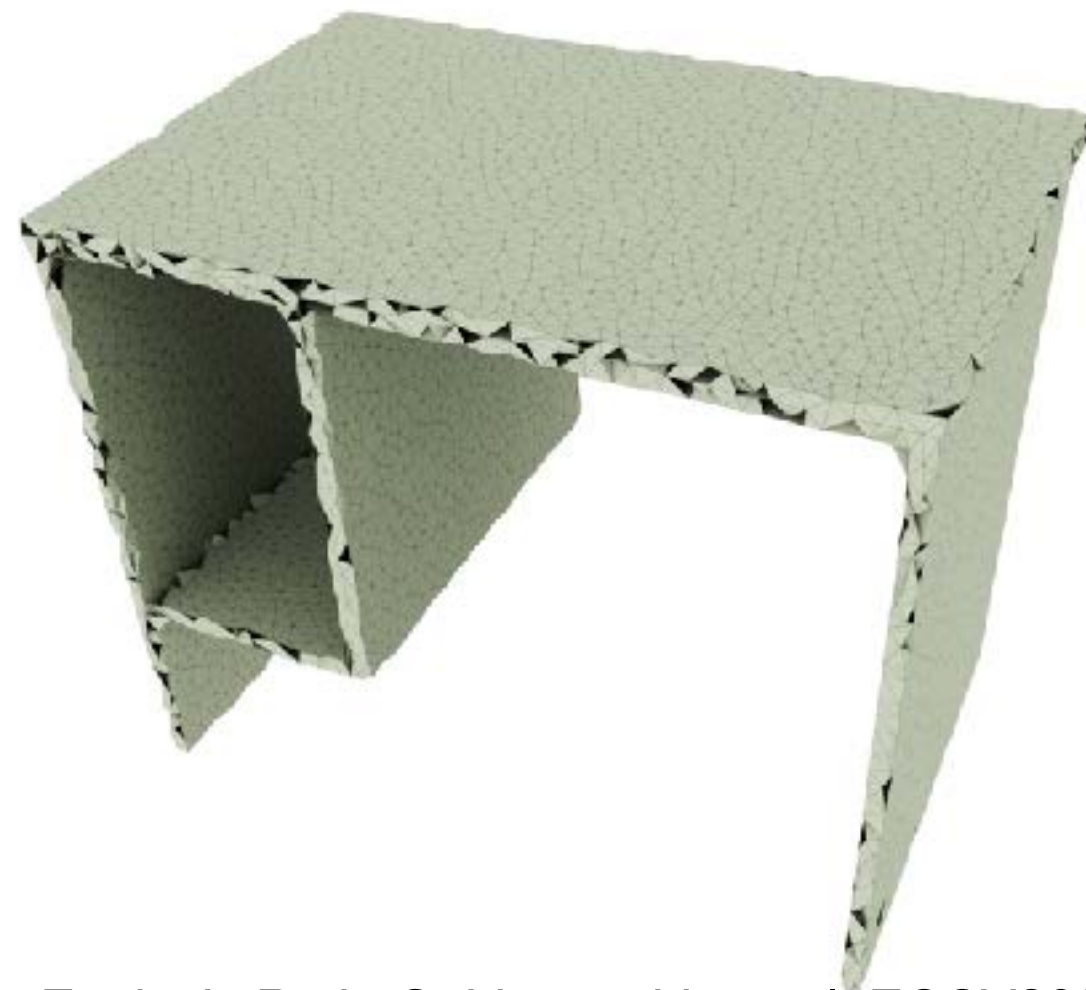
Code: https://github.com/erich666/jgt-code/tree/master/Volume_08/Number_1/Guigue2003

Meshing Results

k -NN

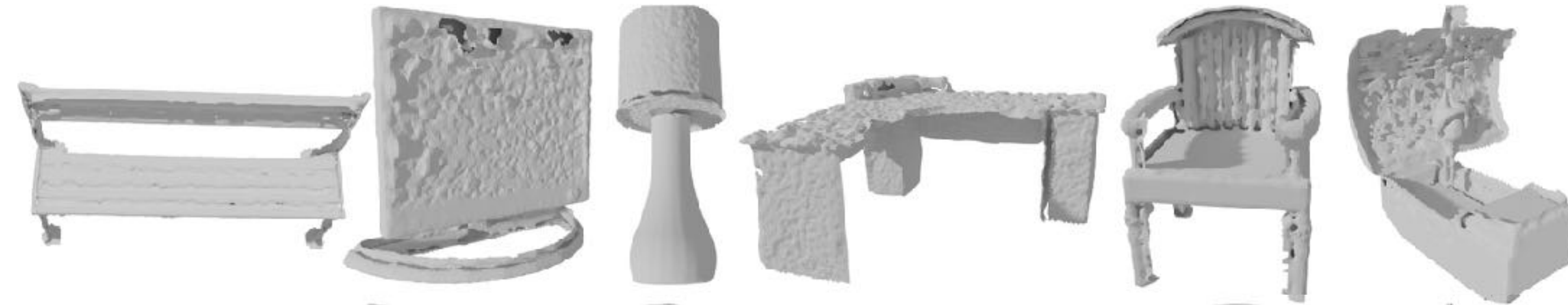


k -NN
with IER

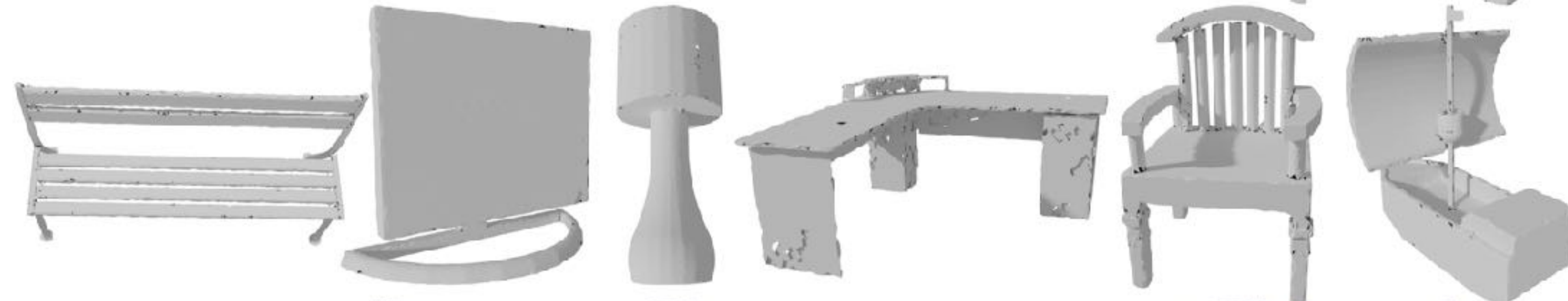


Robustness

Poisson



k -NN
with IER



Ground-
Truth



Summary

- Introduction to intrinsic shape analysis
- Analytic Method for Computing Geodesics
- Learning to Predict Geodesics for Point Cloud
- Applications of Learning Point Cloud Geodesics
 - Normal Estimation
 - Mesh Reconstruction